

EDGE 插件手册

本文档包含 EDGE v3.0 所有插件的分类特性汇总（优先级、Stream 支持、复用性等）以及各插件详细的配置参数说明、调用示例与注意事项。管理接口、安装配置等内容请参阅 [使用手册](#)。

1. 插件的复用功能（别名调用）

为了支持在单个路由中多次执行同一个插件的不同逻辑，网关支持了**插件的复用功能**。

1.1 使用规则

- 可复用的插件（未标注为 `unique` 的插件）可以在一个路由中通过别名的方式多次调用。
- 命名规范：别名使用破折号 `-` 进行连接，格式为：`{原插件名}-{别名}`。每个主插件可以拥有多个不同的别名。
- 声明与注册：别名插件依赖主插件的存在，且必须在 `edge.env` 的 `ex_plugins`（或 `ex_stream_plugins`）配置中进行声明注册。
- 独立属性：所有声明的别名插件都可以像普通插件一样，在路由中配置自己独立的数据属性（Metadata Config / Config）。

1.2 注册示例

在部署配置 `edge.env` 中注册别名插件：

```
ex_plugins:
  plugin_demo-test1: true      # 启用别名插件 1，使用默认优先级
  plugin_demo-test2: 1999     # 启用别名插件 2，并自定义其执行优先级为 1999
```

 **Warning**

凡是标注为 `unique` 的插件（即不可复用的插件，如 `cors`、`proxy_rewrite` 等），**不支持**使用别名进行多次实例化配置。

2. 插件分类与特性汇总

2.1 核心插件

插件名称	优先级 (Priority)	模式 (Stream)	复用性 (Unique)	简要说明	使用文档索引
traceid	12300	stream	unique	自动生成并向请求中插入唯一的追踪 ID，便于日志追踪与排查。	traceid
tracing	12000	-	-	在请求或响应头中注入 Trace 信息，与链路追踪系统配合。	tracing
redirect	5900	-	unique	提供在网关层执行 HTTP 301/302 重定向功能。	redirect
monitor	5500	stream	unique	统计并上报请求量、延迟、流量大小等可观测性指标。	monitor
mocking	-20000	-	-	在网关层直接构造并返回模拟的响应，不请求上游后端。	mocking

2.2 认证相关插件

插件名称	优先级 (Priority)	模式 (Stream)	复用性 (Unique)	简要说明	使用文档索引
auth_basic	7520	-	-	提供标准的 HTTP Basic 访问鉴权。	auth_basic
auth_key	7500	-	-	校验请求参数或 Header 中的 API Key 以进行访问鉴权。	auth_key

2.3 代理相关插件

插件名称	优先级 (Priority)	模式 (Stream)	复用性 (Unique)	简要说明	使用文档索引
proxy_mirror	6510	-	unique	将请求异步镜像发送到指定的其它后端服务。	proxy_mirror
proxy_cache	6509	-	unique	提供缓存后端响应内容的功能，支持磁盘及内存缓存。	proxy_cache
proxy_rewrite	6508	-	unique	对发往上游服务的请求 URI、头部、上游目标等执行改写。	proxy_rewrite

2.4 响应相关插件

插件名称	优先级 (Priority)	模式 (Stream)	复用性 (Unique)	简要说明	使用文档索引
response_rewrite	5899	-	-	动态修改网关返回给客户端的 HTTP 状态码、响应头或响应体。	response_rewrite

2.5 日志相关插件

插件名称	优先级 (Priority)	模式 (Stream)	复用性 (Unique)	简要说明	使用文档索引
log_process	5200	stream	unique	将访问日志记录在当前网关服务器的本地磁盘中。	log_process
log_kafka	5180	-	-	将访问日志以批量、非阻塞的方式上报给 Kafka 服务端。	log_kafka
log_http	5170	-	-	将访问日志通过 HTTP POST 请求批量发送到指定的接收服务器。	log_http
log_syslog	5170	stream	-	将访问日志通过 Syslog 协议异步投递到集中化日志收集端。	log_syslog
log_mirror	5160	-	unique	将请求的流量和全部元数据异步镜像发送至指定接收端。	log_mirror
log_rotate	5100	-	unique	负责本地运行日志的按时间/按大小自动切分与清理。	log_rotate

2.6 安全检查插件

插件名称	优先级 (Priority)	模式 (Stream)	复用性 (Unique)	简要说明	使用文档索引
security_common_uri	8809	-	-	对请求地址 URI 进行安全扫描防护与规范化检测。	security_common_uri
security_common_args	8808	-	-	对 URL 查询参数进行 WAF 常规安全风险检查。	security_common_args
security_common_headers	8807	-	-	对客户端请求头字段进行 WAF 安全特征库扫描。	security_common_headers
security_common_cookie	8806	-	-	对客户端 Cookie 内容实施常规 SQL 注入与跨站扫描。	security_common_cookie
security_common_referer	8805	-	-	校验 Referer 头部是否合规，用于防范 CSRF 与盗链。	security_common_referer
security_common_useragent	8804	-	-	扫描 User-Agent，拦截已知的恶意扫描器与爬虫。	security_common_useragent
security_common_body	8802	-	-	对请求 Body（如 POST 报文）实施 WAF 特征扫描防护。	security_common_body
security_corerule	8800	-	-	执行 OWASP CRS 核心防护规则进行拦截阻断。	security_corerule
security_super_ip	8700	-	-	基于 IP 白名单机制跳过网关层的所有安全检查插件。	security_super_ip
security_super_user	8600	-	-	基于用户标识白名单机制跳过网关层的所有安全检查。	security_super_user
security_super_expr	8550	-	-	根据判定表达式结果判定是否豁免后续的安全阻断。	security_super_expr
security_restrict_uri	8400	-	-	针对特定请求路径设定黑白名单卡控拦截策略。	security_restrict_uri
security_restrict_ip	8300	-	-	对指定的 IP 或 CIDR 网段执行黑白名单卡控拦截。	security_restrict_ip
security_restrict_user	8200	-	-	对指定的客户端用户标识设定黑白名单卡控拦截。	security_restrict_user
security_restrict_expr	8150	-	-	使用自定义逻辑表达式过滤并阻断风险请求。	security_restrict_expr
security_super_redis	7350	-	-	查询 Redis 结果判断是否豁免安全检查。	security_super_redis
security_restrict_redis	7300	-	-	根据 Redis 实时查询结果和风险判定表达式阻断请求。	security_restrict_redis

2.7 流量控制插件

插件名称	优先级 (Priority)	模式 (Stream)	复用性 (Unique)	简要说明	使用文档索引
traffic_limit_count	6200	-	-	对特定维度在指定时间窗口内的总请求次数实施阈值限速。	traffic_limit_count

2.8 其它插件

插件名称	优先级 (Priority)	模式 (Stream)	复用性 (Unique)	简要说明	使用文档索引
pre_functions	9999	stream	-	在网关核心执行阶段之前插入运行自定义 Lua 代码。	pre_functions
real_ip	9500	-	-	对经过代理链传输的客户端源 IP 实施解析修正。	real_ip
cors	9000	-	unique	网关层自动响应 OPTIONS 并注入 CORS 跨域规则响应头。	cors
remote_redis	7900	-	-	读取外部 Redis 的数据并映射为网关层 request 上下文变量。	remote_redis
plugin_demo	7500	-	-	EDGE 插件开发骨架和规范实现的范例演示插件。	plugin_demo
server_info	5990	-	-	向响应头部注入当前网关节点名称和版本标识。	server_info
dns_upstream	2100	stream	-	提供基于 Balancer 机制的智能 DNS 域名解析及后端调度。	dns_upstream
dns_server	2000	stream	-	支持在网关 Stream 模式下提供 DNS 协议解析和转发。	dns_server
post_functions	-1999	stream	-	在网关核心执行阶段之后插入运行自定义 Lua 代码。	post_functions
breaker	-22000	-	-	实现面向后端 upstream 服务节点的健康检测与熔断降级。	breaker

3. 各插件详细使用说明

- [auth_basic 插件使用文档](#)
- [auth_key 插件使用文档](#)
- [breaker 插件使用文档](#)

- [cors 插件使用文档](#)
- [dns_server 插件使用说明书](#)
- [dns_upstream 插件使用说明书](#)
- [log_http 插件使用文档](#)
- [log_kafka 插件使用文档](#)
- [log_mirror 插件使用文档](#)
- [log_process 使用文档](#)
- [log_rotate 使用文档](#)
- [log_syslog 插件使用文档](#)
- [mocking 使用文档](#)
- [monitor 使用文档](#)
- [plugin_demo 使用文档](#)
- [post_functions 插件使用文档](#)
- [pre_functions 插件使用文档](#)
- [proxy_cache 插件使用文档](#)
- [proxy_mirror 插件使用文档](#)
- [proxy_rewrite 使用文档](#)
- [real_ip 插件使用说明书](#)
- [redirect 插件使用文档](#)
- [remote_redis 插件使用文档](#)
- [response_rewrite](#)
- [security_common_args 插件使用文档](#)
- [security_common_body 插件使用文档](#)
- [security_common_cookie 插件使用文档](#)
- [security_common_headers 插件使用文档](#)
- [security_common_referer 插件使用文档](#)
- [security_common_uri 插件使用文档](#)
- [security_common_useragent 插件使用文档](#)
- [security_corerule](#)
- [security_restrict_expr 插件使用文档](#)
- [security_restrict_ip 插件使用文档](#)
- [security_restrict_redis 插件使用文档](#)
- [security_restrict_uri 插件使用文档](#)
- [security_restrict_user 插件使用文档](#)
- [security_super_expr 插件使用文档](#)
- [security_super_ip 插件使用文档](#)
- [security_super_redis 插件使用文档](#)
- [security_super_user 插件使用文档](#)

- [server_info 插件使用文档](#)
- [traceid 插件使用说明书](#)
- [tracing 插件使用文档](#)
- [traffic_limit_count 使用文档](#)

auth_basic 插件使用文档

1. 插件概览

`auth_basic` 插件用于在网关层实现 HTTP 基础认证 (HTTP Basic Authentication)。它通过验证客户端在请求头中携带的 `Authorization` 信息，与网关中配置的**消费者 (Consumers)** 账号密码进行匹配，拦截未授权的访问。

主要应用场景：

- 服务访问受限防护**：保护敏感的后台管理页面、测试环境或仅限注册用户/合作伙伴调用的 API 接口。
- 简易多租户认证**：根据不同的客户端身份创建多个不同的 Consumer，实行差异化的访问授权与流量监控。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

无。

2.3 插件基础属性 (Config)

路由级配置。

字段名	类型	默认值	描述	设置示例
<code>hide_credentials</code>	Boolean	<code>false</code>	认证成功后，是否将请求转发给上游前从 HTTP Header 中剥离删除 <code>Authorization</code> 认证标头。	<code>true</code>

2.4 消费者属性 (Consumer Config)

在创建或更新消费者资源（API 路径：`/edge/admin/consumers`）时配置该用户的 Basic 认证凭证。

字段名	类型	默认值	描述	设置示例
<code>username</code>	String	无	必填。消费者 Basic 认证对应的用户名，建议与消费者的系统唯一 <code>username</code> 保持一致。	<code>"test_user"</code>
<code>password</code>	String	无	必填。消费者 Basic 认证对应的明文密码。	<code>"super_secure_pass"</code>

3. 调用与验证示例

要完整启用并验证 `auth_basic` 插件，共需配置两步：创建对应的消费者凭证，然后配置启用插件的路由。

Important

调用示例中不包含真实敏感密钥，请将占位符 `API_KEY` 替换为您实际环境中的 Admin API Key。

3.1 步骤 1：创建具有 Basic 凭证的消费者 (Consumer)

调用消费者接口，创建一个系统唯一名为 `company_a` 的消费者，并配置其 Basic 认证的用户名和密码。

```
curl -i -X PUT \
-H "Content-Type: application/json" \
-H "X-API-Key: API_KEY" \
-d '{
  "username": "company_a",
  "name": "Company A Consumer",
  "desc": "Consumer for B2B Client A",
  "plugins": {
    "auth_basic": {
      "username": "client_a",
      "password": "CLIENT_A_PASSWORD"
    }
  }
}' \
http://127.0.0.1:9991/edge/admin/consumers
```

3.2 步骤 2：创建启用 auth_basic 插件的路由

创建路由由关联路径 `/api/secure`，并开启 `auth_basic` 插件，配置为隐藏敏感认证信息 (`hide_credentials: true`)，避免凭证透传至后端。

```
curl -i -X PUT \
-H "Content-Type: application/json" \
-H "X-API-Key: API_KEY" \
-d '{
  "uri": "/api/secure",
  "plugins": {
    "auth_basic": {
      "hide_credentials": true
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:8811": 1
    },
    "type": "roundrobin"
  }
}' \
http://127.0.0.1:9991/edge/admin/routes/1004
```

3.3 访问网关进行验证

1. 未携带认证信息的匿名访问（预期被拦截）

向网关发起匿名请求：

```
curl -i http://127.0.0.1:9981/api/secure
```

预期响应输出 (返回 401 并附带 WWW-Authenticate 头):

```
HTTP/1.1 401 Unauthorized
Connection: keep-alive
WWW-Authenticate: Basic realm='.'
Content-Length: 42
Content-Type: application/json

{"message": "Missing authorization in request"}
```

2. 使用错误的凭证访问 (预期被拒绝)

模拟输入错误的密码:

```
curl -i -u "client_a:WRONG_PASSWORD" http://127.0.0.1:9981/api/secure
```

预期响应输出:

```
HTTP/1.1 401 Unauthorized
Connection: keep-alive
WWW-Authenticate: Basic realm='.'
Content-Length: 35
Content-Type: application/json

{"message": "Invalid user authorization"}
```

3. 使用正确的凭证访问 (预期通过)

使用创建消费者时配置的正确用户名和密码发起请求:

```
curl -i -u "client_a:CLIENT_A_PASSWORD" http://127.0.0.1:9981/api/secure
```

预期响应输出:

```
HTTP/1.1 200 OK
Content-Type: text/plain
Connection: keep-alive
...
```

(注意: 由于设置了 `hide_credentials: true`, 后端上游服务器收到该请求时, 其 Header 中将不包含 `Authorization` 头)

auth_key 插件使用文档

1. 插件概览

`auth_key` 插件通过识别请求中指定的 API Key (密钥) 进行身份验证。它支持从指定的请求头 (Header) 或 URL 查询参数 (Query Parameter) 中获取密钥, 并与绑定的消费者 (**Consumers**) 进行匹配, 起到接口调用鉴权及访问审计的作用。

主要应用场景:

- **开放 API 服务接入**：对外提供数据订阅或计算接口时，为合作方发放唯一的 API Key，用以控制访问权限和监控流量消耗。
- **系统集成认证**：简化内部微服务与网关之间、或者多套企业应用系统集成时的安全调用手续。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

无。

2.3 插件基础属性 (Config)

路由级配置。

字段名	类型	默认值	描述	设置示例
header	String	"apikey"	指定从哪一个 HTTP 请求头中提取 API Key。不区分大小写。	"X-API-KEY"
query	String	"apikey"	指定从哪一个 URL 查询参数中提取 API Key。	"api_key"
hide_credentials	Boolean	false	认证通过后，是否从转发给上游的请求中清除携带的 API Key 敏感信息（包含指定的 Header 和 Query 参数）。	true

2.4 消费者属性 (Consumer Config)

在创建或更新消费者资源（API 路径：`/edge/admin/consumers`）时配置该用户的 API Key 凭证。

字段名	类型	默认值	描述	设置示例
key	String	无	必填。该消费者独有的 API Key 字符串。建议使用高强度的随机字符串。	"abcdef1234567890"

3. 调用与验证示例

启用 `auth_key` 插件需要配合消费者凭证一同配置，共分为两步：配置消费者，再配置路由规则。

 **Important**

调用示例中不包含真实敏感密钥，请将占位符 `API_KEY` 替换为您实际环境中的 Admin API Key。

3.1 步骤 1：创建具有 API Key 的消费者 (Consumer)

调用消费者接口，创建一个名为 `company_b` 的消费者，并配置其拥有的唯一 API Key。

```
curl -i -X PUT \  
  -H "Content-Type: application/json" \  
  -H "X-API-Key: API_KEY" \  
  http://127.0.0.1:8080/edge/admin/consumers/company_b
```



```
-d '{
  "username": "company_b",
  "name": "Company B Consumer",
  "desc": "Consumer for SaaS Client B",
  "plugins": {
    "auth_key": {
      "key": "CLIENT_B_API_KEY"
    }
  }
}' \
http://127.0.0.1:9991/edge/admin/consumers
```

3.2 步骤 2：创建启用 auth_key 插件的路由

创建路由关联路径 `/api/data`，开启 `auth_key` 插件，将其配置为：从请求头 `X-API-KEY` 或查询参数 `api_key` 中读取密钥，并且在验证通过后剥离该密钥（`hide_credentials: true`）。

```
curl -i -X PUT \
-H "Content-Type: application/json" \
-H "X-API-Key: API_KEY" \
-d '{
  "uri": "/api/data",
  "plugins": {
    "auth_key": {
      "header": "X-API-KEY",
      "query": "api_key",
      "hide_credentials": true
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:8811": 1
    },
    "type": "roundrobin"
  }
}' \
http://127.0.0.1:9991/edge/admin/routes/1005
```

3.3 访问网关进行验证

1. 未携带任何密钥的访问（预期被拦截）

直接访问网关地址：

```
curl -i http://127.0.0.1:9981/api/data
```

预期响应输出 (返回 401)：

```
HTTP/1.1 401 Unauthorized
Connection: keep-alive
Content-Length: 38
Content-Type: application/json

{"message": "Missing API key in request"}
```

2. 使用错误的密钥访问（预期被拒绝）

无论是 Header 还是 Query 传入了错误的密钥：

```
curl -i -H "X-API-KEY: WRONG_KEY" http://127.0.0.1:9981/api/data
```

预期响应输出 (返回 401)：

```
HTTP/1.1 401 Unauthorized
Connection: keep-alive
Content-Length: 38
Content-Type: application/json

{"message": "Invalid API key in request"}
```

3. 使用正确的 Header 密钥访问（预期通过）

在 `X-API-KEY` 请求头中传入正确的密钥：

```
curl -i -H "X-API-KEY: CLIENT_B_API_KEY" http://127.0.0.1:9981/api/data
```

预期响应输出：

```
HTTP/1.1 200 OK
Content-Type: text/plain
Connection: keep-alive
...
```

(注意：由于设置了 `hide_credentials: true`，后端上游服务器接收到的请求 Header 中将不会包含 `X-API-KEY` 标头)

4. 使用正确的 Query 密钥访问（预期通过）

在 URL 查询参数中传入正确的密钥：

```
curl -i http://127.0.0.1:9981/api/data?api_key=CLIENT_B_API_KEY
```

预期响应输出：

```
HTTP/1.1 200 OK
Content-Type: text/plain
Connection: keep-alive
...
```

(注意：由于设置了 `hide_credentials: true`，请求在被转发给后端上游服务器前，URL 中的 `api_key=CLIENT_B_API_KEY` 查询参数会被自动抹除)

breaker 插件使用文档

1. 插件概览

`breaker` 插件用于在 API 网关接收到请求时提供服务熔断保护（Circuit Breaker）。当检测到后端服务的特定异常（如 `5xx` 响应）达到设定的阈值时，网关会开启熔断保护，在指定的保护期内自动拒绝后续的超频流量，并直接向客户端返回友好的降级响应（如 `503`），避免因个别后端故障拖垮整个网关或服务链条。

主要应用场景：

- 微服务高可用防护：防止某个微服务节点的宕机或超时导致线程积压，进而引发系统雪崩。
- 降级应急控制：对已知的非核心业务进行异常阈值限制，当故障超出阈值时实现自动化切流与快速降级。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

通过管理接口进行全局配置（API 路径：`/edge/admin/plugin_metadata/breaker`），用于设定全局默认的熔断拦截行为与容错参数。

[!IMPORTANT]

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：`PUT`
- 请求路径：`/edge/admin/plugins/reload`

您可以在配置多个插件的数据属性后，统一执行一次热加载操作。

字段名	类型	默认值	描述	设置示例
<code>status</code>	Integer	<code>503</code>	触发熔断被拦截时，网关返回给客户端的 HTTP 状态码。取值范围 <code>200</code> 至 <code>599</code> 。	<code>503</code>
<code>message</code>	String	<code>"Your request is not allowed."</code>	触发熔断被拦截时，返回的 HTTP 响应体消息内容（最长 1024 字符）。	<code>"Service temporarily unavailable."</code>
<code>bypass_missing_key</code>	Boolean	<code>true</code>	当 <code>key</code> 解析失败或变量缺失返回 <code>nil</code> 时，是否跳过熔断逻辑。如果设为 <code>false</code> ，当 Key 缺失时将直接触发熔断拦截。	<code>true</code>
<code>bypass_error_limit</code>	Boolean	<code>true</code>	当本地限流共享内存发生溢出或异常时，是否跳过限制（放行流量）。如果设为 <code>false</code> ，遇到内部故障将直接触发熔断拦截。	<code>true</code>
<code>header_name</code>	String	<code>"X-Edge-Breaker-By"</code>	拦截响应时，若开启了展示，该字段定义响应标头（Header）的名称。	<code>"X-Circuit-Status"</code>
<code>show_breaker_header</code>	Boolean	<code>false</code>	拦截响应中是否添加定义在 <code>header_name</code> 里的响应头，以标识该响应来自网关的熔断拦截。	<code>true</code>

2.3 插件基础属性 (Config)

路由级配置，可覆盖全局定义的插件数据属性 (Metadata Config)。

字段名	类型	默认值	描述	设置示例
key	String	"\${uri}"	熔断器进行跟踪及流量限制的 Key。支持变量插值（例如通过 \${uri}、\${host} 等）。	"\${uri}"
time	Integer	3	最大熔断持续保护时间（秒）。每次触发异常会以指数退避算法计算持续时间（即 $2^{\text{failure_times}}$ 秒），但最大不超过该时间。	10
count	Integer	3	处于熔断保护期内时，允许放行测试的最大请求流量个数。	5
window	Number	1	配合 count 的流量限制统计时间窗口（秒）。	2
unhealthy	Object	无	异常监控状态设置。包含具体的匹配规则与触发阈值，详见下方说明。	见配置示例
healthy	Object	无	恢复监控状态设置。包含具体的匹配规则与恢复阈值，详见下方说明。	见配置示例
group	String	无	熔断共享分组名称。配置相同的路由将共享熔断器状态。	"api_group"
status	Integer	503	同上	503
message	String	"Your request is not allowed."	同上	"Service Unavailable"
bypass_missing_key	Boolean	true	同上	true
bypass_error_limit	Boolean	true	同上	true
header_name	String	"X-Edge-Breaker-By"	同上	"X-Breaker-Status"
show_breaker_header	Boolean	false	同上	true

说明

- unhealthy** 的子字段说明：
 - expr**：标准 Edge 条件表达式列表，用于确定哪些上游响应是不健康的。默认值为 `[ [ "ngx_ups_status", ">=", 500 ] ]`。关于条件表达式的写法细节，请参考 [Edge 表达式列表](#)。
 - count**：触发熔断所需的连续不健康响应次数，默认值为 `3`。
- healthy** 的子字段说明：
 - expr**：标准 Edge 条件表达式列表，用于确定哪些上游响应是健康的。默认值为 `[ [ "ngx_ups_status", "==", 200 ] ]`。详见 [Edge 表达式列表](#)。
 - count**：重置熔断器所需的连续健康响应次数，默认值为 `1`。

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 `/api/breaker` 开启 `breaker` 插件。
我们配置熔断器跟踪 Key 为当前请求的 URI：

- 异常条件为：连续 `3` 次返回状态码 `500`（`unhealthy` 规则）。
- 触发熔断后的最长保护时长为 `10` 秒。
- 熔断期间允许每秒放行 `1` 次请求用于测试后端恢复状况（`count: 1, window: 1`）。
- 恢复正常条件为：放行测试请求得到连续 `2` 次 `200` 正常响应（`healthy` 规则）。
- 被拦截时返回 `503` 及 `"Service Broken"`，并在响应头中添加 `X-Breaker-Blocked: true` 的标识。

Important

调用示例中不包含真实敏感密钥，请将占位符 `API_KEY` 替换为您实际环境中的 Admin API Key。

```
curl -i -X PUT \  
  -H "Content-Type: application/json" \  
  -H "X-API-Key: API_KEY" \  
  -d '{
```

```

    "uri": "/api/breaker",
    "plugins": {
      "breaker": {
        "key": "${uri}",
        "time": 10,
        "count": 1,
        "window": 1,
        "unhealthy": {
          "expr": [ [ "ngx_ups_status", ">=", 500 ] ],
          "count": 3
        },
        "healthy": {
          "expr": [ [ "ngx_ups_status", "==", 200 ] ],
          "count": 2
        },
        "status": 503,
        "message": "Service Broken",
        "header_name": "X-Breaker-Blocked",
        "show_breaker_header": true
      }
    },
    "upstream": {
      "nodes": {
        "127.0.0.1:8811": 1
      },
      "type": "roundrobin"
    }
  }' \
http://127.0.0.1:9991/edge/admin/routes/1006

```

3.2 访问网关进行验证

1. 模拟后端异常触发熔断

向网关发起 3 次能够使后端返回 500 的异常请求（假设 `/api/breaker` 接口对应的后端支持通过查询参数 `error=true` 故意返回 500）：

```

## 连续发送 3 次异常请求
curl -i http://127.0.0.1:9981/api/breaker?error=true
curl -i http://127.0.0.1:9981/api/breaker?error=true
curl -i http://127.0.0.1:9981/api/breaker?error=true

```

第 3 次请求返回后，熔断器被激活开启。

2. 模拟高频访问触发网关直接拦截

在激活状态下，向网关以高频方式发起普通正常请求。因为我们在熔断期只允许每秒放行 1 次（`count: 1`, `window: 1`），超出限额的请求会被网关直接拒绝：

```

## 1 秒内连续发送 2 次普通请求
curl -i http://127.0.0.1:9981/api/breaker
curl -i http://127.0.0.1:9981/api/breaker

```

预期第 2 次超额请求的响应输出（网关直接返回 503 且拦截头生效）：

```
HTTP/1.1 503 Service Temporarily Unavailable
Content-Type: text/plain
Connection: keep-alive
X-Breaker-Blocked: EDGE/3.0
Content-Length: 14

Service Broken
```

3. 模拟后端服务恢复，重置熔断器

停止高频请求，让测试请求成功投递到后端并得到 `200 OK` 正常响应（每秒发送 1 次以确保不触发放行限额拦截，且不带 `error=true` 报错参数）：

```
## 每秒发送 1 次请求，使其被半开测试放行到后端，连续 2 次
sleep 1
curl -i http://127.0.0.1:9981/api/breaker
sleep 1
curl -i http://127.0.0.1:9981/api/breaker
```

当前端连续 2 次得到健康响应后，熔断器自动切换为关闭（Closed）状态。

此后再次高频请求，网关不再进行限速拦截：

```
## 熔断关闭后，大并发请求均可以正常透传到上游服务
curl -i http://127.0.0.1:9981/api/breaker
curl -i http://127.0.0.1:9981/api/breaker
```

cors 插件使用文档

1. 插件概览

`cors` 插件用于为 HTTP 服务自动注入跨域资源共享 (CORS) 相关的响应标头（如 `Access-Control-Allow-Origin` 等），并自动拦截处理浏览器的 `OPTIONS` 预检请求，使前端应用可以安全地跨域调用后端的 API 接口。

主要应用场景：

- **跨域接口调用**：前后端分离架构中，前端网页（如 `https://web.example.com`）需要请求网关下的 API 接口（如 `https://api.example.com`）。
- **多域名跨域共享**：网关支持使用正则表达式或全局元数据定义受信任的域名范围，为这些特定域名提供动态跨域响应。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

通过管理接口进行全局配置（API 路径：`/edge/admin/plugin_metadata/cors`），用于设定全局 Origin 分组映射表。

[!IMPORTANT]

所有插件的数据属性（**Metadata Config**）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法: `PUT`
- 请求路径: `/edge/admin/plugins/reload`

您可以在配置多个插件的数据属性后，统一执行一次热加载操作。

字段名	类型	默认值	描述	设置示例
-----	----	-----	----	------

2.3 插件基础属性 (Config)

路由级配置，可覆盖全局定义的插件数据属性 (Metadata Config)。

字段名	类型	默认值	描述	设置示例
<code>allow_origins</code>	String	<code>"*"</code>	允许跨域访问的 Origin 源。支持具体的 Origin 地址、以逗号分隔的多个 Origin 地址、 <code>*</code> （允许所有）或 <code>**</code> （动态镜像请求中的 Origin 头）。格式必须匹配 <code>^(*\\ **\\ null\\ \\w+://[^,]+(,\\w+://[^,]+)*)\$</code> 。	<code>"https://example.com"</code>
<code>allow_methods</code>	String	<code>"*"</code>	允许的跨域 HTTP 方法。支持 <code>*</code> 、以逗号分隔的多个方法（如 <code>GET,POST</code> ）或 <code>**</code> （转换为全部标准方法： <code>GET,POST,PUT,DELETE,PATCH,HEAD,OPTIONS,CONNECT,TRACE</code> ）。	<code>"GET,POST,OPTIONS"</code>
<code>allow_headers</code>	String	<code>"*"</code>	允许的跨域请求头。支持逗号分隔的具体标头、 <code>*</code> 或 <code>**</code> （动态获取请求头 <code>Access-Control-Request-Headers</code> 的值）。	<code>"Content-Type,Authorization"</code>
<code>expose_headers</code>	String	无	允许浏览器访问的响应标头列表。多个值使用逗号分隔。	<code>"X-Custom-Header,X-Trace-Id"</code>
<code>max_age</code>	Integer	<code>5</code>	预检请求（ <code>OPTIONS</code> ）结果的浏览器缓存时间，单位为秒。	<code>3600</code>
<code>allow_credential</code>	Boolean	<code>false</code>	是否允许浏览器在跨域请求中携带凭证（如 <code>Cookie</code> , <code>Authorization</code> 标头等）。	<code>true</code>
<code>allow_origins_by_regex</code>	Array	无	用于正则匹配允许的 Origin 列表。数组成员为正则表达式。	<code>["^https://.*\\.domain\\.com\$"]</code>
<code>allow_origins_by_metadata</code>	Array	无	引用全局数据属性 (Metadata Config) 中定义的 <code>allow_origins</code> 的键名（分组名）列表。若匹配成功，则将该 Origin 视为合法。	<code>["group1"]</code>
<code>timing_allow_origins</code>	String	无	允许设置 <code>Timing-Allow-Origin</code> 响应头的 Origin 规则，用于资源定时 API 收集。格式规范与 <code>allow_origins</code> 一致。	<code>"https://example.com"</code>

2.4 注意事项

- 凭证安全限制：当开启 `allow_credential: true` 时，**严禁**将 `allow_origins`、`allow_methods`、`allow_headers`、`expose_headers`、`timing_allow_origins` 设为 `*`，否则校验将报错拒绝。
- Vary** 标头：当配置具体的跨域 Origin（非 `*`）时，网关将自动在响应头中加入 `Vary: Origin`，防止不同域的浏览器读取到代理缓存中前一个域的跨域配置响应。

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 `/api/cors` 开启 `cors` 插件，将其配置为允许 `https://example.com` 进行跨域访问，且允许 `Content-Type` 和 `Authorization` 请求头。

 **Important**

调用示例中不包含真实敏感密钥，请将占位符 `API_KEY` 替换为您实际环境中的 Admin API Key。

```
curl -i -X PUT \  
  -H "Content-Type: application/json" \  
  -H "X-API-Key: API_KEY" \  
  -d '{  
    "uri": "/api/cors",  
    "plugins": {  
      "cors": {
```

```
    "allow_origins": "https://example.com",
    "allow_methods": "GET,POST,OPTIONS",
    "allow_headers": "Content-Type,Authorization",
    "max_age": 600,
    "allow_credential": false
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:8811": 1
    },
    "type": "roundrobin"
  }
}' \
http://127.0.0.1:9991/edge/admin/routes/1003
```

3.2 访问网关进行验证

验证预检 OPTIONS 请求

模拟浏览器发送跨域预检请求：

```
curl -i -X OPTIONS \
-H "Origin: https://example.com" \
-H "Access-Control-Request-Method: POST" \
-H "Access-Control-Request-Headers: Authorization" \
http://127.0.0.1:9981/api/cors
```

预期响应输出：

```
HTTP/1.1 200 OK
Connection: keep-alive
Access-Control-Allow-Origin: https://example.com
Access-Control-Allow-Methods: GET,POST,OPTIONS
Access-Control-Max-Age: 600
Access-Control-Allow-Headers: Content-Type,Authorization
Vary: Origin
Content-Length: 0
```

验证普通跨域 GET 请求

发送普通的 GET 请求并携带正确的 `Origin` 标头：

```
curl -i -H "Origin: https://example.com" http://127.0.0.1:9981/api/cors
```

预期响应输出：

```
HTTP/1.1 200 OK
Content-Type: text/plain
Connection: keep-alive
Access-Control-Allow-Origin: https://example.com
Access-Control-Allow-Methods: GET,POST,OPTIONS
Access-Control-Max-Age: 600
Access-Control-Allow-Headers: Content-Type,Authorization
Vary: Origin
...
```


dns_server 插件使用说明书

1. 插件概览

`dns_server` 插件用于在 API 网关 L4 Stream 层（UDP 协议）提供内建的高性能 DNS 域名解析服务。网关可直接作为权威域名服务器，拦截并响应特定域名的 DNS 查询请求。

主要应用场景：

- 边缘 DNS 解析：**在网关边缘直接响应客户端的域名解析，大幅提高域名首次解析的性能。
- 智能 Geo-DNS 调度：**根据客户端的真实地理位置/子网网段信息，动态返回最接近的或专用的解析 IP 地址，实现智能网络流量调度。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

通过管理接口进行全局配置（API 路径：`/stream/edge/admin/plugin_metadata/dns_server`），用于设定全局默认的 DNS 解析映射。

****[!IMPORTANT]**

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：`PUT`
- 请求路径：`/stream/edge/admin/plugins/reload`

您可以在配置多个插件的数据属性后，统一执行一次热加载操作。

字段名	类型	默认值	描述	设置示例
<code>hosts</code>	Object	无	域名解析主机映射表。其键为目标域名，值为包含解析目标和客户端匹配规则的对象，详见下文结构说明。	见下文结构说明

`hosts` 内部字段结构说明：

结构格式为：`{ 域名: { "nodes": { "解析地址1": [客户端源地址列表1], "解析地址2": [客户端源地址列表2] } } }`。

每个域名下配置 `nodes` 对象：

- `nodes` (Object)：**键为解析返回的目标 IP（如 `10.0.0.1`），值为匹配该目标的客户端源 IP/CIDR 网段列表（数组）。
 - 智能解析分流 (Geo-DNS)：**网关接收到客户端 DNS 请求时，若客户端源 IP 匹配某解析地址对应的地址列表，则直接返回该解析地址。
 - 默认轮询响应：**当客户端源 IP 不在任何指定的解析地址对应的地址列表中时，网关将轮询 (**Round-Robin**) 响应这些配置的解析地址，从而实现对不匹配客户端的默认均衡分发。

2.3 插件基础属性 (Config)

路由级配置，可覆盖全局定义的插件数据属性 (Metadata Config)。

字段名	类型	默认值	描述	设置示例
hosts	Object	无	同上	见上文结构说明

3. 调用与验证示例

3.1 创建并启用 Stream 路由

使用 Admin 管理接口创建或更新一条监听 UDP 9971 端口的 Stream 路由，并为其启用 dns_server 插件。我们将为域名 test.local 设定解析规则：

- 当客户端 IP 属于本地 127.0.0.1 时，强制解析到 10.0.0.1；
- 当客户端 IP 属于本地 192.168.0.0/16 局域网段时，强制解析到 10.0.0.2。

Important

调用示例中不包含真实敏感密钥，请将占位符 API_KEY 替换为您实际环境中的 Admin API Key。

```
curl -i -X PUT \  
  -H "Content-Type: application/json" \  
  -H "X-API-Key: API_KEY" \  
  -d '{  
    "server_addr": "0.0.0.0:9971",  
    "protocol": "UDP",  
    "plugins": {  
      "dns_server": {  
        "hosts": {  
          "test.local": {  
            "nodes": {  
              "10.0.0.1": ["127.0.0.1"],  
              "10.0.0.2": ["192.168.0.0/16"]  
            }  
          }  
        }  
      }  
    }  
  }' \  
  http://127.0.0.1:9991/stream/edge/admin/routes/2001
```

3.2 访问网关进行验证

我们可以使用标准的 dig 命令向网关的 Stream UDP 监听端口发送 DNS A 记录查询，验证是否能正确获取到域名解析结果：

```
dig @127.0.0.1 -p 9971 test.local
```

预期输出 (ANSWER SECTION) :
由于我们从本机 127.0.0.1 发起查询，命中 10.0.0.1 的客户端匹配网段，因此解析应准确返回 10.0.0.1：

```
;; QUESTION SECTION:  
;test.local.      IN  A  
  
;; ANSWER SECTION:  
test.local.      600 IN  A 10.0.0.1
```

dns_upstream 插件使用说明书

1. 插件概览

`dns_upstream` 插件用于在 API 网关 L4 Stream 层（UDP 协议）提供内建的高性能智能 DNS 域名解析及动态负载调度服务。网关可直接作为权威域名服务器，拦截并响应特定域名的 DNS 查询请求。

主要应用场景：

- **智能 Geo-DNS 解析调度**：根据客户端的真实地理位置/子网网段信息，动态返回匹配的最佳后端服务器 IP。
- **DNS 级动态负载均衡**：当客户端 IP 不在特定的调度白名单中时，利用网关内置的 Balancer 轮询机制（如 `roundrobin` 等算法）在多个后端服务器 IP 之间动态分配流量，实现高可用调度。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

通过管理接口进行全局配置（API 路径：`/stream/edge/admin/plugin_metadata/dns_upstream`），用于设定全局默认的 DNS 解析映射。

Important
所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：`PUT`
- 请求路径：`/stream/edge/admin/plugins/reload`

您可以在配置多个插件的数据属性后，统一执行一次热加载操作。

字段名	类型	默认值	描述	设置示例
<code>hosts</code>	Object	无	域名解析调度映射表。其键为目标域名，值为包含解析目标和客户端匹配规则的上游配置对象，详见下文结构说明。	见下文结构说明

`hosts` 内部字段结构说明：

结构格式为：`{ 域名: upstream_config }`。
每个域名下可配置标准上游对象：

- `nodes` (Object)：键为后端服务节点（形如 `目标IP:端口`），值为匹配该目标的客户端源 IP/CIDR 网段列表（数组）。
 - **网段精准路由**：网关接收到客户端 DNS 请求时，若客户端源 IP 匹配某后端节点对应的客户端列表，则直接响应返回该节点所对应的 IP 地址。
 - **默认负载分发**：当客户端源 IP 不属于任何指定的后端匹配网段时，网关将使用上游所配置的算法（默认轮询 `roundrobin`）对所有节点进行分发调度，并返回选中节点的 Host/IP 响应客户端。
- `type` (String)：负载均衡算法类型，默认值为 `roundrobin`。
- `scheme` (String)：后端协议，默认值为 `"http"`。

- `pass_host` (String): 透传 Host 头部策略, 默认值为 `"node"`。

2.3 插件基础属性 (Config)

路由级配置, 可覆盖全局定义的插件数据属性 (Metadata Config)。

字段名	类型	默认值	描述	设置示例
<code>hosts</code>	Object	无	同上	见下文结构说明

3. 调用与验证示例

3.1 创建并启用 Stream 路由

使用 Admin 管理接口创建或更新一条监听 UDP `9971` 端口的 Stream 路由, 并为其启用 `dns_upstream` 插件。我们将为域名 `test.local` 设定解析规则:

- 当客户端 IP 属于本地 `127.0.0.1` 时, 解析返回 `10.0.0.1`;
- 当客户端 IP 属于本地 `192.168.0.0/16` 局域网段时, 解析返回 `10.0.0.2`;
- 其它不匹配的客户端请求, 通过 `roundrobin` 算法在配置的节点列表中随机/轮询分发。

📌 Important

调用示例中不包含真实敏感密钥, 请将占位符 `API_KEY` 替换为您实际环境中的 Admin API Key。

```
curl -i -X PUT \  
  -H "Content-Type: application/json" \  
  -H "X-API-Key: API_KEY" \  
  -d '{  
    "server_addr": "0.0.0.0:9971",  
    "protocol": "UDP",  
    "plugins": {  
      "dns_upstream": {  
        "hosts": {  
          "test.local": {  
            "nodes": {  
              "10.0.0.1:80": ["127.0.0.1"],  
              "10.0.0.2:80": ["192.168.0.0/16"]  
            },  
            "type": "roundrobin"  
          }  
        }  
      }  
    }  
  }' \  
  http://127.0.0.1:9991/stream/edge/admin/routes/2002
```

3.2 访问网关进行验证

我们可以使用标准的 `dig` 命令向网关的 Stream UDP 监听端口发送 DNS A 记录查询, 验证是否能正确获取到域名解析结果:

```
dig @127.0.0.1 -p 9971 test.local
```

预期输出（ANSWER SECTION）：

由于我们从本机 127.0.0.1 发起查询，命中 10.0.0.1 的客户端匹配网段，因此解析应准确返回 10.0.0.1：

```
;; QUESTION SECTION:
;test.local.      IN      A

;; ANSWER SECTION:
test.local.      600 IN    A 10.0.0.1
```

log_http 插件使用文档

1. 插件概览

log_http 插件允许网关以异步批处理的方式，将每笔请求的详细元数据（如客户端 IP、访问路径、响应状态码、请求耗时及正文内容等）以 HTTP POST 方式上报到指定的第三方日志服务器。该插件内置发送队列和失败降级本地落盘处理机制。

主要应用场景：

- 实时流量审计与日志监控：将网关层的访问日志发送到自建的日志管理中心进行统一审计或实时大屏展示。
- 慢日志与错误跟踪：配置条件表达式，仅将响应码大于等于 400 或执行时间长的请求详情上报至告警系统。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

全局配置。配置有变更后，需执行一次插件热加载操作使配置生效 (PUT /edge/admin/plugins/reload)。

字段名	类型	默认值	描述	设置示例
logs	Object / Array	默认测试地址	全局默认收集端的 URI 列表及其子参数。	[{"uri": "http://127.0.0.1:9980/log/test"}]
timeout	Number	3	HTTP 连接和发送的超时时间（单位：秒）。	5
keepalive_timeout	Integer	150	连接池空闲连接超时时间（单位：秒）。	60
keepalive_pool	Integer	30	每个 worker 进程的最大空闲连接数。	50
batch_max_size	Integer	1000	触发发送的批次最大日志量。	500

2.3 插件基础属性 (Config)

路由级配置。

字段名	类型	默认值	描述	设置示例
logs	String / Array	无	必填。上报日志的目标 URI（例如 HTTP/HTTPS 收集服务）。	"http://127.0.0.1:9980/log/test"
formats	Array / Object	默认格式	日志的具体字段。支持以 <code>\${var}</code> 指定要上报的字段列表。	["\${remote_addr}", "\${uri}", "\${status}"]
include_req_body	Boolean	false	日志中是否带上请求正文（Body）。	true
include_req_body_expr	Array	无	收集请求体的条件表达式。设置示例必须严格使用标准的 Edge 表达式格式。	[["uri", "=", "/api/upload"]]
include_resp_body	Boolean	false	日志中是否带上响应正文（Body）。	true
include_resp_body_expr	Array	无	收集响应体的条件表达式。	[["status", ">=", 400]]

说明

- 关于条件表达式字段，请引用 [Edge 表达式列表](#)。
- 日志格式可用变量，请引用 [Edge 变量列表](#)。

3. 调用与验证示例

本示例介绍如何创建路由，开启 `log_http` 插件将日志投递到指定的 HTTP 收集端。

Important

调用示例中不包含真实敏感密钥，请将占位符 `API_KEY` 替换为您实际环境中的 Admin API Key。

3.1 步骤 1：创建启用 `log_http` 插件的路由

创建路由关联路径 `/test_log_http`，配置 `log_http` 指向收集地址并设定自定义日志格式。

```
curl -i -X PUT \  
  -H "Content-Type: application/json" \  
  -H "X-API-Key: API_KEY" \  
  -d '{  
    "uri": "/test_log_http",  
    "plugins": {  
      "log_http": {  
        "logs": "http://127.0.0.1:9980/log/test",  
        "formats": [ "${remote_addr}", "${uri}", "${status}" ]  
      }  
    },  
    "upstream": {  
      "nodes": {  
        "127.0.0.1:8811": 1  
      },  
      "type": "roundrobin"  
    }  
  }' \  
  http://127.0.0.1:9991/edge/admin/routes/3005
```

3.2 访问网关进行验证

1. 发起访问触发日志投递

```
curl -i http://127.0.0.1:9981/test_log_http
```

预期响应输出：

```
HTTP/1.1 200 OK
Content-Type: text/plain
Connection: keep-alive
...
```

2. 日志收集端接收结果

由于日志是异步批量上报，当上报触发后（默认为 1000 条或定时器到期），HTTP 日志接收端将收到类似如下结构的 POST 请求：

```
POST /log/test HTTP/1.1
Host: 127.0.0.1:9980
Content-Type: text/plain
X-EDGE-LOG-HTTP: 0000018f773412ab34901f...

127.0.0.1|;/test_log_http|;200
```

(注意：若收集端临时故障不可达，可在网关节点的 `logs/log_http.failed.log` 中查看降级落盘的备份日志数据)

log_kafka 插件使用文档

1. 插件概览

`log_kafka` 插件用于在网关层以异步批处理方式将 HTTP 请求日志写入指定的 Kafka 集群。该插件包含对集群高可用的支持、SASL 认证、采样比过滤以及本地故障失败降级写入文件的机制。

主要应用场景：

- 实时海量访问日志收集：将网关的访问日志以大吞吐异步方式直接送入 Kafka 主题，供大数据分析平台实时流式处理。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

全局配置。配置变更后，需执行一次插件热加载操作使配置生效 (`PUT /edge/admin/plugins/reload`)。

字段名	类型	默认值	描述	设置示例
<code>broker_list</code>	Array of Object	无	Kafka 代理 (Broker) 的 host/port 列表。	<code>[{"host": "127.0.0.1", "port": 9092}]</code>
<code>topic</code>	String	<code>"testtopic"</code>	投递的默认主题名。	<code>"gateway-log"</code>
<code>key</code>	String	<code>"\${ngx_now}"</code>	发送消息时用于 Kafka Partition 选择的 Key，支持变量。	<code>"\${remote_addr}"</code>
<code>producer_config</code>	Object	无	生产者属性设置（如： <code>producer_type: "async"</code> ， <code>request_timeout: 3</code> 等）。	<code>{}</code>
<code>batch_max_size</code>	Integer	<code>1000</code>	批量发送的最大批次日志条数。	<code>500</code>

2.3 插件基础属性 (Config)

路由级配置。

字段名	类型	默认值	描述	设置示例
logs	String / Array	"cluster_test"	关联配置的 Kafka 集群识别标记。	"cluster_prod"
topic	String	同上	覆盖的主题名。	"my-special-topic"
sample_ratio	Number	1	采样比例，范围在 0.00001 与 1 之间。	0.1
include_req_body	Boolean	false	日志中是否包含请求体（Body）。	true
include_req_body_expr	Array	无	触发收集请求体的条件表达式。设置示例必须严格使用标准的 Edge 表达式格式。	[["uri", "==", "/api/upload"]]
include_resp_body	Boolean	false	日志中是否包含响应体（Body）。	true
include_resp_body_expr	Array	无	触发收集响应体的条件表达式。	[["status", ">=", 400]]

说明

- 关于条件表达式字段，请引用 [Edge 表达式列表](#)。
- 格式可用变量，请引用 [Edge 变量列表](#)。

3. 调用与验证示例

本示例介绍如何创建启用 log_kafka 插件的路由。

Important

调用示例中不包含真实敏感密钥，请将占位符 API_KEY 替换为您实际环境中的 Admin API Key。

3.1 步骤 1：创建启用 log_kafka 插件的路由

创建路由关联路径 /test_log_kafka，配置 log_kafka 相关属性：

```
curl -i -X PUT \  
  -H "Content-Type: application/json" \  
  -H "X-API-Key: API_KEY" \  
  -d '{  
    "uri": "/test_log_kafka",  
    "plugins": {  
      "log_kafka": {  
        "logs": "cluster_test",  
        "topic": "testtopic",  
        "sample_ratio": 1  
      }  
    },  
    "upstream": {  
      "nodes": {  
        "127.0.0.1:8811": 1  
      },  
      "type": "roundrobin"  
    }  
  }' \  
  http://127.0.0.1:9991/edge/admin/routes/3006
```

3.2 访问网关进行验证

向网关发起访问请求：


```
curl -i http://127.0.0.1:9981/test_log_kafka
```

预期响应输出：

```
HTTP/1.1 200 OK
Content-Type: text/plain
Connection: keep-alive
...
```

(提示：网关将以异步批处理方式投递消息至指定的 Kafka 集群。若 Kafka 集群不可达，日志将自动写入网关节点的 logs/log_kafka.failed.log 本地文件中降级暂存)

log_mirror 插件使用文档

1. 插件概览

log_mirror 插件允许网关在不影响主客户端请求流程的前提下，异步克隆当前真实的 HTTP 请求（包括请求方法、URI、查询参数、请求体及标头），并向指定的第三方接收端发起镜像请求，常用于流量回放、旁路 WAF 审计或灰度比对。

主要应用场景：

- 生产流量克隆与测试：将真实的在线业务流量镜像复制到灰度环境，进行影子系统压力测试或新版功能比对。
- 安全旁路检测：在后台无感复制一份完整的 HTTP 请求报文并投递至安全审计中心。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

无。

2.3 插件基础属性 (Config)

路由级配置。

字段名	类型	默认值	描述	设置示例
logs	String / Array	无	必填。指定要推送到的一个或多个镜像接收端 URI。	"http://127.0.0.1:9980/mirror/test"
header_name	String	"X-EDGE-LOG-MIRROR"	镜像请求中附加的追踪 ID 的标头名称。	"X-Mirror-Id"
sample_ratio	Number	1	流量镜像采样比例，范围在 0.00001 到 1 之间。	0.1
include_req_body	Boolean	false	镜像中是否连同请求体（Body）一起复制发送。	true
include_req_body_expr	Array	无	触发镜像请求体的条件表达式。设置示例必须严格使用标准的 Edge 表达式格式。	[["uri", "==", "/api/upload"]]
include_resp_body	Boolean	false	是否缓冲响应体（主要用于触发条件判定）。	true
include_resp_body_expr	Array	无	触发镜像的响应体条件表达式。	[["status", ">=", 400]]

- 关于条件表达式字段，请引用 [Edge 表达式列表](#)。
- 镜像可引用的变量，请引用 [Edge 变量列表](#)。

3. 调用与验证示例

本示例介绍如何创建路由，开启 `log_mirror` 插件将请求报文异步发送至镜像端。

Important

调用示例中不包含真实敏感密钥，请将占位符 `API_KEY` 替换为您实际环境中的 Admin API Key。

3.1 步骤 1：创建启用 `log_mirror` 插件的路由

创建路由关联路径 `/test_log_mirror`，配置 `log_mirror` 指向镜像收集地址并允许复制请求体。

```
curl -i -X PUT \  
-H "Content-Type: application/json" \  
-H "X-API-Key: API_KEY" \  
-d '{  
  "uri": "/test_log_mirror",  
  "plugins": {  
    "log_mirror": {  
      "logs": "http://127.0.0.1:9980/mirror/test",  
      "sample_ratio": 1,  
      "include_req_body": true  
    }  
  },  
  "upstream": {  
    "nodes": {  
      "127.0.0.1:8811": 1  
    },  
    "type": "roundrobin"  
  }  
}' \  
http://127.0.0.1:9991/edge/admin/routes/3008
```

3.2 访问网关进行验证

1. 发起访问触发流量镜像

向网关发送包含请求体和自定义标头的 `POST` 请求：

```
curl -i -X POST -H "X-Test-Header: val" -d "my_request_body" http://127.0.0.1:9981/test_log_mirror
```

预期响应输出：

```
HTTP/1.1 200 OK  
Content-Type: text/plain  
Connection: keep-alive  
...
```

2. 镜像端收到回放请求

由于镜像是异步发送，目标镜像收集端将在后台收到网关复制的 HTTP POST 请求。镜像请求除了携带 `Host` 和 `Authorization` 之外，会完全保留原始请求结构：

```
POST /mirror/test HTTP/1.1
Host: 127.0.0.1:9980
Content-Type: application/x-www-form-urlencoded
X-EDGE-LOG-MIRROR: 0000018f773412ab34901f...
X-Test-Header: val

my_request_body
```

(注意：若收集端不可达，可在网关节点的 `logs/log_mirror.failed.log` 中查看降级落盘的备份日志数据)

log_process 使用文档

插件概览

`log_process` 插件是网关提供的高性能文件日志收集与记录插件。支持对客户端请求和响应的多种内置变量进行插值拼接，并生成日志文件落地磁盘。支持缓存批量写盘（提高吞吐性能）以及针对特定请求/响应体（通过条件表达式）的过滤提取与记录。

配置详情

1. 插件静态属性 (Attr Config)

无。

2. 插件数据属性 (Metadata Config)

通过管理节点接口进行全局配置（路径：`/edge/admin/plugin_metadata/log_process`）。配置变更后，必须执行一次插件热加载操作以使配置生效（`PUT /edge/admin/plugins/reload`，详见 [插件管理](#)）；如果同时配置了多个插件的数据属性，可以在全部配置完成后统一执行一次热加载。

字段名	类型	默认值	描述	设置示例
<code>logs</code>	Object/Array	见下文说明	默认的文件日志配置。可以是以日志文件路径为键、值为单项配置的字典，或者是单项配置组成的数组。	见下文示例
<code>formats_sep</code>	String	<code>"\ ;"</code>	默认的日志字段分隔符。	<code>"\ "</code>
<code>json</code>	String	无	处理json数据时，dkjson会按字段顺序进行排序，cjson不排序；	<code>"cjson"</code>
<code>cache_time</code>	Integer	<code>1</code>	目前，只支持0，1	<code>1</code>

说明: `logs` 内每个日志项包含以下子字段：

- `file`：日志目标路径。仅支持字母、数字、点、正斜杠、下划线及减号（例如：`logs/process.log`）。
- `formats`：日志字段格式，支持字符串、数组或包含自定义键值映射的对象。如果是数组，将使用 `formats_sep` 作为分隔符拼接输出。
- `formats_sep`：指定覆盖全局的日志字段分隔符。
- `json`：指定覆盖全局的JSON库（`"cjson"` 或 `"dkjson"`）。
- `cache_time`：指定覆盖全局的日志缓存落盘时间。

3. 插件基础属性 (Config)

路由级属性，启用该插件时配置在路由下。

字段名	类型	默认值	描述	设置示例
logs	String/Array	"logs/process.log" (HTTP) "logs/process.stream.log" (Stream)	指定记录日志的磁盘文件路径，支持配置单个路径字符串或由多个路径组成的数组。详见下方注意事项。	["logs/custom_access.log"]
formats	String/Object/Array	见全局默认格式	覆盖全局的日志字段格式。可配置变量详见下方说明。	["\${time_local}", "\${remote_addr}", "\${status}"]
formats_sep	String	同上	覆盖默认的日志字段分隔符。	"\ "
json	String	同上	处理json数据时，dkjson会按字段顺序进行排序，cjson不排序；	"dkjson"
cache_time	Integer	同上	目前，只支持0，1	0
include_req_body	Boolean	false	处理请求时，即使没有使用到请求体，也要为稍后的日志，预先保存好请求体	true
include_req_body_expr	Array	无	条件表达式。只有在此条件匹配成功时，才会记录请求体。详见下方说明。	[["uri", "==", "/submit"]]
include_resp_body	Boolean	false	处理请求时，即使没有使用到响应体，也要为稍后的日志，预先保存好响应体	true
include_resp_body_expr	Array	无	条件表达式。只有在此条件匹配成功时，才会记录响应体。详见下方说明。	[["status", ">=", 400]]

说明:

- 1. `formats` 支持配置 Nginx/Edge 变量进行动态提取，可用的变量参考 [Edge 变量列表](#)。
- 2. 条件表达式（如 `include_req_body_expr`、`include_resp_body_expr` 等）的示例及配置必须遵循标准的 Edge 条件表达式格式，详见 [Edge 表达式列表](#)。
- 3. 全局默认缺省目的地与日志格式：
当提交的插件数据属性 (Metadata Config) 为 `{}` 或 `{ "logs": {} }` 时，网关会自动配置默认日志路径与默认格式：

o HTTP 模式：

- 默认日志文件路径：`logs/process.log`
- 默认日志格式：如果不指定 `formats`，网关将使用以下内置变量列表依次拼接输出（默认使用 `|;` 分隔）：
 - 1. `${req_start_time#time_format,%Y%m%d%H%M%S,%03d}`（请求开始时间，精确到毫秒）
 - 2. `${http_x-edge-traceid}`（Trace ID）
 - 3. `${clientip}`（真实客户端 IP）
 - 4. `${username}`（经过认证的用户标识）
 - 5. `${deviceid}`（移动客户端设备 ID）
 - 6. `${udid}`（移动设备唯一标识码）
 - 7. `${cookie_JSESSIONID}`（客户端 Cookie 中的 JSESSIONID）
 - 8. `${method}`（HTTP 请求方法）
 - 9. `${uri}`（请求 URI）
 - 10. `${req_args_string#limitn,2048}`（请求查询参数，限长 2048 字节）
 - 11. `${http_referer}`（HTTP 请求中的 Referer 来源）
 - 12. `${http_cookie}`（完整的客户端 Cookie 串）
 - 13. `${http_user-agent}`（客户端 User-Agent）

14. `${req_headers}` (完整的请求头部信息)
15. `${upstream_addr}` (转发的上游后端服务器地址)
16. `${upstream_response_time#fixdefault,0,0}` (上游后端响应耗时, 缺失时缺省值为 0)
17. `${request_time}` (网关处理请求的总耗时)
18. `${status}` (网关返回给客户端的 HTTP 状态码)
19. `${route_id}` (匹配命中的网关路由 ID)
20. `${plugin_riskinfo}` (风控等插件评估出的风险特征描述信息)
21. `${plugin_riskid#fixdefault,,0}` (安全风控策略 ID, 缺失时缺省值为 0)

◦ **Stream 模式:**

- 默认日志文件路径: `logs/process.stream.log`
- 默认日志格式: 如果不指定 `formats`, 网关将使用以下内置变量依次拼接 (默认使用 `|` 分隔):
 1. `${host}` (请求的主机名)
 2. `${time_iso8601}` (ISO 8601 格式的本地时间)
 3. `${remote_addr}` (客户端 IP 地址)

注意事项

- **动态变量解析:** 在 `formats` 配置中写入的所有动态变量, 如 `${uri}`、`${host}` 等, 都将被网关动态提取解析。支持的内置变量请参考 [Edge 变量列表](#)。
- **日志格式与文件配置:** 日志属性 (特别是日志格式如 `formats`) 应优先配置在**数据属性 (Metadata Config)** 中, 而在**基础属性 (Config)** 里建议仅配置需要写入的日志文件路径。这可保证所有写入同一文件的日志格式保持统一。如果确实需要在路由的基础属性中定义格式, 应当格外注意多路由写入同一文件时格式是否一致; 此外, 当指定的日志文件第一次被调用且在数据属性中尚未初始化配置时, 错误日志中可能会出现类似 `not found log_conf: logs/xxx.log` 的警告。
- **磁盘写入延迟:** 为了最大化网关的处理吞吐率, 日志支持内存批量落盘。如果设置了 `cache_time` 大于 0, 写盘操作会有最长 `cache_time` 秒的延迟。若需进行实时调试, 可将 `cache_time` 设置为 0 (但高并发生产环境不推荐设置 0)。
- **表达式过滤 Body:** 保存请求体与响应体会占用额外的内存, 并且响应体缓冲会改变 Content-Length 的发送方式。推荐仅针对必要的调试接口或异常状态码通过 `include_req_body_expr` 与 `include_resp_body_expr` 进行条件性开启。

调用与验证示例

1. 全局配置插件数据属性 (Metadata)

我们可以通过管理节点接口配置全局的默认日志行为 (此处以开启 JSON 格式, 且指定缓存时间为 1 秒为例):

注意: 实际调用管理接口时, 请求体需要先经过 SM4 加密, 返回结果也需要进行解密, 详见 [管理接口调用规范](#)。

```
curl -L -X PUT 'http://127.0.0.1:9990/edge/admin/plugin_metadata/log_process' \  
-H 'X-API-KEY: API_KEY' \  
-H 'Content-Type: application/json' \  
-d '{  
  "json": "cjson",  
  "cache_time": 1,  
  "formats_sep": "|",  
  "logs": {  
    "logs/process.log": {  
      "formats_sep": "|"  
    }  
  }  
}
```

```
}  
'
```

提交配置变更后，执行一次插件热加载（如果配置了多个插件，可以在全部配置完后统一执行一次）

```
curl -L -X PUT 'http://127.0.0.1:9990/edge/admin/plugins/reload' \  
-H 'X-API-KEY: API_KEY'
```

2. 启用路由日志配置示例

现在配置指定路由以启用自定义日志格式（以记录特定请求头、请求 URI 以及当状态码大于等于 400 时捕获响应体为例）：

```
curl -L -X PUT 'http://127.0.0.1:9990/edge/admin/routes/3002' \  
-H 'X-API-KEY: API_KEY' \  
-H 'Content-Type: application/json' \  
-d '{  
  "uri": "/test_log",  
  "plugins": {  
    "log_process": {  
      "logs": ["logs/custom_audit.log"],  
      "cache_time": 0,  
      "formats": [  
        "${time_local}",  
        "${remote_addr}",  
        "${uri}",  
        "${http_user_agent}"  
      ],  
      "formats_sep": "|",  
      "include_resp_body": true,  
      "include_resp_body_expr": [  
        ["status", ">=", 400]  
      ]  
    }  
  },  
  "upstream": {  
    "nodes": {  
      "127.0.0.1:8111": 1  
    },  
    "type": "roundrobin"  
  }  
}'
```

3. 请求验证

步骤 1：向网关发送正常访问请求

```
$ curl -i http://127.0.0.1:9980/test_log  
HTTP/1.1 200 OK  
Content-Length: 13  
...  
hello, world!
```

由于 `cache_time` 设置为 `0`，日志将立即写入磁盘。我们可以查看生成的 `logs/custom_audit.log` 文件的内容：

```
$ cat logs/custom_audit.log
13/Jun/2026:19:58:00 +0800|127.0.0.1|/test_log|curl/7.79.1
```

步骤 2：向网关发送触发 404/500 异常请求（模拟触发响应体记录）

假设向后端发起一个会产生 502 状态码的请求：

```
$ curl -i http://127.0.0.1:9980/test_log?trigger_error=1
HTTP/1.1 502 Bad Gateway
Content-Length: 150
...
```

检查 logs/custom_audit.log，由于触发了条件表达式 ["status", ">=", 400]，相应的错误响应体内容也已被记录：

```
$ cat logs/custom_audit.log
13/Jun/2026:19:58:05 +0800|127.0.0.1|/test_log|curl/7.79.1
```

(注意：如果是默认格式且 include_resp_body 为 true 时，日志内将包含完整的 json 结构数据，包含 response.body 内容)

log_rotate 使用文档

插件概览

log_rotate 插件是网关提供的高性能本地日志切分、打包压缩与生命周期清理插件。该插件作为全局单例运行，无需在特定路由或 Consumer 规则下挂载。网关一旦初始化并启动，该插件即会在后台以定时器（Timer）驱动，自动针对指定的日志路径进行周期性的重命名切割、按需进行 gzip 压缩归档以及超出最大保存保留数量后的超期文件清理。

配置详情

1. 插件静态属性 (Attr Config)

该属性配置存储于 edge.env 文件的 plugin_attr.log_rotate 配置块中。

字段名	类型	默认值	描述	设置示例
enable	Boolean	true	全局开启或关闭日志轮转功能。	true
logs	Object	见下文说明	包含每个日志文件的具体轮转规则。以日志文件相对路径为键。若未配置，默认只轮转系统自带日志。	见下文示例
rotate	Integer	48	全局默认的轮转历史日志最多保留数量。设置为负值表示不限制保留数量。	24
interval	String	"1h"	全局默认的日志轮转周期。支持按月、按日或按小时轮转。	"1d"
compress	Boolean	true	全局默认的是否将轮转切割出的文件打包压缩为 .tar.gz。	true
compress_delay	Boolean	false	全局默认的是否延迟压缩。若为 true，则会将当前切分出来的日志延迟到下一个轮转周期再进行压缩，平抑瞬间耗能。	true

logs 字典内单项配置字段说明：

logs 中的每一个键名（例如 logs/process.log）必须是匹配正则 `^[a-zA-Z0-9./_-]+$` 的合法相对路径。单项配置对象所包含的子字段如下：

- `enable` (Boolean)：该日志文件特有的是否开启轮转的配置。
- `rotate` (Integer)：该日志文件特有的保留份数。若配置，则覆盖全局配置。
- `interval` (String)：该日志文件特有的轮转周期。可选值有：`"1mon"`，`"1d"`，`"1h"`。
- `compress` (Boolean)：该日志文件特有的是否压缩选项。
- `compress_delay` (Boolean)：该日志文件特有的是否延迟压缩选项。

说明：

1. 如果 `logs` 未进行任何显式配置，系统只会为 `logs/error.log` 提供默认的轮转（默认轮转周期为 `1d`，保留份数为 `3`），而 `logs/access.log` 默认不轮转。
2. `log_rotate` 配置继承 fallback 链：单项目志具体配置 -> 全局插件配置 (`plugin_attr.log_rotate`) -> 系统默认缺省值。

2. 插件数据属性 (Metadata Config)

无。

3. 插件基础属性 (Config)

无。

注意事项

- **配置应用流程**：由于 `log_rotate` 为全局静态属性配置，通过修改 `edge.env` 完成后，需要通过命令行重新初始化与重载才能被后台定时任务读取生效。
- **重命名格式**：系统在轮转切割时，会在原文件名与扩展名之间嵌入当前计算出来的时间后缀。例如：
 - 按小时轮转 (`"1h"`)： `process.log` -> `process.2026061404.log`
 - 按天轮转 (`"1d"`)： `process.log` -> `process.20260614.log`
 - 按月轮转 (`"1mon"`)： `process.log` -> `process.202606.log`
- **防 CPU 峰值开销**：当管理的日志文件较大时，打包压缩（`compress`）会在瞬间带来较高的 CPU 负载。建议将 `compress_delay` 设置为 `true`。它会采用延迟压缩机制，并将压缩操作放入下一次轮转周期后，且配合随机抖动时间执行，有效降低瞬时的 I/O 与 CPU 开销。

启用与配置示例

1. 修改 `edge.env` 配置属性

在 `edge.env` 文件的 `plugin_attr` 中，按需添加或修改 `log_rotate` 部分：

```
plugin_attr:
  log_rotate:
    interval: 1h          # 全局默认的日志轮转周期：每小时轮转一次
    rotate: 48            # 全局默认保留的最新的轮转日志备份份数
    compress: true        # 全局默认对切分出来的日志进行打包压缩
    compress_delay: true  # 全局默认开启延迟压缩，缓解 CPU 负载压力
```



```
logs:
  logs/access.log:
    enable: false
  logs/error.log:
    interval: 1d
    rotate: 3
  logs/process.log:      # 对自定义的 log_process 插件产生的日志执行轮转配置
    interval: 1d
    rotate: 30
```

2. 初始化与服务重载以启用

配置编写保存后，通过下列命令重新关联 OpenResty、初始化并重载网关服务：

```
## 重新解析配置模板并初始化环境
bin/edge init

## 重载网关服务以使静态属性和后台定时器生效
bin/edge reload
```

执行完毕后，`log_rotate` 将在网关后台随着特权进程的启动自动运行，不需要任何其他的运维脚本支持。您可以在对应的日志存放目录下看到切割与压缩完成的 `.log.tar.gz` 历史归档。

log_syslog 插件使用文档

1. 插件概览

`log_syslog` 插件用于收集网关层请求与响应的访问日志，并将其通过 TCP、UDP 或 TLS 协议异步批量推送到远程的 Syslog 日志中心。这有利于企业实施集中式安全审计、流量监控以及问题排查。

主要应用场景：

- **集中式日志分析：**向 Splunk、Logstash 或 Graylog 等日志中心推送标准的访问日志。
- **特定异常流量审计：**配合条件过滤表达式，只将请求体或响应体中异常的数据（例如 5xx 响应）记录发送给 Syslog，用于故障诊断。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

通过管理接口进行全局配置（API 路径：`/edge/admin/plugin_metadata/log_syslog`），用以配置全局默认的传输连接池策略。

[!IMPORTANT]

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：`PUT`
- 请求路径：`/edge/admin/plugins/reload`

您可以在配置多个插件的数据属性后，统一执行一次热加载操作。

2.2.1 全局批处理器默认配置

字段名	类型	默认值	描述	设置示例
batch_max_size	Integer	1000	内存批处理队列的最大日志消息条数。达到限额即刻打包发送。	500
buffer_duration	Integer	60	队列未满时的最长等待投递时间（秒）。	30

2.2.2 全局基础与传输配置

字段名	类型	默认值	描述	设置示例
formats_sep	String	"\ ;"	当日志数据为数组且非 JSON 输出时，拼接各字段的分隔符。	","
header_name	String	"X-EDGE-LOG-SYSLOG"	用于排重检查的 Header 名称。	"X-Trace-Logged"
ignore_logged	Boolean	false	是否忽略在链路上已被记录过的日志，防止重复传输。	true
batch_once	Boolean	false	是否将整批日志合并为单次网络 Socket 物理发送。	true
concat_method	String	"json"	合并发送时的字符拼接方案。可选值："json"，"new_line"。	"new_line"
json	String	"cjson"	转换为 JSON 时使用的引擎名称。可选值："cjson"，"dkjson"。	"dkjson"
sock_type	String	"tcp"	套接字通信类型，支持 "tcp" 和 "udp"。	"udp"
scheme	String	"tcp"	通信方案类型。支持 "tcp"，"udp"，"tls"（带 SSL 加密的 TCP 传输）。	"tls"
host	String	"127.0.0.1"	远端 Syslog 服务器的主机域名或 IP 地址。	"syslog.corp.net"
port	Integer	514	远端 Syslog 服务器监听的端口号。	514
ssl	Boolean	false	是否启用 SSL/TLS 加密传输。	true
ssl_verify	Boolean	true	启用 TLS 时，是否严格校验远端 Syslog 的证书。	false
flush_limit	Integer	4096	向系统刷写数据的缓冲区最大字节数。	8192
drop_limit	Integer	2097152	内存溢出保护阈值（字节），如果网关到 Syslog 通信阻塞且内存堆积达到该值，新日志将被丢弃，保障网关本身不发生内存溢出。	1048576
pool_size	Integer	10	网关与远程服务之间保持的 TCP 连接池大小。	20
retry_max_times	Integer	0	发送失败的最大重试次数。	3
retry_interval	Number	0.1	失败重试的间隔时长（秒）。最小 0.001 秒。	0.2
timeout	Number	1	网络读写与建立连接的超时时间（秒）。	2
log_rfc5424	Boolean	true	是否在数据头部封装 RFC 5424 标准的 Syslog 信息头（包含优先级、设备、APP-NAME 等）。	false
log_rfc5424_name	String	"edge"	RFC 5424 头中 APP-NAME 字段的值。	"edge-gateway"

2.3 插件基础属性 (Config)

路由级配置，可覆盖全局定义的插件数据属性 (Metadata Config)。

字段名	类型	默认值	描述	设置示例
logs	String 或 Array	无	必填。目的 Syslog 服务的 URI（格式如 scheme://host:port）或 URI 列表。	["tls://syslog.corp.net:514", "udp://10.0.0.5:514"]
formats	String、Object 或 Array	见说明	自定义日志字段结构。如果不配置，HTTP 模式与 Stream 模式分别采用系统预定义的基础日志字段（参见下方说明）。	见配置示例
include_req_body	Boolean	false	是否收集并推送请求体（Request Body）。	true
include_resp_body	Boolean	false	是否收集并推送响应体（Response Body）。	true
include_resp_body_expr	Array	无	响应体收集的条件表达式。若匹配，网关才会暂存并收集响应体数据。详见 Edge 表达式列表 。	[["status", ">=", 500]]
formats_sep	String	"\ ;"	同上	","
header_name	String	"X-EDGE-LOG-SYSLOG"	同上	"X-Trace-Logged"
ignore_logged	Boolean	false	同上	true
batch_once	Boolean	false	同上	true
concat_method	String	"json"	同上	"new_line"
json	String	"cjson"	同上	"dkjson"

说明

- 请求体收集条件表达式 (include_req_body_expr)：用于控制是否收集并推送请求体。若匹配该条件表达式，网关才会读取并收集请求体（例如：[["request_length", ">", 10240]]，仅在请求体大于 10KB 时收集）。条件表达式必须符合标准的 Edge 表达式规范，详见 [Edge 表达式列表](#)。
- 全局默认目的地默认降级机制：
当提交的插件数据属性 (Metadata Config) 为 {} 或 { "logs": {} } 时，网关会自动通过默认降级机制配置一个发送到 tcp://127.0.0.1:514 的默认日志目标。其默认日志输出格式如下：
- 默认日志格式 (HTTP 模式)：
如果不指定 formats，网关将使用以下预定义变量列表依次拼接输出（默认使用 |; 分隔）：
 - req_start_time (请求开始时间，格式化为 %Y%m%d%H%M%S.%03d)
 - http_x-edge-traceid (Trace ID)
 - clientip (客户端 IP)
 - username (登录名)
 - deviceid (设备 ID)
 - udid (UDID)
 - cookie_JSESSIONID (JSESSIONID cookie 值)
 - method (HTTP 方法)
 - uri (请求 URI)
 - req_args_string (请求 Query 参数，最长截取 2048 字节)
 - http_referer (Referer 头)
 - http_cookie (Cookie 头)
 - http_user-agent (User-Agent 头)
 - req_headers (完整请求头)
 - upstream_addr (上游转发目标地址)
 - upstream_response_time (上游响应时间)
 - request_time (网关总处理耗时)
 - status (响应状态码)

19. `route_id` (匹配的路由 ID)
20. `plugin_riskinfo` (安全风险信息)
21. `plugin_riskid` (触发拦截的安全规则 ID)

- 默认日志格式 (Stream 模式):

在非 HTTP (Stream TCP/UDP) 代理模式下, 默认采用以下格式进行日志传输:

1. `host` (网关主机名)
2. `time_iso8601` (ISO8601 格式时间戳)
3. `remote_addr` (客户端连接源 IP)

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由, 为路径 `/api/log` 开启 `log_syslog` 插件。

- 日志输出目标为本地部署的 Syslog 服务: `tcp://127.0.0.1:514`。
- 自定义输出格式中仅记录: 请求时间、客户端 IP、URI、响应状态码、以及 Trace ID。
- 开启响应体备份, 并且只记录响应状态码大于等于 `500` 时的响应体数据 (`include_resp_body_expr` 规则)。

Important

调用示例中不包含真实敏感密钥, 请将占位符 `API_KEY` 替换为您实际环境中的 Admin API Key。

```
curl -i -X PUT \
-H "Content-Type: application/json" \
-H "X-API-Key: API_KEY" \
-d '{
  "uri": "/api/log",
  "plugins": {
    "log_syslog": {
      "logs": ["tcp://127.0.0.1:514"],
      "formats": {
        "time": "${req_start_time#time_format,%Y%m%d%H%M%S,%03d}",
        "client": "${clientip}",
        "uri": "${uri}",
        "status": "${status}",
        "trace_id": "${http_x-edge-traceid}"
      },
      "include_resp_body": true,
      "include_resp_body_expr": [ ["status", ">=", 500] ]
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:8811": 1
    },
    "type": "roundrobin"
  }
}' \
http://127.0.0.1:9991/edge/admin/routes/1007
```

3.2 访问网关进行验证

发送普通请求

向网关发起一次普通访问请求：

```
curl -i http://127.0.0.1:9981/api/log
```

预期 Syslog 服务端接收到的日志流输出 (单行 JSON)：

```
{"time":"20260622203000.123","client":"127.0.0.1","uri":"/api/log","status":200,"trace_id":"some-uuid-string-here"}
```

发送错误请求触发响应体记录

向网关发起能够引发上游返回 500 的异常请求：

```
curl -i http://127.0.0.1:9981/api/log?error=true
```

预期 Syslog 服务端接收到的日志流输出：

日志输出的 JSON 消息体中，会自动包含上游返回的响应体明文信息：

```
{"time":"20260622203005.456","client":"127.0.0.1","uri":"/api/log","status":500,"trace_id":"another-uuid-string","response":{"body":"Internal Server Error Detail Message"}}
```

检查本地失败日志文件 (若远端连通失败)

若网关无法连通配置的 Syslog 服务器（如 127.0.0.1:514 端口未开启），由于启用了批处理失败保护，网关会将日志降级写入本地：

```
tail -n 5 logs/log_syslog.failed.log
```

预期本地文件输出：

文件中将包含对应的日志字符串明文，以防止日志丢失。

mocking 使用文档

插件概览

`mocking` 插件用于在 Edge 网关层拦截请求并直接返回预定义的模拟响应。它支持返回固定内容，也支持根据 JSON Schema 动态生成随机响应，并允许自定义状态码、响应头及响应延迟。

配置详情

插件静态属性 (Attr Config)

无。

插件数据属性 (Metadata Config)

全局默认配置。

- 接口路径: `/edge/admin/plugin_metadata/mocking`
- 属性列表:

字段名	类型	默认值	描述	设置示例
delay	Number	0	响应延迟时间（秒）。	0.5
status	Integer	200	拦截响应的状态码 (200-599)。	201
content	String/Object	无	固定响应内容。支持字符串或 JSON 对象。支持变量。	{"msg": "success"}
content_schema	Object	无	JSON Schema 定义。若配置则根据 Schema 生成随机数据。	{"type": "object", "properties": {"id": {"type": "integer"}}}
content_type	String	application/json;charset=utf8	响应的 Content-Type。	application/xml
headers	Object	无	自定义响应头，支持变量。	{"X-Server": "Edge"}
header_name	String	X-Mock-By	Mock 标识头的名称。	X-Edge-Mock
show_mock_header	Boolean	false	是否在响应中包含 Mock 标识头。	true
xml_root_tag	String	data	当 Content-Type 为 XML 且使用 Schema 时，XML 的根节点名称。	root

- 设置示例:

```
curl -L -X PUT 'http://[ADMIN_ADDR]/edge/admin/plugin_metadata/mocking' \
-H 'X-API-KEY: api-key' \
-H 'Content-Type: application/json' \
-d '{
  "header_name": "X-Edge-Mock",
  "show_mock_header": true,
  "delay": 0.1
}'
```

插件基础属性 (Config)

同上。

注意事项

1. 优先级: content 的优先级高于 content_schema。若两者同时存在，优先使用 content。
2. Schema 支持: content_schema 自动生成功能目前仅在 content_type 为 application/json 或 application/xml（及其变体）时生效。
3. 路由覆盖: 路由级的 Config 属性将完全覆盖 Metadata 中对应的同名字段。
4. 变量支持: 插件支持在 content 及 headers 中使用动态变量。变量列表请参考 [Edge 变量 \(包含 Nginx 变量\)](#)。

调用与验证示例

1. 获取固定 Mock 响应

配置路由 (启用插件):

```
curl -L -X PUT 'http://[ADMIN_ADDR]/edge/admin/routes/3001' \
-H 'X-API-KEY: api-key' \
-H 'Content-Type: application/json' \
-d '{
  "uri": "/api/mock",
  "plugins": {
    "mocking": {
      "status": 200,
      "content": "{\"path\": \"${uri}\", \"msg\": \"fixed content\"}"
    }
  },
}
```

```
"upstream": {
  "nodes": {
    "[UPSTREAM_ADDR]": 1
  }
}
```

请求与验证:

```
curl -i http://[GATEWAY_ADDR]/api/mock
## 预期响应:
## HTTP/1.1 200 OK
## Content-Type: application/json;charset=utf8
## X-Edge-Mock: EDGE/x.y.z
## {"path": "/api/mock", "msg": "fixed content"}
```

2. 获取 Schema 随机响应

配置路由 (启用插件):

```
curl -L -X PUT 'http://[ADMIN_ADDR]/edge/admin/routes/3002' \
-H 'X-API-KEY: api-key' \
-H 'Content-Type: application/json' \
-d '{
  "uri": "/api/random",
  "plugins": {
    "mocking": {
      "status": 200,
      "content_schema": {
        "type": "object",
        "properties": {
          "id": { "type": "integer" },
          "name": { "type": "string" }
        }
      }
    }
  },
  "upstream": {
    "nodes": {
      "[UPSTREAM_ADDR]": 1
    }
  }
}
```

请求与验证:

```
curl -i http://[GATEWAY_ADDR]/api/random
## 预期响应 (示例):
## {"id": 1234, "name": "abcde"}
```

monitor 使用文档

插件概览

`monitor` 插件是网关层提供的监控与度量数据收集插件。在路由上启用此插件后，网关会实时监控处理的 HTTP 请求或 Stream TCP 流量，并自动收集带宽吞吐、耗时分布、状态码等关键度量指标。收集到的指标主要支持通过 Prometheus API 拉取并在 Grafana 中进行可视化展示。

配置详情

1. 插件静态属性 (Attr Config)

静态属性定义在网关主部署配置文件 `edge.env` 的 `plugin_attr` 下的对应插件名属性中。

字段名	类型	默认值	描述	设置示例
<code>metric_prefix</code>	String	<code>edge_</code>	导出的监控指标前缀。	<code>"edge_"</code>
<code>metric_sync_interval</code>	Integer	<code>1</code>	指标从共享内存向 Prometheus 注册表同步的间隔时间（单位：秒）。最低为 1。	<code>5</code>
<code>metric_buckets</code>	Array	<code>[10, 20, 50, 100, 200, 500, 1000, 2000, 5000, 10000]</code>	耗时统计（直方图）的耗时区间桶定义，以毫秒为单位。	<code>[10, 50, 100, 500, 1000]</code>

2. 插件数据属性 (Metadata Config)

- 无

3. 插件基础属性 (Config)

基础属性是路由级配置，当为指定路由启用该插件时进行配置。

字段名	类型	默认值	描述	设置示例
<code>prefer_route_name</code>	Boolean	<code>false</code>	是否使用路由或服务的名称（Name）来标识命中的路由或服务。若为 <code>false</code> ，则使用内部 ID 标识。	<code>true</code>
<code>prefer_restrict_uri</code>	Boolean	<code>true</code>	监控中记录的匹配 URI 是否优先展示为 URI 白名单插件允许的 URI 地址，设置为 <code>false</code> 则仅使用路由配置的 URI 地址。	<code>true</code>
<code>plugin_restrict_uri</code>	String Array	<code>"^security_restrict_uri"</code>	当 <code>prefer_restrict_uri</code> 开启时，触发拦截匹配的插件正则模式或插件名称列表。	<code>"^security_restrict_uri"</code>
<code>custom_uri</code>	String	无	用于统计的自定义 URI，支持网关内置动态变量（如 <code>\${uri}</code> 等）。	<code>"\${uri}"</code>

注意事项

- 命名规范：**如果在路由级基础配置中将 `prefer_route_name` 设为 `true`，建议对网关中所有的路由和服务进行统一、规范的英文或拼音命名。这是因为路由和服务名称在系统中没有强唯一性限制，不规范的命名可能会导致指标合并后难以区分。
- 变量支持：**`custom_uri` 字段支持使用网关内置变量，常用的变量如 `${uri}`（请求 URI 路径）、`${host}`（请求主机名）等。完整变量支持详情请引用：[Edge 变量列表](#)。

监控指标列表

启用 `monitor` 插件后，网关将向 Prometheus 导出并提供以下关键度量数据指标以供查看和绘图：

HTTP 阶段监控指标

- `edge_bandwidth`：服务消耗的物理总流量大小（单位：字节）。可以通过 `type="ingress"`（入流量）和 `type="egress"`（出流量）区分流量方向。
- `edge_http_requests_total`：自网关启动以来累计处理的客户端 HTTP 请求总量。
- `edge_http_status`：响应状态码统计，可通过 `code` 标签（如 200、404、502 等）对各路由和 `matched_uri` 的命中频次进行详细观测。
- `edge_http_latency`：HTTP 请求各类耗时的分布直方图（单位：毫秒）。可通过 `type` 区分总耗时（`request`）、后端连接耗时（`upstream`）及网关自身开销（`edge`）等。导出的直方图指标包含以下三项：
 - `edge_http_latency_bucket`：耗时计数，统计 10, 20, 50, 100, 200, 500, 1000, 2000, 5000, 10000ms 以及大于 10000ms 的请求耗时分布数。
 - `edge_http_latency_count`：各耗时来源的请求计数。
 - `edge_http_latency_sum`：各耗时来源的累计总请求耗时。
- `edge_nginx_http_current_connections`：网关底层 Nginx 的当前各种连接状态数据统计（如活跃连接、等待连接等）。
- `edge_shared_dict_capacity_bytes` / `edge_shared_dict_free_space_bytes`：网关内部各个共享内存字典的总体容量与剩余空闲空间监控。
- `edge_upstream_status`：各个后端上游节点健康检查状态的存活率指标。
- `edge_plugin_risk` / `edge_plugin_riskid`：网关各防守安全插件的累计拦截统计，支持下钻细化到具体的 WAF 规则 ID。
- `edge_plugin_latency`：各具体业务逻辑插件在处理链中的实际执行开销耗时分布直方图。

L4 Stream 阶段监控指标

- `edge_stream_connection_total`：每个 L4 代理路由的累计连接次数计数。

调用与验证示例

1. 启用插件配置示例

我们可以通过管理节点提供的 RESTful 接口更新路由配置以启用 `monitor` 插件。以下是一个完整的更新路由 `3001` 配置的示例：

注意：实际调用管理接口时，请求体需要先经过 SM4 加密，返回结果也需要进行解密，详见 [管理接口调用规范](#)。

```
curl -L -X PUT 'http://127.0.0.1:9990/edge/admin/routes/3001' \
-H 'X-API-KEY: API_KEY' \
-H 'Content-Type: application/json' \
-d '{
  "uri": "/hello",
  "plugins": {
    "monitor": {
      "prefer_route_name": true
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:8111": 1
    },
    "type": "roundrobin"
  }
}'
```

① Note

请根据实际部署环境将 `API_KEY` 替换为您管理节点真实的 API Key 授权凭证。

2. 请求验证与查看监控指标

第一步：向已配置的路由发送多次流量请求

```
curl -i http://127.0.0.1:9980/hello
```

第二步：获取 Prometheus 指标数据

网关会在 `log` 阶段自动收集性能指标，可通过管理端口访问 Prometheus 导出端点进行查看：

```
curl -s http://127.0.0.1:9990/edge/monitor/metrics
```

预期指标响应完整展示：

```
## HELP edge_bandwidth Total bandwidth in bytes consumed per service in EDGE
## TYPE edge_bandwidth counter
edge_bandwidth{type="egress",route="3001",service="",consumer="",node="127.0.0.1:8111"} 8964
edge_bandwidth{type="ingress",route="3001",service="",consumer="",node="127.0.0.1:8111"} 738
## HELP edge_http_latency HTTP request latency in milliseconds per service in EDGE
## TYPE edge_http_latency histogram
edge_http_latency_bucket{type="client_recv",route="3001",service="",consumer="",node="127.0.0.1:8111",le="10"} 9
edge_http_latency_bucket{type="client_recv",route="3001",service="",consumer="",node="127.0.0.1:8111",le="20"} 9
edge_http_latency_bucket{type="client_recv",route="3001",service="",consumer="",node="127.0.0.1:8111",le="50"} 9
edge_http_latency_bucket{type="client_recv",route="3001",service="",consumer="",node="127.0.0.1:8111",le="100"} 9
edge_http_latency_bucket{type="client_recv",route="3001",service="",consumer="",node="127.0.0.1:8111",le="200"} 9
edge_http_latency_bucket{type="client_recv",route="3001",service="",consumer="",node="127.0.0.1:8111",le="500"} 9
edge_http_latency_bucket{type="client_recv",route="3001",service="",consumer="",node="127.0.0.1:8111",le="1000"} 9
edge_http_latency_bucket{type="client_recv",route="3001",service="",consumer="",node="127.0.0.1:8111",le="2000"} 9
edge_http_latency_bucket{type="client_recv",route="3001",service="",consumer="",node="127.0.0.1:8111",le="5000"} 9
edge_http_latency_bucket{type="client_recv",route="3001",service="",consumer="",node="127.0.0.1:8111",le="10000"} 9
edge_http_latency_bucket{type="client_recv",route="3001",service="",consumer="",node="127.0.0.1:8111",le="+Inf"} 9
edge_http_latency_bucket{type="client_send",route="3001",service="",consumer="",node="127.0.0.1:8111",le="10"} 9
edge_http_latency_bucket{type="client_send",route="3001",service="",consumer="",node="127.0.0.1:8111",le="20"} 9
edge_http_latency_bucket{type="client_send",route="3001",service="",consumer="",node="127.0.0.1:8111",le="50"} 9
edge_http_latency_bucket{type="client_send",route="3001",service="",consumer="",node="127.0.0.1:8111",le="100"} 9
edge_http_latency_bucket{type="client_send",route="3001",service="",consumer="",node="127.0.0.1:8111",le="200"} 9
```

[illegible]

```

edge_http_latency_bucket{type="upstream",route="3001",service="",consumer="",node="127.0.0.1:8111",le="20"} 9
edge_http_latency_bucket{type="upstream",route="3001",service="",consumer="",node="127.0.0.1:8111",le="50"} 9
edge_http_latency_bucket{type="upstream",route="3001",service="",consumer="",node="127.0.0.1:8111",le="100"} 9
edge_http_latency_bucket{type="upstream",route="3001",service="",consumer="",node="127.0.0.1:8111",le="200"} 9
edge_http_latency_bucket{type="upstream",route="3001",service="",consumer="",node="127.0.0.1:8111",le="500"} 9
edge_http_latency_bucket{type="upstream",route="3001",service="",consumer="",node="127.0.0.1:8111",le="1000"} 9
edge_http_latency_bucket{type="upstream",route="3001",service="",consumer="",node="127.0.0.1:8111",le="2000"} 9
edge_http_latency_bucket{type="upstream",route="3001",service="",consumer="",node="127.0.0.1:8111",le="5000"} 9
edge_http_latency_bucket{type="upstream",route="3001",service="",consumer="",node="127.0.0.1:8111",le="10000"} 9
edge_http_latency_bucket{type="upstream",route="3001",service="",consumer="",node="127.0.0.1:8111",le="+Inf"} 9
edge_http_latency_count{type="client_recv",route="3001",service="",consumer="",node="127.0.0.1:8111"} 9
edge_http_latency_count{type="client_send",route="3001",service="",consumer="",node="127.0.0.1:8111"} 9
edge_http_latency_count{type="edge",route="3001",service="",consumer="",node="127.0.0.1:8111"} 9
edge_http_latency_count{type="request",route="3001",service="",consumer="",node="127.0.0.1:8111"} 9
edge_http_latency_count{type="upstream",route="3001",service="",consumer="",node="127.0.0.1:8111"} 9
edge_http_latency_max{type="client_recv",route="3001",service="",consumer="",node="127.0.0.1:8111"} 0
edge_http_latency_max{type="client_send",route="3001",service="",consumer="",node="127.0.0.1:8111"} 0
edge_http_latency_max{type="edge",route="3001",service="",consumer="",node="127.0.0.1:8111"} 0
edge_http_latency_max{type="request",route="3001",service="",consumer="",node="127.0.0.1:8111"} 2
edge_http_latency_max{type="upstream",route="3001",service="",consumer="",node="127.0.0.1:8111"} 2
edge_http_latency_min{type="client_recv",route="3001",service="",consumer="",node="127.0.0.1:8111"} 0
edge_http_latency_min{type="client_send",route="3001",service="",consumer="",node="127.0.0.1:8111"} 0
edge_http_latency_min{type="edge",route="3001",service="",consumer="",node="127.0.0.1:8111"} 0
edge_http_latency_min{type="request",route="3001",service="",consumer="",node="127.0.0.1:8111"} 0
edge_http_latency_min{type="upstream",route="3001",service="",consumer="",node="127.0.0.1:8111"} 1
edge_http_latency_sum{type="client_recv",route="3001",service="",consumer="",node="127.0.0.1:8111"} 0
edge_http_latency_sum{type="client_send",route="3001",service="",consumer="",node="127.0.0.1:8111"} 0
edge_http_latency_sum{type="edge",route="3001",service="",consumer="",node="127.0.0.1:8111"} 0
edge_http_latency_sum{type="request",route="3001",service="",consumer="",node="127.0.0.1:8111"} 9
edge_http_latency_sum{type="upstream",route="3001",service="",consumer="",node="127.0.0.1:8111"} 10
## HELP edge_http_requests_total The total number of client requests since EDGE started
## TYPE edge_http_requests_total gauge
edge_http_requests_total 18
## HELP edge_http_status HTTP status codes per service in EDGE
## TYPE edge_http_status counter
edge_http_status{code="200",route="3001",matched_uri="/hello",matched_host="",service="",consumer="",node="127.0.0.1:8111"} 9
## HELP edge_metric_errors_total Number of metric errors
## TYPE edge_metric_errors_total counter
edge_metric_errors_total 0

```

```
## HELP edge_nginx_http_current_connections Number of HTTP connections
## TYPE edge_nginx_http_current_connections gauge
edge_nginx_http_current_connections{state="accepted"} 18
edge_nginx_http_current_connections{state="active"} 1
edge_nginx_http_current_connections{state="handled"} 18
edge_nginx_http_current_connections{state="reading"} 0
edge_nginx_http_current_connections{state="waiting"} 0
edge_nginx_http_current_connections{state="writing"} 1
## HELP edge_node_info Info of EDGE node
## TYPE edge_node_info gauge
edge_node_info{hostname="zengbiao.local",version="3.1.1.26060311"} 1
## HELP edge_shared_dict_capacity_bytes The capacity of each nginx shared DICT since EDGE start
## TYPE edge_shared_dict_capacity_bytes gauge
edge_shared_dict_capacity_bytes{name="admin_locks"} 10485760
edge_shared_dict_capacity_bytes{name="balancer_ewma"} 10485760
edge_shared_dict_capacity_bytes{name="balancer_ewma_last_touched_at"} 10485760
edge_shared_dict_capacity_bytes{name="balancer_ewma_locks"} 10485760
edge_shared_dict_capacity_bytes{name="cluster_health_check"} 10485760
edge_shared_dict_capacity_bytes{name="cluster_health_check_locks"} 10485760
edge_shared_dict_capacity_bytes{name="cluster_health_check_status"} 10485760
edge_shared_dict_capacity_bytes{name="discovery_nacos"} 10485760
edge_shared_dict_capacity_bytes{name="discovery_tars"} 10485760
edge_shared_dict_capacity_bytes{name="fdict_cache"} 104857600
edge_shared_dict_capacity_bytes{name="fdict_cache_locks"} 10485760
edge_shared_dict_capacity_bytes{name="fdict_cache_version"} 1048576
edge_shared_dict_capacity_bytes{name="file_locks"} 10485760
edge_shared_dict_capacity_bytes{name="internal_status"} 10485760
edge_shared_dict_capacity_bytes{name="lrucache_lock"} 10485760
edge_shared_dict_capacity_bytes{name="memory_cache"} 104857600
edge_shared_dict_capacity_bytes{name="monitor_metrics"} 209715200
edge_shared_dict_capacity_bytes{name="monitor_metrics1"} 52428800
edge_shared_dict_capacity_bytes{name="monitor_metrics2"} 52428800
edge_shared_dict_capacity_bytes{name="monitor_metricsn_locks"} 10485760
edge_shared_dict_capacity_bytes{name="monitor_performance"} 31457280
edge_shared_dict_capacity_bytes{name="monitor_security"} 31457280
edge_shared_dict_capacity_bytes{name="monitor_status"} 31457280
edge_shared_dict_capacity_bytes{name="plugin_breaker"} 104857600
edge_shared_dict_capacity_bytes{name="plugin_breaker_limit"} 104857600
edge_shared_dict_capacity_bytes{name="plugin_breaker_locks"} 10485760
edge_shared_dict_capacity_bytes{name="plugin_limit_locks"} 10485760
edge_shared_dict_capacity_bytes{name="plugin_limit_status"} 52428800
edge_shared_dict_capacity_bytes{name="plugin_remote_redis"} 209715200
edge_shared_dict_capacity_bytes{name="plugin_security_fengkong"} 104857600
edge_shared_dict_capacity_bytes{name="plugin_security_fengkong_env"} 104857600
edge_shared_dict_capacity_bytes{name="plugin_security_restrict_redis"} 104857600
edge_shared_dict_capacity_bytes{name="plugin_security_super_redis"} 104857600
edge_shared_dict_capacity_bytes{name="plugin_traffic_limit_count"} 209715200
edge_shared_dict_capacity_bytes{name="plugin_traffic_limit_count_locks"} 10485760
edge_shared_dict_capacity_bytes{name="redis_cluster_locks"} 10485760
edge_shared_dict_capacity_bytes{name="redis_pool"} 10485760
edge_shared_dict_capacity_bytes{name="redis_pool_locks"} 10485760
edge_shared_dict_capacity_bytes{name="redis_pool_status"} 52428800
edge_shared_dict_capacity_bytes{name="upstream_healthcheck"} 10485760
edge_shared_dict_capacity_bytes{name="worker_events"} 10485760
## HELP edge_shared_dict_free_space_bytes The free space of each nginx shared DICT since EDGE start
## TYPE edge_shared_dict_free_space_bytes gauge
edge_shared_dict_free_space_bytes{name="admin_locks"} 10412032
edge_shared_dict_free_space_bytes{name="balancer_ewma"} 10412032
```

```
edge_shared_dict_free_space_bytes{name="balancer_ewma_last_touched_at"} 10412032
edge_shared_dict_free_space_bytes{name="balancer_ewma_locks"} 10412032
edge_shared_dict_free_space_bytes{name="cluster_health_check"} 10412032
edge_shared_dict_free_space_bytes{name="cluster_health_check_locks"} 10412032
edge_shared_dict_free_space_bytes{name="cluster_health_check_status"} 10412032
edge_shared_dict_free_space_bytes{name="discovery_nacos"} 10412032
edge_shared_dict_free_space_bytes{name="discovery_tars"} 10412032
edge_shared_dict_free_space_bytes{name="fdict_cache"} 104230912
edge_shared_dict_free_space_bytes{name="fdict_cache_locks"} 10412032
edge_shared_dict_free_space_bytes{name="fdict_cache_version"} 1032192
edge_shared_dict_free_space_bytes{name="file_locks"} 10412032
edge_shared_dict_free_space_bytes{name="internal_status"} 10407936
edge_shared_dict_free_space_bytes{name="lrucache_lock"} 10412032
edge_shared_dict_free_space_bytes{name="memory_cache"} 104235008
edge_shared_dict_free_space_bytes{name="monitor_metrics"} 208396288
edge_shared_dict_free_space_bytes{name="monitor_metrics1"} 52113408
edge_shared_dict_free_space_bytes{name="monitor_metrics2"} 52113408
edge_shared_dict_free_space_bytes{name="monitor_metricsn_locks"} 10412032
edge_shared_dict_free_space_bytes{name="monitor_performance"} 31264768
edge_shared_dict_free_space_bytes{name="monitor_security"} 31264768
edge_shared_dict_free_space_bytes{name="monitor_status"} 31264768
edge_shared_dict_free_space_bytes{name="plugin_breaker"} 104235008
edge_shared_dict_free_space_bytes{name="plugin_breaker_limit"} 104235008
edge_shared_dict_free_space_bytes{name="plugin_breaker_locks"} 10412032
edge_shared_dict_free_space_bytes{name="plugin_limit_locks"} 10412032
edge_shared_dict_free_space_bytes{name="plugin_limit_status"} 52113408
edge_shared_dict_free_space_bytes{name="plugin_remote_redis"} 208482304
edge_shared_dict_free_space_bytes{name="plugin_security_fengkong"} 104235008
edge_shared_dict_free_space_bytes{name="plugin_security_fengkong_env"} 104235008
edge_shared_dict_free_space_bytes{name="plugin_security_restrict_redis"} 104235008
edge_shared_dict_free_space_bytes{name="plugin_security_super_redis"} 104235008
edge_shared_dict_free_space_bytes{name="plugin_traffic_limit_count"} 208482304
edge_shared_dict_free_space_bytes{name="plugin_traffic_limit_count_locks"} 10412032
edge_shared_dict_free_space_bytes{name="redis_cluster_locks"} 10412032
edge_shared_dict_free_space_bytes{name="redis_pool"} 10412032
edge_shared_dict_free_space_bytes{name="redis_pool_locks"} 10412032
edge_shared_dict_free_space_bytes{name="redis_pool_status"} 52113408
edge_shared_dict_free_space_bytes{name="upstream_healthcheck"} 10412032
edge_shared_dict_free_space_bytes{name="worker_events"} 10407936
```

plugin_demo 使用文档

配置说明

1. 数据属性配置 (Metadata)

配置默认拦截行为：

```
curl -X PUT http://admin:9991/edge/admin/plugin_metadata/plugin_demo \
-H "Content-Type: application/json" \
-d '{
  "status": 403,
  "policy": "local"
}'
```

2. 路由级配置 (Config)

在具体路由上启用并自定义消息：

- **status:** 403
- **message:** "Access Denied by Demo Plugin"
- **group:** "security"

调用示例

正常请求

```
curl -i http://localhost:9981/api/test
## 返回 200 OK
```

模拟拦截请求

设置 `http_deny` 标志（示例逻辑）：

```
curl -i http://localhost:9981/api/test -H "X-Deny: yes"
## 返回 403 Your request is not allowed. (取决于具体环境配置)
```

post_functions 插件使用文档

1. 插件概览

`post_functions` 插件允许用户在网关请求周期的各个阶段之后（如 `rewrite` 完毕后、`access` 完毕后等）执行自定义的 Lua 代码。由于其执行时机位于末尾，通常用于统计分析、性能打点、或提取请求处理结果注入审计日志等场景。

主要应用场景：

- **后置耗时打点：** 测量在完成该路由所有上游和业务计算后进入记录日志前的指标。
- **动态修改响应体：** 在 `body_filter` 之后将全部流式响应数据收集并统一进行某种格式处理。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

无。

2.3 插件基础属性 (Config)

路由级配置。

字段名	类型	默认值	描述	设置示例
<code>preread</code>	Array of String	无	在 <code>preread</code> 阶段执行的 Lua 代码块列表。	<code>["return function(conf, ctx) ngx.log(ngx.INFO, \"preread phase\") end"]</code>
<code>rewrite</code>	Array of String	无	在 <code>rewrite</code> 阶段执行的 Lua 代码块列表。	<code>["return function(conf, ctx) return 403, \"Forbidden\" end"]</code>
<code>access</code>	Array of String	无	在 <code>access</code> 阶段执行的 Lua 代码块列表。	<code>["return function(conf, ctx) ngx.log(ngx.INFO, \"access phase\") end"]</code>
<code>before_proxy</code>	Array of String	无	在 <code>before_proxy</code> 阶段执行的 Lua 代码块列表。	<code>["return function(conf, ctx) ngx.log(ngx.INFO, \"before_proxy phase\") end"]</code>
<code>header_filter</code>	Array of String	无	在 <code>header_filter</code> 阶段执行的 Lua 代码块列表。	<code>["return function(conf, ctx) ngx.header[\"X-Custom-Header\"] = \"Post\" end"]</code>
<code>body_filter</code>	Array of String	无	在 <code>body_filter</code> 阶段执行的 Lua 代码块列表。	<code>["return function(conf, ctx) ngx.arg[1] = string.upper(ngx.arg[1] or \"\") end"]</code>
<code>log</code>	Array of String	无	在 <code>log</code> 阶段执行的 Lua 代码块列表。	<code>["return function(conf, ctx) ngx.log(ngx.INFO, \"log phase\") end"]</code>

说明

- 配置的 Lua 代码块必须使用 `return` 语句返回一个接收 `(conf, ctx)` 参数的 Lua 函数（如 `return function(conf, ctx) ... end`）。

3. 调用与验证示例

本示例介绍如何创建启用 `post_functions` 插件的路由，实现在 `rewrite` 阶段之后拦截非法请求并返回 `403 Forbidden` 的功能。

Important

调用示例中不包含真实敏感密钥，请将占位符 `API_KEY` 替换为您实际环境中的 Admin API Key。

3.1 步骤 1：创建启用 `post_functions` 插件的路由

创建路由关联路径 `/test_post`，在插件配置的 `rewrite` 数组中填入自定义拦截代码。

```
curl -i -X PUT \  
-H "Content-Type: application/json" \  
-H "X-API-Key: API_KEY" \  
-d '{  
  "uri": "/test_post",  
  "plugins": {  
    "post_functions": {  
      "rewrite": [  
        "return function(conf, ctx) if ngx.var.http_x_debug == \"block\" then return 403,  
        \"Blocked by post_functions\" end end"  
      ]  
    },  
  },  
  "upstream": {  
    "nodes": {  
      "127.0.0.1:8811": 1  
    },  
    "type": "roundrobin"  
  }  
}' \  

```



```
http://127.0.0.1:9991/edge/admin/routes/3002
```

3.2 访问网关进行验证

1. 正常访问（未触发拦截条件）

向网关发起不带特定 Header 的请求：

```
curl -i http://127.0.0.1:9981/test_post
```

预期响应输出：

```
HTTP/1.1 200 OK
Content-Type: text/plain
Connection: keep-alive
...
```

2. 携带拦截标头访问（预期触发 403 拦截）

```
curl -i -H "X-Debug: block" http://127.0.0.1:9981/test_post
```

预期响应输出：

```
HTTP/1.1 403 Forbidden
Content-Length: 27
Content-Type: text/plain
Connection: keep-alive

Blocked by post_functions
```

pre_functions 插件使用文档

1. 插件概览

`pre_functions` 插件允许用户在网关请求周期的各个阶段之前执行自定义的 Lua 代码。它提供了一种极为灵活的机制，供用户在线上环境免重构、免重载直接注入定制逻辑。

主要应用场景：

- **前置卡控与过滤**：在网关的最前端对特定的请求头、参数进行解析，提前拒绝非法的请求（如返回 403）。
- **动态 Header 注入**：根据自定义规则，动态地在请求或响应中插入调试标识 Header。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

无。

2.3 插件基础属性 (Config)

路由级配置。

字段名	类型	默认值	描述	设置示例
<code>preread</code>	Array of String	无	在 <code>preread</code> 阶段执行的 Lua 代码块列表。	<code>["return function(conf, ctx) ngx.log(ngx.INFO, \"preread phase\") end"]</code>
<code>rewrite</code>	Array of String	无	在 <code>rewrite</code> 阶段执行的 Lua 代码块列表。	<code>["return function(conf, ctx) return 403, \"Forbidden\" end"]</code>
<code>access</code>	Array of String	无	在 <code>access</code> 阶段执行 of Lua 代码块列表。	<code>["return function(conf, ctx) ngx.log(ngx.INFO, \"access phase\") end"]</code>
<code>before_proxy</code>	Array of String	无	在 <code>before_proxy</code> 阶段执行 of Lua 代码块列表。	<code>["return function(conf, ctx) ngx.log(ngx.INFO, \"before_proxy phase\") end"]</code>
<code>header_filter</code>	Array of String	无	在 <code>header_filter</code> 阶段执行 of Lua 代码块列表。	<code>["return function(conf, ctx) ngx.header[\"X-Custom-Header\"] = \"Pre\" end"]</code>
<code>body_filter</code>	Array of String	无	在 <code>body_filter</code> 阶段执行 of Lua 代码块列表。	<code>["return function(conf, ctx) ngx.arg[1] = string.upper(ngx.arg[1] or \"\") end"]</code>
<code>log</code>	Array of String	无	在 <code>log</code> 阶段执行 of Lua 代码块列表。	<code>["return function(conf, ctx) ngx.log(ngx.INFO, \"log phase\") end"]</code>

说明

- 配置的 Lua 代码块必须使用 `return` 语句返回一个接收 `(conf, ctx)` 参数的 Lua 函数（如 `return function(conf, ctx) ... end`）。
- 自定义代码块应尽可能保持轻量，避免阻塞式的 I/O 操作。

3. 调用与验证示例

本示例介绍如何创建启用 `pre_functions` 插件的路由，实现在 `rewrite` 阶段拦截非法请求并返回 `403 Forbidden` 的功能。

Important

调用示例中不包含真实敏感密钥，请将占位符 `API_KEY` 替换为您实际环境中的 Admin API Key。

3.1 步骤 1：创建启用 `pre_functions` 插件的路由

创建路由由关联路径 `/test_pre`，在插件配置的 `rewrite` 数组中填入自定义拦截代码。

```
curl -i -X PUT \  
  -H "Content-Type: application/json" \  
  -H "X-API-Key: API_KEY" \  
  -d '{  
    "uri": "/test_pre",  
    "plugins": {  
      "pre_functions": {  
        "rewrite": [  
          "return function(conf, ctx) if ngx.var.http_x_debug == \"block\" then return 403, \"Blocked by pre_functions\" end end"  
        ]  
      }  
    },  
    "upstream": {  
      "nodes": {
```

```
    "127.0.0.1:8811": 1
  },
  "type": "roundrobin"
}
}' \
http://127.0.0.1:9991/edge/admin/routes/3001
```

3.2 访问网关进行验证

1. 正常访问（未触发拦截条件）

向网关发起不带特定 Header 的请求：

```
curl -i http://127.0.0.1:9981/test_pre
```

预期响应输出：

```
HTTP/1.1 200 OK
Content-Type: text/plain
Connection: keep-alive
...
```

2. 携带拦截标头访问（预期触发 403 拦截）

```
curl -i -H "X-Debug: block" http://127.0.0.1:9981/test_pre
```

预期响应输出：

```
HTTP/1.1 403 Forbidden
Content-Length: 26
Content-Type: text/plain
Connection: keep-alive

Blocked by pre_functions
```

proxy_cache 插件使用文档

1. 插件概览

`proxy_cache` 插件为网关提供响应缓存能力，支持将上游的 HTTP 响应缓存在本地磁盘或共享内存中，并在后续相同请求命中时直接返回缓存内容，从而大幅减少对上游的透传请求，降低后端压力，提升响应速度。

主要应用场景：

- **静态资源加速**：对图片、JS、CSS 等内容进行本地磁盘缓存，避免重复向后端拉取。
- **热点 API 缓存**：对高频读接口的响应进行内存缓存，实现毫秒级命中。
- **降低上游负载**：在流量高峰期，缓存层吸收大量重复请求，保障上游服务稳定。

2. 配置详情

2.1 插件静态属性 (Attr Config)

通过 `edge.env` 中的 `plugin_attr.proxy_cache` 配置缓存区域（Zone），可定义多个磁盘或内存缓存区。

字段名	类型	默认值	描述	设置示例
<code>valid</code>	String / Integer	<code>"10s"</code>	默认全局缓存有效时长。如果是整数，最小值为 <code>1</code> ，单位为秒；如果是字符串，支持带有时间单位的格式，具体见下方说明。	<code>"10s"</code>
<code>zones</code>	Array<Object>	无	缓存区域列表，每个元素定义一个命名缓存区。必须至少包含 1 项，且每项必须唯一。	见下方说明

说明

- `valid` 属性的字符串时间单位格式：
 - 秒：`s`（例如：`30s` 表示 30 秒）
 - 分钟：`m`（例如：`5m` 表示 5 分钟）
 - 小时：`h`（例如：`2h` 表示 2 小时）
 - 天：`d`（例如：`6d` 表示 6 天）
- `zones` 数组中每个元素为一个缓存区定义，其字段如下：

子字段	类型	描述
<code>name</code>	String	缓存区域名称（必填），对应基础属性中的 <code>cache_zone</code> 。
<code>memory_size</code>	String / Integer	内存共享字典大小（支持单位见下方大小说明）。与 <code>disk_path</code> 二选一；设置此项则为内存缓存区。
<code>disk_path</code>	String	磁盘缓存目录路径（如 <code>"/tmp/edge_cache"</code> ）。与 <code>memory_size</code> 二选一；设置此项则为磁盘缓存区。
<code>disk_size</code>	String / Integer	磁盘缓存最大容量（支持单位见下方大小说明），仅在 <code>disk_path</code> 模式下有效。
<code>disk_levels</code>	String	磁盘目录层级结构（如 <code>"1:2"</code> ），仅在 <code>disk_path</code> 模式下有效。

- 存储大小单位限制说明（适用于 `memory_size` 与 `disk_size`）：
 - 字节：默认单位。如果只写纯数字（如 `512`），则默认表示 512 字节。
 - 千字节：`k` 或 `K`（例如：`10k` 或 `10K`）
 - 兆字节：`m` 或 `M`（例如：`10m` 或 `10M`）
 - 千兆字节：`g` 或 `G`（例如：`10g` 或 `10G`）

`edge.env` 配置示例：

```
plugin_attr:
  proxy_cache:
    zones:
      - name: edge_proxy_cache1
        disk_path: /tmp/edge_disk_cache
        disk_size: 1G
        disk_levels: "1:2"
      - name: memory_cache
        memory_size: 50m
```

2.2 插件数据属性 (Metadata Config)

无。

2.3 插件基础属性 (Config)

路由级配置，在路由的 `plugins` 节点中配置 `proxy_cache` 对象。

字段名	类型	默认值	描述	设置示例
<code>cache_type</code>	String	<code>"disk"</code>	缓存存储类型，支持 <code>"disk"</code> （磁盘缓存）和 <code>"memory"</code> （内存缓存）。	<code>"memory"</code>
<code>cache_zone</code>	String	<code>"edge_proxy_cache1"</code> (disk) / <code>"memory_cache"</code> (memory)	缓存区域名称，需与静态属性中定义的 <code>zones[].name</code> 对应，且类型须与 <code>cache_type</code> 匹配。	<code>"edge_proxy_cache1"</code>
<code>cache_key</code>	String / Array<String>	<code>"\${host},\${request_uri}"</code>	缓存键，支持单个字符串或最多 8 个字符串组成的数组。支持 Edge 变量插值（如 <code>\${host}</code> 、 <code>\${uri}</code> ）。	<code>"\${host},\${uri}"</code>
<code>cache_method</code>	Array<String>	<code>["GET", "HEAD"]</code>	允许缓存的 HTTP 请求方法，枚举值为 <code>GET</code> 、 <code>POST</code> 、 <code>HEAD</code> 。	<code>["GET", "POST"]</code>
<code>cache_http_status</code>	Array<Integer>	<code>[200, 301, 404]</code>	允许缓存的上游响应状态码列表，取值范围 200 ~ 599。	<code>[200, 206]</code>
<code>cache_valid</code>	Integer	<code>10</code>	缓存有效时长（秒），最小值为 1。	<code>60</code>
<code>cache_control</code>	Boolean	<code>false</code>	是否遵循上游响应的 <code>Cache-Control</code> 头来控制缓存行为（包含 <code>no-cache</code> 、 <code>no-store</code> 、 <code>private</code> 、 <code>max-age</code> 等指令）。	<code>true</code>
<code>hide_cache_headers</code>	Boolean	<code>false</code>	是否在返回给客户端的响应中隐藏 <code>Cache-Control</code> 和 <code>Expires</code> 头。	<code>true</code>
<code>show_resp_cache_header</code>	Boolean	<code>false</code>	是否在响应中添加缓存状态标识头（名称由 <code>header_name</code> 指定）。	<code>true</code>
<code>header_name</code>	String	<code>"X-EDGE-Cache-Status"</code>	缓存状态响应头名称，仅在 <code>show_resp_cache_header</code> 为 <code>true</code> 时生效。	<code>"X-Cache-Status"</code>
<code>no_cache_expr</code>	Array	无	条件表达式，命中时跳过缓存存储（不写缓存）。语法参见 Edge 表达式列表 。	<code>[["arg_nocache", "==", "1"]]</code>
<code>bypass_cache_expr</code>	Array	无	条件表达式，命中时绕过缓存读取（直接穿透到上游）。语法参见 Edge 表达式列表 。	<code>[["http_pragma", "==", "no-cache"]]</code>

关于 `cache_key` 变量支持

`cache_key` 字段支持 Edge 变量插值，常用变量如 `${host}`、`${uri}`、`${request_uri}`、`${args}` 等，完整列表参见 [Edge 变量列表](#)。

关于条件表达式

`no_cache_expr` 和 `bypass_cache_expr` 均使用标准 Edge 表达式格式，示例：`[["arg_nocache", "==", "1"]]`。详见 [Edge 表达式列表](#)。

3. 调用与验证示例

3.1 磁盘缓存路由

为路径 `/api/resource` 开启磁盘缓存，缓存 GET 请求的 200 响应，有效期 60 秒，并在响应头中展示缓存状态：

```
curl -X PUT http://127.0.0.1:9991/edge/admin/routes/10 \  
-H "Content-Type: application/json" \  
-H "X-API-Key: API_KEY" \  
-d '{  
  "uri": "/api/resource/*",  
  "plugins": {  
    "proxy_cache": {  
      "cache_type": "disk",
```

```

        "cache_zone": "edge_proxy_cache1",
        "cache_key": "${host},${request_uri}",
        "cache_method": [ "GET", "HEAD" ],
        "cache_http_status": [ 200 ],
        "cache_valid": 60,
        "show_resp_cache_header": true,
        "header_name": "X-Cache-Status"
    }
},
"upstream": {
    "nodes": [
        { "host": "127.0.0.1", "port": 8811, "weight": 1 }
    ],
    "type": "roundrobin"
}
}'

```

3.2 内存缓存路由

使用内存缓存区，缓存时长 30 秒，遵循 `Cache-Control` 头：

```

curl -X PUT http://127.0.0.1:9991/edge/admin/routes/11 \
-H "Content-Type: application/json" \
-H "X-API-Key: API_KEY" \
-d '{
    "uri": "/api/hot/*",
    "plugins": {
        "proxy_cache": {
            "cache_type": "memory",
            "cache_zone": "memory_cache",
            "cache_valid": 30,
            "cache_control": true,
            "show_resp_cache_header": true
        }
    },
    "upstream": {
        "nodes": [
            { "host": "127.0.0.1", "port": 8811, "weight": 1 }
        ],
        "type": "roundrobin"
    }
}'

```

3.3 验证缓存效果

首次请求（MISS）：

```

curl -v http://127.0.0.1:9981/api/resource/data
## 响应头预期: X-Cache-Status: MISS

```

再次请求（HIT）：

```

curl -v http://127.0.0.1:9981/api/resource/data
## 响应头预期: X-Cache-Status: HIT

```

3.4 清除缓存

对指定资源发送 `PURGE` 方法请求：

```
curl -X PURGE http://127.0.0.1:9981/api/resource/data
## 成功返回 200，资源未找到返回 404
```

proxy_mirror 插件使用文档

1. 插件概览

`proxy_mirror` 插件用于将进入网关的请求异步镜像至一个指定的后端地址，同时不影响原始请求的正常代理与响应。常见应用场景包括：

- 流量录制与回放：将线上流量复制到测试/预发环境，用于问题复现与压测。
- 灰度验证：将真实流量同步发送至新版本服务，观察其行为而不影响线上用户。
- 日志镜像：将请求发送至独立的日志收集服务，进行全量审计或分析。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

无。

2.3 插件基础属性 (Config)

路由级配置，在路由的 `plugins` 节点中配置 `proxy_mirror` 对象。

字段名	类型	默认值	描述	设置示例
host	String	无（必填）	镜像目标地址，格式为 <code>scheme://host[:port]</code> ，支持 <code>http</code> 、 <code>https</code> 、 <code>grpc</code> 、 <code>grpcs</code> 协议。	<code>"http://mirror.internal:8080"</code>
path	String	无	镜像请求的目标路径。未设置时沿用原请求路径。	<code>"/mirror"</code>
path_mode	String	<code>"replace"</code>	路径处理模式。 <code>"replace"</code> 表示用 <code>path</code> 完全替换原路径； <code>"prefix"</code> 表示将 <code>path</code> 拼接在原路径前面。	<code>"prefix"</code>
sample_ratio	Number	<code>1</code>	镜像采样比例，取值范围 <code>0.00001 ~ 1</code> 。 <code>1</code> 表示对所有请求进行镜像； <code>0.1</code> 表示随机采样 10% 的请求进行镜像。	<code>0.5</code>

3. 调用与验证示例

3.1 创建并启用路由

使用管理接口为路径 `/api/mirror` 启用 `proxy_mirror` 插件，将所有流量镜像至 `http://10.0.0.100:8080`：

```
curl -X PUT http://127.0.0.1:9991/edge/admin/routes/1 \
-H "Content-Type: application/json" \
-H "X-API-Key: API_KEY" \
-d '{
  "uri": "/api/mirror/*",
```

```
"plugins": {
  "proxy_mirror": {
    "host": "http://10.0.0.100:8080",
    "sample_ratio": 1
  }
},
"upstream": {
  "nodes": [
    { "host": "127.0.0.1", "port": 8811, "weight": 1 }
  ],
  "type": "roundrobin"
}
}'
```

3.2 采样镜像示例

对 50% 的请求进行镜像，并将路径以 `/shadow` 为前缀：

```
curl -X PUT http://127.0.0.1:9991/edge/admin/routes/2 \
-H "Content-Type: application/json" \
-H "X-API-Key: API_KEY" \
-d '{
  "uri": "/api/data/*",
  "plugins": {
    "proxy_mirror": {
      "host": "http://10.0.0.200:9000",
      "path": "/shadow",
      "path_mode": "prefix",
      "sample_ratio": 0.5
    }
  },
  "upstream": {
    "nodes": [
      { "host": "127.0.0.1", "port": 8811, "weight": 1 }
    ],
    "type": "roundrobin"
  }
}'
```

3.3 验证镜像生效

向网关发送请求：

```
curl -v http://127.0.0.1:9981/api/mirror/test
```

预期行为：

- 原始请求正常响应（来自上游 `127.0.0.1:8811`）。
- 镜像目标 `http://10.0.0.100:8080` 同时收到相同请求（异步发送，不影响响应时延）。

proxy_rewrite 使用文档

1. 插件概览

`proxy_rewrite` 插件用于在 API 网关将请求代理转发给后端（上游）服务之前，动态修改或重写客户端的请求内容。典型的应用场景包括：

- 动态更改代理请求的 HTTP 方法（如 `GET`，`POST` 等）。
- 更改转发时的请求 Scheme（协议）为 `http` 或 `https`。
- 修改转发的目标 Host 标头。
- 修改请求的 URI 路径，支持正则匹配替换或添加路径前缀。
- 在请求头中添加、重写或通过正则匹配替换标头信息。
- 重写或通过正则匹配替换请求体（Request Body）内容。
- 支持条件表达式以精细化控制何时应用请求头或请求体的修改。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

无。

2.3 插件基础属性 (Config)

路由级配置，可直接配置在特定路由（`Route`）中。支持根据 HTTP 请求方法或基于表达式来进行匹配重写。

字段名	类型	默认值	描述	设置示例
<code>method</code>	String	无	重写后的目标请求方法。如 <code>"POST"</code> ， <code>"GET"</code> 。	<code>"POST"</code>
<code>scheme</code>	String	无	重写后的 Scheme，支持 <code>"http"</code> 或 <code>"https"</code> 。	<code>"https"</code>
<code>host</code>	String	无	重写后的请求 Host。支持带端口。	<code>"backend.service:8080"</code>
<code>uri</code>	String	无	重写后的请求 URI 路径。支持变量插值解析。	<code>"/new-api/v1/\${uri}"</code>
<code>regex_uri</code>	Array	无	包含 2 个字符串的数组：[<code>"匹配正则"</code> ， <code>"替换值"</code>]，用于在原始 URI 上执行正则重写。	<code>["^/old-prefix/(.*)", "/new-prefix/\$1"]</code>
<code>add_headers</code>	Object	无	在原有请求头基础上追加请求头信息。支持变量动态生成。键为 header 名，值支持 String/Number 或复杂结构。	<code>{ "X-Gateway-Name": "Edge-Prod" }</code>
<code>headers</code>	Object	无	覆盖或修改现有的请求头信息。支持通过正则替换或变量动态生成。键为 header 名，值支持 String/Number 或复杂结构。	<code>{ "Host": "service.internal" }</code>
<code>body</code>	String/Array/Object	无	替换原始客户端请求体的内容。当 <code>plain_text</code> 为 <code>false</code> 时，支持变量插值。	<code>"New Body Content"</code>
<code>regex_body</code>	Array	无	包含用于在原始请求体上执行正则匹配与替换的规则列表。	<code>[["World", "Edge", 1]]</code>
<code>plain_text</code>	Boolean	<code>false</code>	设置为 <code>true</code> 时，表示 <code>body</code> 内容为纯文本，不对其中的任何变量进行解析。	<code>true</code>
<code>include_add_headers_expr</code>	Array	无	条件表达式规则，仅在满足该条件时执行 <code>add_headers</code> 。	<code>[["uri", "=", "/test"]]</code>
<code>include_headers_expr</code>	Array	无	条件表达式规则，仅在满足该条件时执行 <code>headers</code> 修改。	<code>[["uri", "=", "/test"]]</code>
<code>include_body_expr</code>	Array	无	条件表达式规则，仅在满足该条件时执行 <code>body</code> 和 <code>regex_body</code> 替换。	<code>[["uri", "=", "/test"]]</code>

说明:

- `regex_body` 的规则列表每一项的格式为 [`"正则表达式"`，`"替换字符串"`，`"替换次数"`]。其中，“替换次数”默认值为 `0`（表示全部替换）。注意：第四个参数“正则选项”为保留选项，不开放给普通用户使用。
- `headers` 与 `add_headers` 的值除了支持简单的 String/Number 外，还支持正则匹配的高级格式（同样隐藏了保留的正则选项参数）：

- 格式一：["提取值", "正则表达式", "替换字符串", "替换次数"] (如 ["", "World", "Edge", 1])，当提取值为空字符串 "" 时，表示在原 Header 值上执行正则替换。
- 格式二：["提取值", 正则规则数组]，用于对同一个源值运行多重正则替换规则。
- 格式三：直接传递正则规则数组，在原始 Header 值上做多重正则替换。
- 条件表达式 (如 include_*_expr 等) 设置示例必须严格使用标准的 Edge 表达式格式，其语法及可用变量详见 [Edge 表达式列表](#)。

2.4 注意事项

- 内置变量解析：重写配置 (包括 uri, headers, add_headers 以及 body 字段) 支持注入 Nginx/Edge 内置变量，如 \${host} 或 \${remote_addr}，详情见 [Edge 变量列表](#)。请注意，plain_text 字段仅对 body 字段生效 (设置为 true 时，不对 body 中的变量进行解析；而 uri、headers、add_headers 则不受其影响，始终会执行变量解析)。

3. 调用与验证示例

以下示例演示如何调用管理接口，配置并应用 proxy_rewrite 插件。

注意事项：实际调用管理接口时，请求体需要先经过 SM4 加密，返回结果也需要进行解密，详见 [管理接口调用规范](#)。以下示例统一使用大写形式的占位符 (如 API_KEY) 指代敏感信息，占位符未包裹中括号以防歧义。

3.1 覆盖代理 Host 和请求 URI

通过管理接口为指定路由配置插件。将所有请求代理转发到后端的 Host 改写为 upstream.service.local，并且重写请求 URI 以追加版本号前缀 /v2，同时在请求中注入客户端的真实 IP。

通过管理接口配置路由：

```
curl -L -X PUT 'http://ADMIN_ADDR/edge/admin/routes/2001' \  
-H 'X-API-KEY: API_KEY' \  
-H 'Content-Type: application/json' \  
-d '{  
  "uri": "/api/*",  
  "plugins": {  
    "proxy_rewrite": {  
      "host": "upstream.service.local",  
      "uri": "/v2${uri}",  
      "headers": {  
        "X-Real-IP": "${remote_addr}"  
      }  
    }  
  },  
  "upstream": {  
    "nodes": {  
      "127.0.0.1:8111": 1  
    }  
  }  
}'
```

发起客户端请求进行验证：

```
## 发起请求
curl -i http://localhost:9980/api/users

## 后端 (127.0.0.1:8111) 实际收到的重写后请求为:
## Path: /v2/api/users
## Host Header: upstream.service.local
## X-Real-IP Header: 127.0.0.1
```

3.2 使用条件表达式替换请求体中的关键字

对特定的请求执行请求体的正则查找与替换。当请求 URI 为 `/submit` 时，将请求体中的敏感词 `password` 替换为 `*****`。

通过管理接口配置路由：

```
curl -L -X PUT 'http://ADMIN_ADDR/edge/admin/routes/2002' \
-H 'X-API-KEY: API_KEY' \
-H 'Content-Type: application/json' \
-d '{
  "uri": "/submit",
  "plugins": {
    "proxy_rewrite": {
      "regex_body": [
        [
          "password",
          "*****",
          0
        ]
      ],
      "include_body_expr": [
        ["uri", "==", "/submit"]
      ]
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:8111": 1
    }
  }
}'
```

发起客户端请求进行验证：

```
## 客户端发送包含敏感数据的 POST 请求
curl -i -X POST http://localhost:9980/submit \
-H "Content-Type: text/plain" \
-d "user=admin&password=my_secret_password"

## 后端 (127.0.0.1:8111) 实际收到的重写后请求体数据为:
## user=admin&*****=my_secret_password
```

real_ip 插件使用说明书

1. 插件概览

`real_ip` 插件用于在 API 网关接收到请求时，从请求头（如 `X-Forwarded-For`、`X-Real-IP`）或自定义变量中提取真实的客户端 IP 地址和源端口号，并覆盖重写网关连接的底层源 IP（即更新内置变量 `$remote_addr` 和 `$remote_port`）。

主要应用场景：

- **安全检查与限速：**网关前置了 CDN、高防 WAF 或 L4/L7 负载均衡器时，确保 IP 访问控制（IP 黑白名单）和限速插件能够拿到真正的客户端真实 IP，而不是代理节点的 IP。
- **精确日志记录：**使网关本地日志和上游后端接收到的 `remote_addr` 均更新为客户端的实际 IP，便于追踪分析。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

通过管理接口进行全局配置（API 路径：`/edge/admin/plugin_metadata/real_ip`），用于设定全局默认的 IP 重写规则。

[!IMPORTANT]
所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：`PUT`
- 请求路径：`/edge/admin/plugins/reload`

您可以在配置多个插件的数据属性后，统一执行一次热加载操作。

字段名	类型	默认值	描述	设置示例
<code>source</code>	String	<code>"\${http_x_forwarded_for}"</code>	提取客户端真实 IP 地址的数据源，支持变量插值（例如通过 <code>\${http_header_name}</code> 读取指定的标头）。至少包含 1 个字符。	<code>"\${http_x_real_ip}"</code>
<code>trusted</code>	Array	无	受信任的代理服务器 IP 或网段（CIDR 格式）列表。若直接连接的客户端 IP 属于该列表，网关才会信任并解析数据源；若不配置，网关无条件信任数据源并进行解析。	<code>["10.0.0.0/8", "192.168.1.1"]</code>
<code>recursive</code>	Boolean	<code>false</code>	是否开启递归解析模式。若为 <code>true</code> ，网关会与 <code>trusted</code> 配合从右往左依次遍历代理 IP 列表，剔除属于可信白名单的代理 IP，直至遇到第一个不属于白名单的 IP 作为真实的客户端 IP；若为 <code>false</code> ，则直接将数据源中最右侧的 IP 作为真实的客户端 IP。	<code>true</code>

2.3 插件基础属性 (Config)

路由级配置，可覆盖全局定义的插件数据属性 (Metadata Config)。

字段名	类型	默认值	描述	设置示例
<code>source</code>	String	<code>"\${http_x_forwarded_for}"</code>	同上	<code>"\${http_x_forwarded_for}"</code>
<code>trusted</code>	Array	无	同上	<code>["192.168.1.0/24", "127.0.0.1"]</code>
<code>recursive</code>	Boolean	<code>false</code>	同上	<code>true</code>

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 `/api/ip` 开启 `real_ip` 插件，将其配置为：仅信任来自本地回环地址 `127.0.0.1` 发起的代理请求，并通过 `X-Forwarded-For` 头部递归获取客户端的真实 IP。

📌 Important

调用示例中不包含真实敏感密钥，请将占位符 `API_KEY` 替换为您实际环境中的 Admin API Key。

```
curl -i -X PUT \
-H "Content-Type: application/json" \
-H "X-API-Key: API_KEY" \
-d '{
  "uri": "/api/ip",
  "plugins": {
    "real_ip": {
      "source": "${http_x_forwarded_for}",
      "trusted": ["127.0.0.1"],
      "recursive": true
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:8811": 1
    },
    "type": "roundrobin"
  }
}' \
http://127.0.0.1:9991/edge/admin/routes/1002
```

3.2 访问网关进行验证

向网关发起请求，并通过 `X-Forwarded-For` 头传递多级代理链（例如 `1.2.3.4, 127.0.0.1`）。

```
curl -i -H "X-Forwarded-For: 1.2.3.4, 127.0.0.1" http://127.0.0.1:9981/api/ip
```

解析过程：

1. 网关直接连接源 IP 为 `127.0.0.1`，匹配 `trusted` 列表。
2. 从右往左遍历 `X-Forwarded-For`（即 `127.0.0.1 -> 1.2.3.4`）。
3. `127.0.0.1` 属于可信代理，被剔除。
4. `1.2.3.4` 不属于可信代理白名单，匹配终止，网关最终将 `$remote_addr` 重写为 `1.2.3.4`。

预期响应输出 (Backend 收到重写后的 `$remote_addr`):

后端服务器收到的客户端来源 IP 地址将被成功改写为 `1.2.3.4`。

redirect 插件使用文档

1. 插件概览

`redirect` 插件用于在网关层实现 HTTP 重定向跳转。用户可以配置静态目标地址、包含动态变量（如 `${host}`，`${request_uri}`）的跳转地址，或利用正则表达式进行匹配与替换，同时支持将 HTTP 请求强制升级为 HTTPS。

主要应用场景：

- **站点迁移/改版跳转**：永久或临时将某个 URL 或路径路由重定向到新地址。
- **强制 HTTPS 跳转**：全站或特定路由强制 HTTP 流量重定向到 HTTPS，保障通信安全。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

无。

2.3 插件基础属性 (Config)

路由级配置。

字段名	类型	默认值	描述	设置示例
uri	String	无	重定向的目标 URL/URI，支持动态变量替换。与 regex_uri 和 http_to_https 互斥。	"https://\${host}\${request_uri}"
regex_uri	Array of String	无	包含两个元素的数组，分别代表匹配正则表达式及替换规则。与 uri 和 http_to_https 互斥。	["^/old-api/(.*)", "/new-api/\$1"]
http_to_https	Boolean	无	是否将 HTTP 请求跳转到 HTTPS。与 uri 和 regex_uri 互斥。	true
status	Integer	302	自定义重定向响应的状态码，取值范围为 200 到 599。	301
append_query_string	Boolean	true	重定向时是否默认附加原始请求的 Query 参数。	false
encode_uri	Boolean	false	是否对重定向的 URI 部分进行 URL 编码。	true

说明

- 配置时必须提供 uri、regex_uri 或 http_to_https 其中之一。

3. 调用与验证示例

本示例介绍如何创建路由，并启用 redirect 插件进行 301 永久重定向跳转。

Important

调用示例中不包含真实敏感密钥，请将占位符 API_KEY 替换为您实际环境中的 Admin API Key。

3.1 步骤 1：创建启用 redirect 插件的路由

创建路由关联路径 /old-path，跳转至新地址 /new-path，并返回状态码 301（永久重定向），同时保留参数。

```
curl -i -X PUT \  
  -H "Content-Type: application/json" \  
  -H "X-API-Key: API_KEY" \  
  -d '{  
    "uri": "/old-path",  
    "plugins": {  
      "redirect": {  
        "uri": "/new-path",  
        "status": 301,  
        "append_query_string": true  
      }  
    },  
    "upstream": {  
      "nodes": {  
        "127.0.0.1:8811": 1  
      }  
    }  
  }'
```

```
    },
    "type": "roundrobin"
  }
} ' \
http://127.0.0.1:9991/edge/admin/routes/3003
```

3.2 访问网关进行验证

向网关发起携带参数的访问请求：

```
curl -i "http://127.0.0.1:9981/old-path?user=test"
```

预期响应输出（状态码为 301，并在 Location 头中正确带上参数且指向新地址）：

```
HTTP/1.1 301 Moved Permanently
Content-Length: 0
Connection: keep-alive
Location: /new-path?user=test
...
```

remote_redis 插件使用文档

1. 插件概览

`remote_redis` 插件用于在网关层实时连接 Redis，将查询得到的值自动映射为自定义的网关变量。这可以让后续执行的其他插件（例如 `response_rewrite` 修改响应头、`proxy_rewrite` 重写上游请求头等）能够自由引用该变量值，实现动态的数据传递。

主要应用场景：

- **动态变量提取与重写**：去 Redis 中根据用户 ID 查询对应的详细属性信息，并将其自动填入代理转发给后端服务器的 Header 头部。
- **自定义响应信息**：将查询的缓存值直接渲染进响应 Header 中返回给客户端。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（`/edge/admin/plugin_metadata/remote_redis`）进行全局配置。

****[IMPORTANT]**

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：`PUT`
- 请求路径：`/edge/admin/plugins/reload` ******

2.1.1 全局基础配置

字段名	类型	默认值	描述	设置示例
key	String / Array	"\${username}"	提取用户或客户端标识值的变量模板或多键列表。	"\${http_x_user}"
key_prefix	String	" "	拼接至 Redis 查询 Key 之前的前缀。	" "
key_mapping	String / Array	无	映射成的网关上下文变量名称。可为字符串或数组，与 key 逐个对应。	"mapped_user"
redis_dc	String	"TEST"	匹配的 Redis 数据中心。	"TEST"
redis_conf	Object	见传输配置	Redis 连接网络配置属性。	见传输配置。
redis_exptime	Number	60	正常查询结果在网关本地的缓存有效期（秒）。	60
redis_none_exptime	Number	10	查询结果为空时的本地缓存有效期（秒）。	10
bypass_missing_key	Boolean	true	若 Key 变量提取为空时是否直接放行。	true
bypass_error_key	Boolean	true	若 Redis 连接或查询出错时是否放行。	true

2.1.2 Redis 网络传输配置 (redis_conf)

字段名	类型	默认值	描述	设置示例
CLUSTER_[a-zA-Z0-9]+ (如 CLUSTER_TEST)	Array	[["127.0.0.1", 6379]]	Redis 节点地址列表。每一个节点地址格式为 [ip, port, ...]。	[["127.0.0.1", 16379]]
MODE	String	"redis"	连接模式。可选值: "redis" (单点模式), "rediscluster" (集群模式), "proxycluster" (代理集群模式)。	"redis"
AUTH	String	无	Redis 认证密码。	"<REDIS_PASSWORD>"
POOL	String	无	连接池的自定义名称。	"my_pool"
POOL_SIZE	Integer	2	连接池大小限制。	10
BACKLOG	Integer	-1	连接池请求积压的最大大小。	100
TIMEOUT	Number	1	网络读写与建立连接的超时时间（秒）。	2
CONNECT_TIMEOUT	Number	1	建立连接的超时时间（秒）。	1
SEND_TIMEOUT	Number	1	向 Redis 发送数据的超时时间（秒）。	1
READ_TIMEOUT	Number	1	从 Redis 读取响应的超时时间（秒）。	1
DATABASE	Integer	0	数据库索引 (DB Index, 仅单点模式生效)。	0
MAX_CONNECT_NODES	Integer	2	允许连接的最大 Redis 节点个数。	4
MAX_IDLE_TIME	Integer	60	连接池中空闲连接的超时时间（秒）。	60
CONNECTION_MAX_SIZE	Integer	1024	每个网关进程的连接池最大容量。	100
LATENCY_METRICS	Boolean	false	是否开启延迟指标监测收集。	true
BREAKER_NODES	Number	0.5	触发熔断节点的比率阈值。	0.5
LIMIT_NODES	Number	0.5	限制节点的比率阈值。	0.5
LIMIT_COUNT	Object	无	限制规则的计数频率配置。	见配置示例

2.2 插件基础属性 (Config)

路由级配置。格式与 数据属性 (Metadata Config) 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 /test_remote_redis 开启本插件：

- 连接部署在 127.0.0.1:16379 的 Redis 服务，密码为 "<REDIS_PASSWORD>"。
- 以 x-User 请求头作为 Key 进行提取，并映射为变量 mapped_user。
- 同时配置 response_rewrite 插件，将请求返回头 x-Mapped-User 重写为刚才映射的 \${mapped_user} 变量值。


```
curl -i -X PUT \
-H "Content-Type: application/json" \
-H "X-API-Key: API_KEY" \
-d '{
  "uri": "/test_remote_redis",
  "plugins": {
    "remote_redis": {
      "key": "${http_x_user}",
      "key_mapping": "mapped_user",
      "redis_dc": "TEST",
      "redis_conf": {
        "CLUSTER_TEST": [
          ["127.0.0.1", 16379]
        ],
        "AUTH": "<REDIS_PASSWORD>"
      }
    },
    "response_rewrite": {
      "headers": {
        "X-Mapped-User": "${mapped_user}"
      }
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:80": 1
    }
  }
}' \
http://127.0.0.1:9991/edge/admin/routes/3033
```

3.2 访问网关进行验证

正常请求

预先向 Redis 中写入对应的属性（键 `myuser`，值为 `mapped_value`）：

```
redis-cli -p 16379 -a <REDIS_PASSWORD> SET myuser mapped_value
```

向网关发送携带 `X-User: myuser` 的请求：

```
curl -i -H "X-User: myuser" "http://127.0.0.1:9981/test_remote_redis"
```

预期输出：

网关允许通过，且返回头中包含重写后对应的 Redis 映射数据：

```
HTTP/1.1 200 OK
X-Mapped-User: mapped_value
Content-Length: 6

Passed
```

response_rewrite

1. 插件概览

`response_rewrite` 插件用于在 API 网关将响应发送给客户端之前，动态重写响应报文的内容。典型的应用场景包括：

- 修改或覆盖后端的响应状态码（HTTP Status）。
- 动态添加、修改或删除响应头（Response Headers），例如添加安全标头或跨域配置。
- 修改或替换响应体（Response Body），支持纯文本替换、变量注入以及基于正则表达式的内容替换。
- 能够依据条件表达式（Expressions）针对特定的响应内容执行重写操作。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

通过管理接口全局配置（路径：`/edge/admin/plugin_metadata/response_rewrite`）。

字段名	类型	默认值	描述	设置示例
<code>status</code>	Integer	无	重写的 HTTP 响应状态码，支持 200 到 599。	<code>200</code>
<code>add_headers</code>	Object	无	在原有响应头基础上追加的标头键值对配置。	<code>{ "X-Add-Header": "Edge" }</code>
<code>headers</code>	Object	无	覆盖或设置的响应标头键值对配置（如果原来已有则修改）。	<code>{ "Server": "Edge-Gateway" }</code>
<code>body</code>	String/Array/Object	无	用来替换原始后端响应体的内容。	<code>"New Body Content"</code>
<code>regex_body</code>	Array	无	包含用于在原始响应体上执行正则匹配与替换的规则列表。	<code>[["world", "Edge", 1]]</code>
<code>plain_text</code>	Boolean	<code>false</code>	设置为 <code>true</code> 时，表示 <code>body</code> 内容为纯文本，不对其中的变量进行解析。	<code>true</code>

说明: `regex_body` 规则列表每一项的格式为 `[ "正则表达式", "替换字符串", "替换次数" ]`。其中，“替换次数”默认值为 `0`（表示全部替换）。

2.3 插件基础属性 (Config)

路由级配置，不仅包含上述数据属性（`_schema`），还额外支持根据 HTTP 请求方法或基于表达式来进行匹配重写。

字段名	类型	默认值	描述	设置示例
status	Integer	同上	同上	同上
add_headers	Object	同上	同上	同上
headers	Object	同上	同上	同上
body	String/Array/Object	同上	同上	同上
regex_body	Array	同上	同上	同上
plain_text	Boolean	同上	同上	同上
include_add_headers_expr	Array	无	条件表达式规则，仅在满足该条件时执行 add_headers。	[["status", "==" , 200]]
include_headers_expr	Array	无	条件表达式规则，仅在满足该条件时执行 headers 修改。	[["status", "==" , 200]]
include_body_expr	Array	无	条件表达式规则，仅在满足该条件时执行 body 和 regex_body 替换。	[["status", "==" , 200]]
(HTTP Method)	Object	无	可以指定特定的请求方法（如 "GET", "POST"）作为键名，值为包含 status, body, headers 等配置的对象，表示仅针对该方法的特定重写规则。	{ "GET": { "status": 201 } }

说明：条件表达式（如 `include_*_expr` 等）设置示例必须严格使用标准的 Edge 表达式格式，其语法及可用变量详见 [Edge 表达式列表](#)。

2.4 注意事项

- **动态变量支持:** 在 `add_headers`、`headers` 的值中，以及 `body` 字段（当 `plain_text` 不为 `true` 时）支持注入 Nginx/Edge 变量，如 `${host}` 或 `${remote_addr}`，请参考 [Edge 变量列表](#)。
- **响应体过滤限制:** 使用 `body` 或 `regex_body` 修改响应体时，如果后端的响应是流式的或被分块传输（Chunked），网关会先缓冲响应体内容再执行替换，可能会增加一定的内存消耗或请求延迟。
- **配置覆盖:** 路由级别的配置与基于 HTTP 方法（如 `GET`）的配置如果存在冲突，以具体方法块内的配置为准。

3. 调用与验证示例

3.1 覆盖响应头并修改响应体

通过管理接口配置包含该插件的路由：

注意：实际调用管理接口时，请求体需要先经过 SM4 加密，返回结果也需要进行解密，详见 [管理接口调用规范](#)。

```
curl -L -X PUT 'http://[ADMIN_ADDR]/edge/admin/routes/3001' \  
-H 'X-API-KEY: API_KEY' \  
-H 'Content-Type: application/json' \  
-d '{  
  "uri": "/test_rewrite",  
  "plugins": {  
    "response_rewrite": {  
      "status": 200,  
      "headers": {  
        "X-Powered-By": "OpenResty Edge",  
        "Server": "CustomServer/1.0"  
      },  
      "body": "This request was intercepted and rewritten by Edge. Client IP: ${remote_addr}"  
    }  
  }  
}
```

```
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:8111": 1
    }
  }
}
```

访问网关验证:

```
## 发起请求
$ curl -i http://127.0.0.1/test_rewrite

## 预期响应
HTTP/1.1 200 OK
Content-Type: text/plain
Connection: keep-alive
X-Powered-By: OpenResty Edge
Server: CustomServer/1.0

This request was intercepted and rewritten by Edge. Client IP: 127.0.0.1
```

3.2 使用正则表达式替换响应体

假设后端（上游）服务返回内容为 "Hello, World!", 我们希望将响应体中的 World 动态替换为 Edge。

通过管理接口配置包含该插件的路由:

```
curl -L -X PUT 'http://[ADMIN_ADDR]/edge/admin/routes/3002' \
-H 'X-API-KEY: API_KEY' \
-H 'Content-Type: application/json' \
-d '{
  "uri": "/test_regex",
  "plugins": {
    "response_rewrite": {
      "regex_body": [
        [
          "World",
          "Edge",
          1
        ]
      ]
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:8111": 1
    }
  }
}
```

访问网关验证:

```
## 发起请求
$ curl -i http://127.0.0.1/test_regex

## 预期响应
HTTP/1.1 200 OK
Content-Type: text/plain
...
Hello, Edge!
```

security_common_args 插件使用文档

1. 插件概览

security_common_args 插件用于对 HTTP 请求的查询参数（URL 中的 Query Arguments）进行安全规则匹配与阻断检测，防御如 SQL 注入、XSS 跨站脚本、系统命令注入等通过 URL 参数进行的 Web 攻击。

主要应用场景：

- SQL 注入防护：拦截含有 UNION SELECT、sys.user 等数据库关键词或特殊字符的 URL 访问。
- 参数格式校验：只允许特定参数（如 user_id）为纯数字网段，不匹配则一律阻断，降低接口被遍历风险。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（/edge/admin/plugin_metadata/security_common_args）进行全局拦截状态与全局默认规则配置。

[!IMPORTANT]

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：PUT
- 请求路径：/edge/admin/plugins/reload

2.1.1 全局基础配置

字段名	类型	默认值	描述	设置示例
status	Integer	403	被黑名单阻断拦截时，网关返回给客户端的 HTTP 状态码。	403
message	String	"Your request is not allowed."	被阻断拦截时的返回正文消息。	"Blocked by WAF Args"
denylist	Array	无	全局正则黑名单。匹配该名单中任一正则的查询参数值都将被直接阻断。	["blocked_arg", "union\\s+select"]
allowlist	Array	无	全局正则白名单。匹配该名单中任一正则的参数值将直接放行，免受黑名单检测。	["^safe_token_[a-zA-Z0-9]+\$"]
log_allowed	Boolean	false	是否记录被安全规则放行的正常请求日志。	true

2.2 插件基础属性 (Config)

路由级配置。格式与 数据属性 (Metadata Config) 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 `/test_common_args` 开启本插件：

- 对查询参数中的敏感词 `"blocked_arg"` 进行拦截。
- 自定义拦截返回的响应为 `403`，消息内容为 `"Blocked by WAF Args"`。

```
curl -i -X PUT \  
  -H "Content-Type: application/json" \  
  -H "X-API-Key: API_KEY" \  
  -d '{  
    "uri": "/test_common_args",  
    "plugins": {  
      "security_common_args": {  
        "denylist": [ "blocked_arg" ],  
        "status": 403,  
        "message": "Blocked by WAF Args"  
      }  
    },  
    "upstream": {  
      "nodes": {  
        "127.0.0.1:80": 1  
      }  
    }  
  }' \  
  http://127.0.0.1:9991/edge/admin/routes/3020
```

3.2 访问网关进行验证

正常请求

发起不含有黑名单词汇的查询请求：

```
curl -i "http://127.0.0.1:9981/test_common_args?name=safe_value"
```

预期输出：

网关允许通过，由上游成功处理，响应 HTTP `200`。

异常请求阻断

发起含有恶意黑名单词汇的查询请求：

```
curl -i "http://127.0.0.1:9981/test_common_args?name=blocked_arg"
```

预期输出：

网关直接进行拦截，不向后转发，且返回：

```
HTTP/1.1 403 Forbidden  
Content-Length: 19  
  
Blocked by WAF Args
```

security_common_body 插件使用文档

1. 插件概览

`security_common_body` 插件用于对 HTTP 请求体（Request Body，如表单或 JSON）进行安全规则匹配与阻断检测，防御命令注入、恶意代码上传、跨站攻击载荷（XSS）等通过 POST 请求正文进行的 Web 攻击。

主要应用场景：

- **恶意 Payload 拦截：**拦截含有 `<script>` 标签、`eval()` 等恶意执行函数的正文请求。
- **敏感关键字防御：**阻断包含受限制的后台命令或者特权业务字样的请求。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（`/edge/admin/plugin_metadata/security_common_body`）进行全局拦截状态与全局默认规则配置。

[!IMPORTANT]

所有插件的数据属性（**Metadata Config**）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：`PUT`
- 请求路径：`/edge/admin/plugins/reload`

2.1.1 全局基础配置

字段名	类型	默认值	描述	设置示例
<code>status</code>	Integer	<code>403</code>	被黑名单阻断拦截时，网关返回给客户端的 HTTP 状态码。	<code>403</code>
<code>message</code>	String	<code>"Your request is not allowed."</code>	被阻断拦截时的返回正文消息。	<code>"Blocked by WAF Body"</code>
<code>denylist</code>	Array	无	全局正则黑名单。匹配该名单中任一正则的请求体值都将被直接阻断。	<code>["blocked_body", "eval\\(\"</code>
<code>allowlist</code>	Array	无	全局正则白名单。匹配该名单中任一正则的请求体将直接放行。	<code>["^safe_body_[a-zA-Z0-9]+ \$"]</code>
<code>log_allowed</code>	Boolean	<code>false</code>	是否记录被安全规则放行的正常请求日志。	<code>true</code>

2.2 插件基础属性 (Config)

路由级配置。格式与 **数据属性 (Metadata Config)** 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 `/test_common_body` 开启本插件：

- 对请求体中的敏感词 `"blocked_body"` 进行拦截。
- 自定义拦截返回的响应为 `403`，消息内容为 `"Blocked by WAF Body"`。

```
curl -i -X PUT \
```

```
-H "Content-Type: application/json" \  
-H "X-API-Key: API_KEY" \  
-d '{  
  "uri": "/test_common_body",  
  "plugins": {  
    "security_common_body": {  
      "denylist": ["blocked_body"],  
      "status": 403,  
      "message": "Blocked by WAF Body"  
    }  
  },  
  "upstream": {  
    "nodes": {  
      "127.0.0.1:80": 1  
    }  
  }  
}' \  
http://127.0.0.1:9991/edge/admin/routes/3021
```

3.2 访问网关进行验证

正常请求

发起不含有黑名单词汇的 POST 请求：

```
curl -i -X POST -d "safe_body" "http://127.0.0.1:9981/test_common_body"
```

预期输出：

网关允许通过，响应 HTTP 200。

异常请求阻断

发起含有恶意黑名单词汇的 POST 请求：

```
curl -i -X POST -d "blocked_body" "http://127.0.0.1:9981/test_common_body"
```

预期输出：

网关直接进行拦截，返回：

```
HTTP/1.1 403 Forbidden  
Content-Length: 19  
  
Blocked by WAF Body
```

security_common_cookie 插件使用文档

1. 插件概览

`security_common_cookie` 插件用于对 HTTP 请求中的 Cookie 头部参数进行安全规则匹配与阻断检测，防御如 SQL 注入、Cookie 伪造欺骗、XSS 跨站脚本、系统越权行为等通过 Cookie 进行的 Web 攻击。

主要应用场景：

- **Cookie 安全防护**：拦截 Cookie 中包含恶意注入代码、`<script>` 标签或特权字段篡改的非法请求。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（`/edge/admin/plugin_metadata/security_common_cookie`）进行全局拦截状态与全局默认规则配置。

[!IMPORTANT]

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：`PUT`
- 请求路径：`/edge/admin/plugins/reload`

2.1.1 全局基础配置

字段名	类型	默认值	描述	设置示例
<code>status</code>	Integer	<code>403</code>	被黑名单阻断拦截时，网关返回给客户端的 HTTP 状态码。	<code>403</code>
<code>message</code>	String	<code>"Your request is not allowed."</code>	被阻断拦截时的返回正文消息。	<code>"Blocked by WAF Cookie"</code>
<code>denylist</code>	Array	无	全局正则黑名单。匹配该名单中任一正则的 Cookie 值都将被直接阻断。	<code>["blocked_cookie", "union\\s+select"]</code>
<code>allowlist</code>	Array	无	全局正则白名单。匹配该名单中任一正则的 Cookie 将直接放行。	<code>["^safe_cookie_[a-zA-Z0-9]+\$"]</code>
<code>log_allowed</code>	Boolean	<code>false</code>	是否记录被安全规则放行的正常请求日志。	<code>true</code>

2.2 插件基础属性 (Config)

路由级配置。格式与 数据属性 (Metadata Config) 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 `/test_common_cookie` 开启本插件：

- 对 Cookie 中的敏感词 `"blocked_cookie"` 进行拦截。
- 自定义拦截返回的响应为 `403`，消息内容为 `"Blocked by WAF Cookie"`。

```
curl -i -X PUT \
-H "Content-Type: application/json" \
-H "X-API-Key: API_KEY" \
-d '{
  "uri": "/test_common_cookie",
  "plugins": {
    "security_common_cookie": {
      "denylist": ["blocked_cookie"],
      "status": 403,
      "message": "Blocked by WAF Cookie"
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:80": 1
    }
  }
}
```

```
}  
}  
}' \  
http://127.0.0.1:9991/edge/admin/routes/3022
```

3.2 访问网关进行验证

正常请求

发起不含有黑名单词汇的 Cookie 请求：

```
curl -i -H "Cookie: foo=safe_cookie" "http://127.0.0.1:9981/test_common_cookie"
```

预期输出：

网关允许通过，响应 HTTP 200。

异常请求阻断

发起含有恶意黑名单词汇的 Cookie 请求：

```
curl -i -H "Cookie: foo=blocked_cookie" "http://127.0.0.1:9981/test_common_cookie"
```

预期输出：

网关直接进行拦截，返回：

```
HTTP/1.1 403 Forbidden  
Content-Length: 21  
  
Blocked by WAF Cookie
```

security_common_headers 插件使用文档

1. 插件概览

`security_common_headers` 插件用于对 HTTP 请求头部（Headers）的字段值进行安全规则匹配与阻断检测，防御如 SQL 注入、Header 伪造欺骗、XSS 跨站脚本等通过 HTTP 请求头进行的 Web 攻击。

主要应用场景：

- **Header 安全防护**：拦截请求头中包含恶意注入代码、`<script>` 标签或特权字段篡改的非法请求。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（`/edge/admin/plugin_metadata/security_common_headers`）进行全局拦截状态与全局默认规则配置。

[!IMPORTANT]

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：`PUT`
- 请求路径：`/edge/admin/plugins/reload`

2.1.1 全局基础配置

字段名	类型	默认值	描述	设置示例
status	Integer	403	被黑名单阻断拦截时，网关返回给客户端的 HTTP 状态码。	403
message	String	"Your request is not allowed."	被阻断拦截时的返回正文消息。	"Blocked by WAF Headers"
denylist	Array	无	全局正则黑名单。匹配该名单中任一正则的请求头值都将被直接阻断。	["blocked_header", "union\\s+select"]
allowlist	Array	无	全局正则白名单。匹配该名单中任一正则的请求头将直接放行。	["^safe_header_[a-zA-Z0-9]+\$"]
log_allowed	Boolean	false	是否记录被安全规则放行的正常请求日志。	true

2.2 插件基础属性 (Config)

路由级配置。格式与 数据属性 (Metadata Config) 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 /test_common_headers 开启本插件：

- 对请求头中的敏感词 "blocked_header" 进行拦截。
- 自定义拦截返回的响应为 403，消息内容为 "Blocked by WAF Headers"。

```
curl -i -X PUT \  
-H "Content-Type: application/json" \  
-H "X-API-Key: API_KEY" \  
-d '{  
  "uri": "/test_common_headers",  
  "plugins": {  
    "security_common_headers": {  
      "denylist": [ "blocked_header" ],  
      "status": 403,  
      "message": "Blocked by WAF Headers"  
    }  
  },  
  "upstream": {  
    "nodes": {  
      "127.0.0.1:80": 1  
    }  
  }  
}' \  
http://127.0.0.1:9991/edge/admin/routes/3023
```

3.2 访问网关进行验证

正常请求

发起不含有黑名单词汇的 Header 请求：

```
curl -i -H "X-Test-Header: safe_header" "http://127.0.0.1:9981/test_common_headers"
```

预期输出：
网关允许通过，响应 HTTP 200。

异常请求阻断

发起含有恶意黑名单词汇的 Header 请求：

```
curl -i -H "X-Test-Header: blocked_header" "http://127.0.0.1:9981/test_common_headers"
```

预期输出：
网关直接进行拦截，返回：

```
HTTP/1.1 403 Forbidden
Content-Length: 22

Blocked by WAF Headers
```

security_common_referer 插件使用文档

1. 插件概览

security_common_referer 插件用于对 HTTP 请求头部中的 Referer（来源页面链接）进行安全规则匹配与阻断检测，防御防盗链滥用、恶意跳转、以及简单的 CSRF 攻击防御。

主要应用场景：

- 防盗链防护：拦截来自黑名单恶意域名、盗链资源站的非法 Referer 资源请求。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（/edge/admin/plugin_metadata/security_common_referer）进行全局拦截状态与全局默认规则配置。

[!IMPORTANT]

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：PUT
- 请求路径：/edge/admin/plugins/reload

2.1.1 全局基础配置

字段名	类型	默认值	描述	设置示例
status	Integer	403	被黑名单阻断拦截时，网关返回给客户端的 HTTP 状态码。	403
message	String	"Your request is not allowed."	被阻断拦截时的返回正文消息。	"Blocked by WAF Referer"
denylist	Array	无	全局正则黑名单。匹配该名单中任一正则的 Referer 值都将被直接阻断。	["blocked_referer", "evil-site\\.com"]
allowlist	Array	无	全局正则白名单。匹配该名单中任一正则的 Referer 将直接放行。	["^https://safe-domain\\.com"]
log_allowed	Boolean	false	是否记录被安全规则放行的正常请求日志。	true

2.2 插件基础属性 (Config)

路由级配置。格式与 数据属性 (Metadata Config) 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 `/test_common_referer` 开启本插件：

- 对 Referer 中的敏感词 `"blocked_referer"` 进行拦截。
- 自定义拦截返回的响应为 `403`，消息内容为 `"Blocked by WAF Referer"`。

```
curl -i -X PUT \
-H "Content-Type: application/json" \
-H "X-Api-Key: API_KEY" \
-d '{
  "uri": "/test_common_referer",
  "plugins": {
    "security_common_referer": {
      "denylist": ["blocked_referer"],
      "status": 403,
      "message": "Blocked by WAF Referer"
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:80": 1
    }
  }
}' \
http://127.0.0.1:9991/edge/admin/routes/3024
```

3.2 访问网关进行验证

正常请求

发起不含有黑名单词汇的 Referer 请求：

```
curl -i -H "Referer: https://safe-domain.com/index" "http://127.0.0.1:9981/test_common_referer"
```

预期输出：

网关允许通过，响应 HTTP `200`。

异常请求阻断

发起含有恶意黑名单词汇的 Referer 请求：

```
curl -i -H "Referer: https://blocked_referer.com/attack" "http://127.0.0.1:9981/test_common_referer"
```

预期输出：

网关直接进行拦截，返回：

```
HTTP/1.1 403 Forbidden
Content-Length: 22

Blocked by WAF Referer
```

security_common_uri 插件使用文档

1. 插件概览

security_common_uri 插件用于对 HTTP 请求的 URI 路径进行安全规则匹配与阻断检测，防御目录遍历攻击（Path Traversal）、非法后台接口探测、恶意爬取敏感文件等通过请求路径进行的 Web 攻击。

主要应用场景：

- 目录遍历防护：拦截请求路径中包含 `../`、`etc/passwd`、或以 `.git/`、`.svn/` 结尾的非法探测请求。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（`/edge/admin/plugin_metadata/security_common_uri`）进行全局拦截状态与全局默认规则配置。

[!IMPORTANT]

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：`PUT`
- 请求路径：`/edge/admin/plugins/reload`

2.1.1 全局基础配置

字段名	类型	默认值	描述	设置示例
status	Integer	403	被黑名单阻断拦截时，网关返回给客户端的 HTTP 状态码。	403
message	String	"Your request is not allowed."	被阻断拦截时的返回正文消息。	"Blocked by WAF URI"
denylist	Array	无	全局正则黑名单。匹配该名单中任一正则的请求 URI 路径都将被直接阻断。	["blocked_uri", "\\..git/"]
allowlist	Array	无	全局正则白名单。匹配该名单中任一正则的 URI 将直接放行。	["^/static/"]
log_allowed	Boolean	false	是否记录被安全规则放行的正常请求日志。	true

2.2 插件基础属性 (Config)

路由级配置。格式与 数据属性 (Metadata Config) 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径匹配带有 `/test_common_uri` 的接口开启本插件：

- 对 URI 中的敏感词 "blocked_uri" 进行拦截。
- 自定义拦截返回的响应为 403，消息内容为 "Blocked by WAF URI"。

```
curl -i -X PUT \  
-H "Content-Type: application/json" \  
-H "X-API-Key: API_KEY" \  
-d '{  
  "uri": "/test_common_uri*",  
  "plugins": {  
    "security_common_uri": {  
      "denylist": ["blocked_uri"],  
      "status": 403,  
      "message": "Blocked by WAF URI"  
    }  
  },  
  "upstream": {  
    "nodes": {  
      "127.0.0.1:80": 1  
    }  
  }  
}' \  
http://127.0.0.1:9991/edge/admin/routes/3025
```

3.2 访问网关进行验证

正常请求

发起不含有黑名单词汇的 URI 请求：

```
curl -i "http://127.0.0.1:9981/test_common_uri_safe"
```

预期输出：

网关允许通过，响应 HTTP 200。

异常请求阻断

发起含有恶意黑名单词汇的 URI 请求：

```
curl -i "http://127.0.0.1:9981/test_common_uri_blocked_uri"
```

预期输出：

网关直接进行拦截，返回：

```
HTTP/1.1 403 Forbidden  
Content-Length: 18  
  
Blocked by WAF URI
```

security_common_useragent 插件使用文档

1. 插件概览

security_common_useragent 插件用于对 HTTP 请求头部中的 User-Agent 字段进行安全规则匹配与阻断检测，防御如自动化扫描器（如 sqlmap, nmap）、爬虫以及不合规的客户端访问。

主要应用场景：

- 恶意扫描器拦截：拦截 User-Agent 中含有常见的漏洞探测扫描工具、Python、Go 默认 HTTP 客户端等标头的请求。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（`/edge/admin/plugin_metadata/security_common_useragent`）进行全局拦截状态与全局默认规则配置。

[!IMPORTANT]
所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：PUT
- 请求路径：`/edge/admin/plugins/reload`

2.1.1 全局基础配置

字段名	类型	默认值	描述	设置示例
status	Integer	403	被黑名单阻断拦截时，网关返回给客户端的 HTTP 状态码。	403
message	String	"Your request is not allowed."	被阻断拦截时的返回正文消息。	"Blocked by WAF UserAgent"
denylist	Array	无	全局正则黑名单。匹配该名单中任一正则的 User-Agent 值都将被直接阻断。	["blocked_ua", "sqlmap"]
allowlist	Array	无	全局正则白名单。匹配该名单中任一正则的 User-Agent 将直接放行。	["^Mozilla/.*\$"]
log_allowed	Boolean	false	是否记录被安全规则放行的正常请求日志。	true

2.2 插件基础属性 (Config)

路由级配置。格式与 数据属性 (Metadata Config) 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 `/test_common_useragent` 开启本插件：

- 对 User-Agent 中的敏感词 `"blocked_ua"` 进行拦截。
- 自定义拦截返回的响应为 `403`，消息内容为 `"Blocked by WAF UserAgent"`。

```
curl -i -X PUT \  
-H "Content-Type: application/json" \  
-H "X-API-Key: API_KEY" \  
-d '{  
  "uri": "/test_common_useragent",  
  "plugins": {  
    "security_common_useragent": {  
      "denylist": ["blocked_ua"],  
      "status": 403,  

```



```
      "message": "Blocked by WAF UserAgent"
    },
    "upstream": {
      "nodes": {
        "127.0.0.1:80": 1
      }
    }
  } \
http://127.0.0.1:9991/edge/admin/routes/3026
```

3.2 访问网关进行验证

正常请求

发起不含有黑名单词汇的 User-Agent 请求：

```
curl -i -H "User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)"
"http://127.0.0.1:9981/test_common_useragent"
```

预期输出：

网关允许通过，响应 HTTP 200。

异常请求阻断

发起含有恶意黑名单词汇的 User-Agent 请求：

```
curl -i -H "User-Agent: blocked_ua" "http://127.0.0.1:9981/test_common_useragent"
```

预期输出：

网关直接进行拦截，返回：

```
HTTP/1.1 403 Forbidden
Content-Length: 24

Blocked by WAF UserAgent
```

security_corerule

1. 插件概览

`security_corerule` 插件是一个基于 OWASP Core Rule Set (CRS) 的安全检查引擎，用于拦截恶意的 HTTP 请求（如 SQL 注入、跨站脚本攻击、命令注入等）。其典型应用场景包括：

- 启用基础 Web 应用防火墙（WAF）来保护后端服务。
- 针对特殊场景在路由级别上配置白名单和黑名单，跳过特定的请求检查或强制进行拦截。
- 指定针对 URI、参数、Cookie 和请求体的解析方式，以适应特定业务场景的数据格式。

2. 配置详情

2.1 插件静态属性 (Attr Config)

该属性定义在 `edge.env` 文件的 `plugin_attr` 配置中。

字段名	类型	默认值	描述	设置示例
rules	String	无	安全规则所在的相对或绝对路径，需要部署在 <code>conf/security/</code> 以外的路径，请在相对路径末尾添加 <code>/</code> 或使用双层目录结构，例如：配置 <code>my_rules</code> ，仍会查找 <code>conf/security/my_rules</code> ，配置 <code>my_rules/</code> 或 <code>security/my_rules</code> ，则会在当前目录（ <code>edge.env</code> 所在目录）对应的子目录查找，如果未配置，将默认加载 <code>conf/security/rules</code> 目录下的规则集。	"security/rules/"

2.2 插件数据属性 (Metadata Config)

通过管理接口全局配置（路径：`/edge/admin/plugin_metadata/security_corerule`）。

字段名	类型	默认值	描述	设置示例
status	Integer	403	被安全规则拦截后的 HTTP 响应状态码，支持 200 到 599。	403
message	String	无	被安全规则拦截后返回的错误信息内容。	"Request blocked by security rules."
log_matched_maxsize	Integer	128	最大写入日志的匹配内容。	128
ignore_rule	String/Array	无	忽略（不检查）的规则 ID 列表。	["920420"]
ignore_ruleset	String/Array	无	忽略的整个规则集名称列表。	["REQUEST-920-PROTOCOL-ENFORCEMENT"]
parseargs_decoders	Array	["hex", "base64"]	用于参数解析的解码器列表。	["hex", "base64"]
parseargs_collections	Array	["URI_ARGS", "REQUEST_ARGS"]	参与参数解析的采集集合项。	["REQUEST_ARGS"]
disable_parseargs_decoders	Boolean	true	是否完全禁用参数解码处理器。	true

2.3 插件基础属性 (Config)

路由级配置，不仅包含上述数据属性，还支持通过动态变量提取进行规则检查配置。

字段名	类型	默认值	描述	设置示例
status	Integer	同上	同上	同上
message	String	同上	同上	同上
log_matched_maxsize	Integer	同上	同上	同上
ignore_rule	String/Array	同上	同上	同上
ignore_ruleset	String/Array	同上	同上	同上
parseargs_decoders	Array	同上	同上	同上
parseargs_collections	Array	同上	同上	同上
disable_parseargs_decoders	Boolean	同上	同上	同上

2.4 注意事项

- 性能开销：由于 OWASP 核心规则集较大且部分采用了复杂的正则表达式匹配，请合理设置 `ignore_ruleset` 与 `ignore_rule`，以降低 CPU 损耗，或仅在对安全要求较高的路由下开启此插件。

3. 调用与验证示例

3.1 开启插件并配置返回信息

通过管理接口配置包含该插件的路由：

📌 Important

实际调用管理接口时，请求体需要先经过 SM4 加密，返回结果也需要进行解密，详见 [管理接口调用规范](#)。
下面调用示例中的 `ADMIN_ADDR` 和 `API_KEY` 均为环境占位符，执行时请替换为实际地址与真实密钥值。

```
curl -L -X PUT 'http://ADMIN_ADDR/edge/admin/routes/3001' \  
-H 'X-API-KEY: API_KEY' \  
-H 'Content-Type: application/json' \  
-d '{  
  "uri": "/secure_api",  
  "plugins": {  
    "security_corerule": {  
      "status": 403,  
      "message": "Access Denied by WAF.",  
      "ignore_rule": ["920350"]  
    }  
  },  
  "upstream": {  
    "nodes": {  
      "127.0.0.1:8111": 1  
    }  
  }  
}'
```

访问网关验证（发送带有恶意注入特征 of 请求）：

```
## 发起带有 SQL 注入攻击特征的请求  
$ curl -i 'http://127.0.0.1/secure_api?id=1%20OR%201=1'  
  
## 预期响应  
HTTP/1.1 403 Forbidden  
Content-Type: text/plain  
Connection: keep-alive  
  
Access Denied by WAF.
```

security_restrict_expr 插件使用文档

1. 插件概览

`security_restrict_expr` 插件用于使用条件表达式（Edge Expression）进行灵活的高级安全拦截控制。通过自定义条件的组合判定，您可以构建满足多字段联合关联逻辑的访问准入控制。

主要应用场景：

- **定制化安全控制**：当请求的多个标头或参数同时满足特定关系时触发阻断，比如“特定请求头非空且来自于非受信任网段”。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（`/edge/admin/plugin_metadata/security_restrict_expr`）进行全局配置。

****[IMPORTANT]**

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法: `PUT`
- 请求路径: `/edge/admin/plugins/reload**`

字段名	类型	默认值	描述	设置示例
<code>status</code>	Integer	<code>403</code>	被黑名单阻断拦截时，网关返回给客户端的 HTTP 状态码。	<code>403</code>
<code>message</code>	String	<code>"Your request is not allowed."</code>	被阻断拦截时的返回正文消息。	<code>"Blocked by Expression"</code>
<code>black_expr</code>	Array	无	用来触发拦截阻断的条件表达式列表。	<code>[["http_x_block", "=", "yes"]]</code>
<code>white_expr</code>	Array	无	用来直接放行的免检测条件表达式列表。	<code>[["clientip", "in", "10.0.0.0/8"]]</code>

说明

- 关于条件表达式的可用字段及运算符规范，请引用 [Edge 表达式列表](#)。

2.2 插件基础属性 (Config)

路由级配置。格式与 **数据属性 (Metadata Config)** 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 `/test_restrict_expr` 开启本插件：

- 当请求头中带有 `X-Block: yes` 时进行拦截。
- 自定义拦截返回的响应为 `403`，消息内容为 `"Blocked by Expression"`。

```
curl -i -X PUT \  
-H "Content-Type: application/json" \  
-H "X-API-Key: API_KEY" \  
-d '{  
  "uri": "/test_restrict_expr",  
  "plugins": {  
    "security_restrict_expr": {  
      "black_expr": [  
        [ "http_x_block", "=", "yes" ]  
      ],  
      "status": 403,  
      "message": "Blocked by Expression"  
    }  
  },  
  "upstream": {  
    "nodes": {  
      "127.0.0.1:80": 1  
    }  
  }  
}' \  
http://127.0.0.1:9991/edge/admin/routes/3030
```

3.2 访问网关进行验证

异常请求阻断

发起测试请求，带有触发匹配的 `X-Block: yes` 标头：

```
curl -i -H "X-Block: yes" "http://127.0.0.1:9981/test_restrict_expr"
```

预期输出：

网关识别并进行拦截，返回：

```
HTTP/1.1 403 Forbidden
Content-Length: 21

Blocked by Expression
```

正常请求通过

发起测试请求，携带非匹配值 `X-Block: no` 标头：

```
curl -i -H "X-Block: no" "http://127.0.0.1:9981/test_restrict_expr"
```

预期输出：

网关顺利放行，响应 HTTP `200 Passed`。

security_restrict_ip 插件使用文档

1. 插件概览

`security_restrict_ip` 插件用于对 HTTP 请求的客户端 IP 地址或 IP 网段进行安全拦截与豁免配置。它支持配置 IP 黑名单（Blacklist）和白名单（Whitelist），并根据指定的规则对请求进行阻断或放行。

主要应用场景：

- **黑客 IP 封禁**：将高频探测、扫描端口或发布垃圾信息的公网 IP/网段直接加入黑名单限制。
- **后台仅白名单访问**：对特定的管理服务路由，只允许受信的合作伙伴或内网 IP 段（如 `10.0.0.0/8`）发起连接。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（`/edge/admin/plugin_metadata/security_restrict_ip`）进行全局拦截状态与全局默认规则配置。

[!IMPORTANT]

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：`PUT`
- 请求路径：`/edge/admin/plugins/reload`

字段名	类型	默认值	描述	设置示例
key	String	"\${remote_addr}"	提取客户端 IP 值的变量模板。如果是 CDN 等前置网关，可使用 X-Forwarded-For 提取。	"\${http_x_forwarded_for}"
status	Integer	403	被黑名单阻断拦截时，网关返回给客户端的 HTTP 状态码。	403
message	String	"Your request is not allowed."	被阻断拦截时的返回正文消息。	"Blocked by IP Blacklist"
blacklist	Array	无	IP/CIDR 黑名单列表。	["127.0.0.1", "192.168.1.0/24"]
whitelist	Array	无	IP/CIDR 白名单列表。	["10.0.0.0/8"]
bypass_missing_key	Boolean	true	若 IP 提取失败时，是否默认直接放行。	false
bypass_error_key	Boolean	true	若 IP 提取异常时，是否放行。	false

2.2 插件基础属性 (Config)

路由级配置。格式与 数据属性 (Metadata Config) 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由 (以黑名单为例)

使用 Admin 管理接口创建或更新路由，为路径 `/test_restrict_ip` 开启本插件：

- 对源 IP `"127.0.0.1"` 进行黑名单拦截。
- 自定义拦截返回的响应为 `403`，消息内容为 `"Blocked by IP Blacklist"`。

```
curl -i -X PUT \  
-H "Content-Type: application/json" \  
-H "X-API-Key: API_KEY" \  
-d '{  
  "uri": "/test_restrict_ip",  
  "plugins": {  
    "security_restrict_ip": {  
      "blacklist": ["127.0.0.1"],  
      "status": 403,  
      "message": "Blocked by IP Blacklist"  
    }  
  },  
  "upstream": {  
    "nodes": {  
      "127.0.0.1:80": 1  
    }  
  }  
}' \  
http://127.0.0.1:9991/edge/admin/routes/3027
```

3.2 访问网关进行验证

异常请求阻断 (以 127.0.0.1 本地测试发起)

发起测试请求：

```
curl -i "http://127.0.0.1:9981/test_restrict_ip"
```

预期输出：

由于发起 IP 为 127.0.0.1 属于黑名单，网关直接拦截，返回：

```
HTTP/1.1 403 Forbidden
Content-Length: 24

Blocked by IP Blacklist
```

security_restrict_redis 插件使用文档

1. 插件概览

security_restrict_redis 插件用于在网关层实时连接外部的 Redis 服务器（或 Redis 集群），根据动态提取的用户或客户端标识（Key）进行查询，并基于 Redis 返回的数据结果通过规则表达式（risk_expr）判定是否阻断请求。这非常适合与外部风控系统、动态黑名单管理等场景深度结合。

主要应用场景：

- 实时风控系统对接：外部安全引擎识别出恶意请求后，写入 Redis（例如键 "0-baduser"，值为 "0"），网关实时查询并在发现判定值匹配时直接阻断。
- 高频封禁与解封：利用 Redis 的过期时间机制（TTL）实现全自动、无感知的临时封禁机制。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（/edge/admin/plugin_metadata/security_restrict_redis）进行全局拦截状态与全局默认规则配置。

**[!IMPORTANT]

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：PUT
- 请求路径：/edge/admin/plugins/reload**

2.1.1 全局基础配置

字段名	类型	默认值	描述	设置示例
key	String	"\${username}"	提取用户或客户端标识值的变量模板。	"\${http_x_user}"
key_prefix	String	"0-"	拼接至 Redis 查询 Key 之前的前缀。	"0-"
status	Integer	403	被规则匹配阻断拦截时，网关返回给客户端的 HTTP 状态码。	403
message	String	"Your request is not allowed."	被阻断拦截时的返回正文消息。	"Blocked by Redis Rule"
risk_expr	Array	[["redis.1", "=", "0"]]	基于返回的 Redis 数组结果的判定条件表达式。	[["redis.1", "=", "0"]]
redis_dc	String	"TEST"	匹配的 Redis 数据中心。	"TEST"
redis_conf	Object	见传输配置	Redis 连接网络配置属性。	见传输配置。
redis_exptime	Number	60	正常查询结果在网关本地的缓存有效期（秒）。	60
redis_none_exptime	Number	10	查询结果为空时的本地缓存有效期（秒）。	10
bypass_missing_key	Boolean	true	若 Key 变量提取为空时是否直接放行。	false
bypass_error_key	Boolean	true	若 Redis 连接或查询出错时是否放行。	false

2.1.2 Redis 网络传输配置 (redis_conf)

字段名	类型	默认值	描述	设置示例
CLUSTER_[a-zA-Z0-9]+ (如 CLUSTER_TEST)	Array	[["127.0.0.1", 6379]]	Redis 节点地址列表。每一个节点地址格式为 [ip, port, ...]。	[["127.0.0.1", 16379]]
MODE	String	"redis"	连接模式。可选值: "redis" (单点模式), "rediscluster" (集群模式), "proxycluster" (代理集群模式)。	"redis"
AUTH	String	无	Redis 认证密码。	"<REDIS_PASSWORD>"
POOL	String	无	连接池的自定义名称。	"my_pool"
POOL_SIZE	Integer	2	连接池大小限制。	10
BACKLOG	Integer	-1	连接池请求积压的最大大小。	100
TIMEOUT	Number	1	网络读写与建立连接的超时时间 (秒)。	2
CONNECT_TIMEOUT	Number	1	建立连接的超时时间 (秒)。	1
SEND_TIMEOUT	Number	1	向 Redis 发送数据的超时时间 (秒)。	1
READ_TIMEOUT	Number	1	从 Redis 读取响应的超时时间 (秒)。	1
DATABASE	Integer	0	数据库索引 (DB Index, 仅单点模式生效)。	0
MAX_CONNECT_NODES	Integer	2	允许连接的最大 Redis 节点个数。	4
MAX_IDLE_TIME	Integer	60	连接池中空闲连接的超时时间 (秒)。	60
CONNECTION_MAX_SIZE	Integer	1024	每个网关进程的连接池最大容量。	100
LATENCY_METRICS	Boolean	false	是否开启延迟指标监测收集。	true
BREAKER_NODES	Number	0.5	触发熔断节点的比率阈值。	0.5
LIMIT_NODES	Number	0.5	限制节点的比率阈值。	0.5
LIMIT_COUNT	Object	无	限制规则的计数频率配置。	见配置示例

2.2 插件基础属性 (Config)

路由级配置。格式与 数据属性 (Metadata Config) 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 /test_restrict_redis 开启本插件：

- 以请求头中的 x-User 作为 Key 提取。前缀为 "0-"。
- 连接部署在 127.0.0.1:16379 的 Redis 服务，密码为 "<REDIS_PASSWORD>"。
- 当从 Redis 中查到的键的值第一项为 "0" 时，判定为高风险阻断，返回 403 和 "Blocked by Redis Rule"。

```
curl -i -X PUT \  
  -H "Content-Type: application/json" \  
  -H "X-API-Key: API_KEY" \  
  -d '{  
    "uri": "/test_restrict_redis",  
    "plugins": {  
      "security_restrict_redis": {  
        "key": "${http_x_user}",  
        "risk_expr": [  
          ["redis.1", "=", "0"]  
        ],  
        "status": 403,  

```



```
    "message": "Blocked by Redis Rule",
    "redis_dc": "TEST",
    "redis_conf": {
      "CLUSTER_TEST": [
        ["127.0.0.1", 16379]
      ],
      "AUTH": "<REDIS_PASSWORD>"
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:80": 1
    }
  }
}
}' \
http://127.0.0.1:9991/edge/admin/routes/3031
```

3.2 访问网关进行验证

正常请求

预先向 Redis 中写入允许的键（O-gooduser，值为 "1"）：

```
redis-cli -p 16379 -a <REDIS_PASSWORD> SET O-gooduser 1
```

向网关发送携带 X-User: gooduser 的请求：

```
curl -i -H "X-User: gooduser" "http://127.0.0.1:9981/test_restrict_redis"
```

预期输出：

网关允许通过，响应 HTTP 200 Passed。

异常请求阻断

预先向 Redis 中写入被限制的黑名单键（O-baduser，值为 "0"）：

```
redis-cli -p 16379 -a <REDIS_PASSWORD> SET O-baduser 0
```

向网关发送携带 X-User: baduser 的请求：

```
curl -i -H "X-User: baduser" "http://127.0.0.1:9981/test_restrict_redis"
```

预期输出：

网关识别该用户在 Redis 中的值，命中风险规则，直接进行拦截：

```
HTTP/1.1 403 Forbidden
Content-Length: 21

Blocked by Redis Rule
```

security_restrict_uri 插件使用文档

1. 插件概览

`security_restrict_uri` 插件用于对 HTTP 请求的 URI 路径进行安全规则黑白名单限制，防御非法敏感接口探测与目录越权。

主要应用场景：

- 敏感接口路径拦截：将包含特定管理端或者配置信息的 URI 路径加入黑名单限制。
- 特定开放路径放行：对开放的 API（如 `/api/public/`）加入白名单以防误杀。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（`/edge/admin/plugin_metadata/security_restrict_uri`）进行全局配置。

**[!IMPORTANT]

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：`PUT`
- 请求路径：`/edge/admin/plugins/reload` **

字段名	类型	默认值	描述	设置示例
<code>status</code>	Integer	<code>403</code>	被黑名单阻断拦截时，网关返回给客户端的 HTTP 状态码。	<code>403</code>
<code>message</code>	String	<code>"Your request is not allowed."</code>	被阻断拦截时的返回正文消息。	<code>"Blocked by URI Blacklist"</code>
<code>blacklist</code>	Array	无	URI 黑名单路径列表。	<code>["/admin/config", "/test_restrict_uri/blocked"]</code>
<code>whitelist</code>	Array	无	URI 白名单路径列表。	<code>["/api/public"]</code>
<code>bypass_missing_key</code>	Boolean	<code>true</code>	若 URI 提取失败时，是否默认直接放行。	<code>false</code>
<code>bypass_error_key</code>	Boolean	<code>true</code>	若 URI 提取异常时，是否放行。	<code>false</code>

2.2 插件基础属性 (Config)

路由级配置。格式与 数据属性 (Metadata Config) 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由 (以黑名单为例)

使用 Admin 管理接口创建或更新路由，为路径 `/*` 开启本插件：

- 对源 URI `/test_restrict_uri/blocked` 进行黑名单拦截。
- 自定义拦截返回的响应为 `403`，消息内容为 `"Blocked by URI Blacklist"`。

```
curl -i -X PUT \  
-H "Content-Type: application/json" \  
-H "X-API-Key: API_KEY" \  
-d '{  
  "uri": "/*",  
  "plugins": {  
    "security_restrict_uri": {
```

```
    "blacklist": ["/test_restrict_uri/blocked"],
    "status": 403,
    "message": "Blocked by URI Blacklist"
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:80": 1
    }
  }
} \
http://127.0.0.1:9991/edge/admin/routes/3028
```

3.2 访问网关进行验证

异常路径阻断

发起测试请求：

```
curl -i "http://127.0.0.1:9981/test_restrict_uri/blocked"
```

预期输出：

由于请求 URI 匹配了黑名单，网关直接拦截，返回：

```
HTTP/1.1 403 Forbidden
Content-Length: 24

Blocked by URI Blacklist
```

security_restrict_user 插件使用文档

1. 插件概览

`security_restrict_user` 插件用于根据请求中的用户标识（如用户账户、ID、角色等）进行访问限制或免除，提供精细化的用户级别准入控制。

主要应用场景：

- **恶意账号封禁**：直接拦截已被标记为风控危险账号、异常行为账户的请求。
- **高级用户豁免**：仅放行已购 VIP 或属于白名单内的受信任用户账号。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（`/edge/admin/plugin_metadata/security_restrict_user`）进行全局配置。

****[IMPORTANT]**

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：`PUT`
- 请求路径：`/edge/admin/plugins/reload` ******

字段名	类型	默认值	描述	设置示例
key	String	"\${username}"	提取用户标识值的变量模板。	"\${http_x_user}"
status	Integer	403	被黑名单阻断拦截时，网关返回给客户端的 HTTP 状态码。	403
message	String	"Your request is not allowed."	被阻断拦截时的返回正文消息。	"Blocked by User Blacklist"
blacklist	Array	无	用户黑名单列表。	["baduser", "blocked_buyer"]
whitelist	Array	无	用户白名单列表。	["gooduser", "vip_customer"]
bypass_missing_key	Boolean	true	若用户标识提取失败时，是否默认直接放行。	false
bypass_error_key	Boolean	true	若用户标识提取异常时，是否放行。	false

2.2 插件基础属性 (Config)

路由级配置。格式与 数据属性 (Metadata Config) 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由 (以黑名单为例)

使用 Admin 管理接口创建或更新路由，为路径 `/test_restrict_user` 开启本插件：

- 以请求头中的 `X-User` 作为用户标识进行校验。
- 对用户 `"baduser"` 进行黑名单拦截。
- 自定义拦截返回的响应为 `403`，消息内容为 `"Blocked by User Blacklist"`。

```
curl -i -X PUT \  
  -H "Content-Type: application/json" \  
  -H "X-API-Key: API_KEY" \  
  -d '{  
    "uri": "/test_restrict_user",  
    "plugins": {  
      "security_restrict_user": {  
        "key": "${http_x_user}",  
        "blacklist": ["baduser"],  
        "status": 403,  
        "message": "Blocked by User Blacklist"  
      }  
    },  
    "upstream": {  
      "nodes": {  
        "127.0.0.1:80": 1  
      }  
    }  
  }' \  
  http://127.0.0.1:9991/edge/admin/routes/3029
```

3.2 访问网关进行验证

异常请求阻断

发起测试请求，带有被禁用的用户 Header：

```
curl -i -H "X-User: baduser" "http://127.0.0.1:9981/test_restrict_user"
```

预期输出：
网关识别并进行拦截，返回：

```
HTTP/1.1 403 Forbidden
Content-Length: 26

Blocked by User Blacklist
```

security_super_expr 插件使用文档

1. 插件概览

`security_super_expr` 插件属于超级白名单安全插件。它用于在网关层通过自定义条件表达式（Edge Expression）匹配请求特征，一旦匹配成功，网关会跳过/忽略后续指定的安全控制及黑名单限制插件，从而防止特定合规请求被误拦截。

主要应用场景：

- **定制化复合免杀控制**：当请求的多个标头或参数同时满足特定关系时免除后续限制，比如“来源 IP 在公司出口且请求头携带特权测试标志”。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（`/edge/admin/plugin_metadata/security_super_expr`）进行全局配置。

****[!IMPORTANT]**
所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：`PUT`
- 请求路径：`/edge/admin/plugins/reload` ******

字段名	类型	默认值	描述	设置示例
<code>white_expr</code>	Array	无	边缘条件表达式格式的黑名单判定规则。	<code>[["http_x_safe", "==", "yes"]]</code>
<code>ignore_plugins</code>	Array	见说明	匹配白名单条件后要跳过忽略的后续插件列表。	<code>["security_restrict_ip"]</code>

说明

- `ignore_plugins` 默认配置为跳过如下常见安全及限流插件：
 - `security_restrict_ip`
 - `security_restrict_uri`
 - `security_restrict_user`
 - `security_restrict_expr`
 - `security_restrict_redis`
 - `traffic_limit_count`

2.2 插件基础属性 (Config)

路由级配置。格式与 数据属性 (Metadata Config) 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 `/test_super_expr` 开启本插件：

- 同时配置 `security_restrict_expr` 拦截 `X-Block: yes` 请求。
- 配置 `security_super_expr`，当请求头中含有 `X-Safe: yes` 时忽略后续的 `security_restrict_expr` 检查。

```
curl -i -X PUT \
-H "Content-Type: application/json" \
-H "X-API-Key: API_KEY" \
-d '{
  "uri": "/test_super_expr",
  "plugins": {
    "security_restrict_expr": {
      "black_expr": [
        ["http_x_block", "==", "yes"]
      ],
      "status": 403,
      "message": "Blocked by restrict expr"
    },
    "security_super_expr": {
      "white_expr": [
        ["http_x_safe", "==", "yes"]
      ],
      "ignore_plugins": ["security_restrict_expr"]
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:80": 1
    }
  }
}' \
http://127.0.0.1:9991/edge/admin/routes/3036
```

3.2 访问网关进行验证

豁免放行通过

发起携带黑名单特征但命中超级白名单表达式的测试请求：

```
curl -i -H "X-Block: yes" -H "X-Safe: yes" "http://127.0.0.1:9981/test_super_expr"
```

预期输出：

尽管请求命中 `security_restrict_expr` 拦截规则，但由于 `X-Safe: yes` 匹配了 `security_super_expr` 的超级白名单条件，直接忽略了后面的限制插件，响应 HTTP 200 Passed。

security_super_ip 插件使用文档

1. 插件概览

`security_super_ip` 插件属于超级白名单安全插件。它用于在网关层匹配客户端 IP，一旦匹配成功，网关会跳过/忽略后续指定的安全控制及黑名单限制插件，从而防止受信客户端被误杀或误拦截。

主要应用场景：

- **合作伙伴出口豁免：** 将合作伙伴或受信系统的 IP/网段加入超级白名单，使其不受后续常规防注入、WAF 限流等规则的影响。
- **内网开发调试测试豁免：** 避免内网员工开发调试时触发风控拦截。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（`/edge/admin/plugin_metadata/security_super_ip`）进行全局配置。

**[!IMPORTANT]

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

• 请求方法：`PUT`

• 请求路径：`/edge/admin/plugins/reload` **

字段名	类型	默认值	描述	设置示例
key	String	<code>"\${remote_addr}"</code>	提取客户端 IP 值的变量模板。如果是 CDN 等前置网关，可使用 X-Forwarded-For 提取。	<code>"\${http_x_forwarded_for}"</code>
superlist	Array	无	超级 IP/CIDR 白名单列表。	<code>["127.0.0.1", "10.0.0.0/8"]</code>
ignore_plugins	Array	见说明	匹配白名单后要跳过忽略的后续插件列表。	<code>["security_restrict_ip"]</code>

说明

• `ignore_plugins` 默认配置为跳过如下常见安全及限流插件：

◦ `security_restrict_ip`

◦ `security_restrict_uri`

◦ `security_restrict_user`

◦ `security_restrict_expr`

◦ `security_restrict_redis`

◦ `traffic_limit_count`

2.2 插件基础属性 (Config)

路由级配置。格式与 **数据属性 (Metadata Config)** 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 `/test_super_ip` 开启本插件：

- 同时配置 `security_restrict_ip` 插件拦截 `127.0.0.1`，并且配置 `security_super_ip` 对 `127.0.0.1` 豁免。
- `security_super_ip` 设置跳过 `security_restrict_ip` 插件拦截。

```
curl -i -X PUT \
-H "Content-Type: application/json" \
-H "X-API-Key: API_KEY" \
-d '{
  "uri": "/test_super_ip",
  "plugins": {
    "security_restrict_ip": {
      "blacklist": ["127.0.0.1"],
      "status": 403,
      "message": "Blocked by IP restriction"
    },
    "security_super_ip": {
      "superlist": ["127.0.0.1"],
      "ignore_plugins": ["security_restrict_ip"]
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:80": 1
    }
  }
}' \
http://127.0.0.1:9991/edge/admin/routes/3034
```

3.2 访问网关进行验证

豁免放行通过

发起测试请求：

```
curl -i "http://127.0.0.1:9981/test_super_ip"
```

预期输出：

尽管发起 IP 127.0.0.1 在 security_restrict_ip 的黑名单中，但因其命中了 security_super_ip 的 superlist，该限制插件被跳过，网关顺利放行，响应 HTTP 200 Passed。

security_super_redis 插件使用文档

1. 插件概览

security_super_redis 插件属于超级白名单安全插件。它用于在网关层实时连接外部的 Redis 服务器（或 Redis 集群），根据动态提取的用户或客户端标识（Key）进行查询，一旦匹配符合条件表达式（risk_expr），网关会跳过/忽略后续指定的安全控制及黑名单限制插件，实现动态的超级白名单豁免通道。

主要应用场景：

- 动态特权白名单免杀：外部安全引擎计算出免检测用户账户并写入 Redis，网关实时读取并配置跳过后续的各种限制性安全插件。
- 高等级用户绿色通道：在大促或高负载情况下，针对特定 VIP 用户账号绕过网关限流插件（traffic_limit_count）。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（`/edge/admin/plugin_metadata/security_super_redis`）进行全局配置。

****[!IMPORTANT]**

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：`PUT`
- 请求路径：`/edge/admin/plugins/reload`

2.1.1 全局基础配置

字段名	类型	默认值	描述	设置示例
key	String	"\${remote_addr}"	提取用户或客户端标识值的变量模板。	"\${http_x_user}"
key_prefix	String	"w-"	拼接至 Redis 查询 Key 之前的前缀。	"w-"
risk_expr	Array	[["redis.1", "=", "super"]]	基于返回的 Redis 数组结果判定是否命中超级白名单的条件表达式。	[["redis.1", "=", "super"]]
ignore_plugins	Array	见说明	匹配超级白名单后要跳过忽略的后续插件列表。	["security_restrict_redis"]
redis_dc	String	"TEST"	匹配的 Redis 数据中心。	"TEST"
redis_conf	Object	见传输配置	Redis 连接网络配置属性。	见传输配置。
redis_exptime	Number	60	正常查询结果在网关本地的缓存有效期（秒）。	60
redis_none_exptime	Number	10	查询结果为空时的本地缓存有效期（秒）。	10
bypass_missing_key	Boolean	true	若 Key 变量提取为空时是否默认直接放行。	true
bypass_error_key	Boolean	true	若 Redis 连接或查询出错时是否放行。	true

2.1.2 Redis 网络传输配置 (redis_conf)

字段名	类型	默认值	描述	设置示例
CLUSTER_[a-zA-Z0-9]+ (如 CLUSTER_TEST)	Array	[["127.0.0.1", 6379]]	Redis 节点地址列表。每一个节点地址格式为 [ip, port, ...]。	[["127.0.0.1", 16379]]
MODE	String	"redis"	连接模式。可选值: "redis" (单点模式), "rediscluster" (集群模式), "proxycluster" (代理集群模式)。	"redis"
AUTH	String	无	Redis 认证密码。	"<REDIS_PASSWORD>"
POOL	String	无	连接池的自定义名称。	"my_pool"
POOL_SIZE	Integer	2	连接池大小限制。	10
BACKLOG	Integer	-1	连接池请求积压的最大大小。	100
TIMEOUT	Number	1	网络读写与建立连接的超时时间（秒）。	2
CONNECT_TIMEOUT	Number	1	建立连接的超时时间（秒）。	1
SEND_TIMEOUT	Number	1	向 Redis 发送数据的超时时间（秒）。	1
READ_TIMEOUT	Number	1	从 Redis 读取响应的超时时间（秒）。	1
DATABASE	Integer	0	数据库索引 (DB Index, 仅单点模式生效)。	0
MAX_CONNECT_NODES	Integer	2	允许连接的最大 Redis 节点个数。	4
MAX_IDLE_TIME	Integer	60	连接池中空闲连接的超时时间（秒）。	60
CONNECTION_MAX_SIZE	Integer	1024	每个网关进程的连接池最大容量。	100
LATENCY_METRICS	Boolean	false	是否开启延迟指标监测收集。	true
BREAKER_NODES	Number	0.5	触发熔断节点的比率阈值。	0.5
LIMIT_NODES	Number	0.5	限制节点的比率阈值。	0.5
LIMIT_COUNT	Object	无	限制规则的计数频率配置。	见配置示例

2.2 插件基础属性 (Config)

路由级配置。格式与 数据属性 (Metadata Config) 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 `/test_super_redis` 开启本插件：

- 同时配置 `security_restrict_redis` 拦截 `0-` 前缀且值为 `"0"` 的账号。
- 配置 `security_super_redis` 提取 `X-User` 头并加上前缀 `w-` 去 Redis 中查询，若其值为 `"super"` 则判定命中超级白名单，并配置忽略 `security_restrict_redis` 插件。

```
curl -i -X PUT \  
-H "Content-Type: application/json" \  
-H "X-API-Key: API_KEY" \  
-d '{  
  "uri": "/test_super_redis",  
  "plugins": {  
    "security_restrict_redis": {  
      "key": "${http_x_user}",  
      "risk_expr": [  
        ["redis.1", "=", "0"]  
      ],  
      "status": 403,  
      "message": "Blocked by Redis Rule",  
      "redis_dc": "TEST",  
      "redis_conf": {  
        "CLUSTER_TEST": [  
          ["127.0.0.1", 16379]  
        ],  
        "AUTH": "<REDIS_PASSWORD>"  
      }  
    },  
    "security_super_redis": {  
      "key": "${http_x_user}",  
      "risk_expr": [  
        ["redis.1", "=", "super"]  
      ],  
      "ignore_plugins": ["security_restrict_redis"],  
      "redis_dc": "TEST",  
      "redis_conf": {  
        "CLUSTER_TEST": [  
          ["127.0.0.1", 16379]  
        ],  
        "AUTH": "<REDIS_PASSWORD>"  
      }  
    }  
  },  
  "upstream": {  
    "nodes": {  
      "127.0.0.1:80": 1  
    }  
  }  
}' \  
http://127.0.0.1:9991/edge/admin/routes/3032
```

3.2 访问网关进行验证

豁免放行通过

预先向 Redis 中写入超级白名单键（w-super_user，值为 "super"）：

```
redis-cli -p 16379 -a <REDIS_PASSWORD> SET W-super_user super
```

向网关发送携带 x-User: super_user 的请求：

```
curl -i -H "X-User: super_user" "http://127.0.0.1:9981/test_super_redis"
```

预期输出：
尽管由于没有 o-super_user 的限制，但是因为匹配了超级白名单规则，后续可能存在的限制规则直接被跳过，网关直接放行并响应 HTTP 200 Passed。

security_super_user 插件使用文档

1. 插件概览

security_super_user 插件属于超级白名单安全插件。它用于在网关层匹配用户标识，一旦匹配成功，网关会跳过/忽略后续指定的安全控制及黑名单限制插件，从而防止授信用户被误杀或误拦截。

主要应用场景：

- 管理员账户放行：将平台管理员或受信账号加入超级白名单，使其不受后续常规防注入、WAF 限流等规则的影响。
- 特权测试账户豁免：免除自动化压力测试账号触发风控拦截。

2. 配置详情

2.1 插件数据属性 (Metadata Config)

通过管理端接口（/edge/admin/plugin_metadata/security_super_user）进行全局配置。

**[IMPORTANT]

所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：PUT
- 请求路径：/edge/admin/plugins/reload**

字段名	类型	默认值	描述	设置示例
key	String	"\${username}"	提取用户标识值的变量模板。	"\${http_x_user}"
superlist	Array	无	超级用户白名单列表。	["super_user", "internal_admin"]
ignore_plugins	Array	见说明	匹配白名单后要跳过忽略的后续插件列表。	["security_restrict_user"]

说明

- ignore_plugins 默认配置为跳过如下常见安全及限流插件：
 - security_restrict_ip
 - security_restrict_uri
 - security_restrict_user
 - security_restrict_expr
 - security_restrict_redis
 - traffic_limit_count

2.2 插件基础属性 (Config)

路由级配置。格式与 数据属性 (Metadata Config) 保持完全一致，可对单个具体路由覆盖全局配置。

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 `/test_super_user` 开启本插件：

- 同时配置 `security_restrict_user` 插件限制用户 `"baduser"` 并且只允许 `"gooduser"`，而且配置 `security_super_user` 对 `"super_user"` 豁免。
- `security_super_user` 设置以请求头 `X-User` 判定，并跳过 `security_restrict_user` 插件拦截。

```
curl -i -X PUT \  
-H "Content-Type: application/json" \  
-H "X-Api-Key: API_KEY" \  
-d '{  
  "uri": "/test_super_user",  
  "plugins": {  
    "security_restrict_user": {  
      "key": "${http_x_user}",  
      "blacklist": ["super_user"],  
      "status": 403,  
      "message": "Blocked by user restriction"  
    },  
    "security_super_user": {  
      "key": "${http_x_user}",  
      "superlist": ["super_user"],  
      "ignore_plugins": ["security_restrict_user"]  
    }  
  },  
  "upstream": {  
    "nodes": {  
      "127.0.0.1:80": 1  
    }  
  }  
}' \  
http://127.0.0.1:9991/edge/admin/routes/3035
```

3.2 访问网关进行验证

豁免放行通过

发起测试请求：

```
curl -i -H "X-User: super_user" "http://127.0.0.1:9981/test_super_user"
```

预期输出：

尽管用户 `super_user` 在 `security_restrict_user` 的黑名单中，但因其命中了 `security_super_user` 的 `superlist`，该限制插件被跳过，网关顺利放行，响应 HTTP `200 Passed`。

server_info 插件使用文档

1. 插件概览

`server_info` 插件是一个全局范围的系统插件，用于向外暴露当前网关节点的标识（ID）、版本号、主机名（hostname）以及启动时间等核心元数据接口，主要用于存活心跳与外部监控指标抓取。

主要应用场景：

- 节点元数据自动发现：监控平面通过拉取该接口获取各物理节点的版本、软件运行位置和存活状态。
- 定时器健康度监测：查看 OpenResty 后台待执行和正在运行的定时任务并发数。

2. 配置详情

2.1 插件静态属性 (Attr Config)

存储于 `edge.env` 文件的 `plugin_attr` 下 `server_info` 属性，默认配置即可正常使用，无需手动调整。

2.2 插件数据属性 (Metadata Config)

无。

2.3 插件基础属性 (Config)

无。

3. 调用与验证示例

`server_info` 插件开启后，外部可以通过网关对应的端口直接访问暴露的接口。

3.1 步骤 1：开启 `server_info` 插件公开路由

在 `edge.env` 文件的 `plugin_attr` 中配置 `server_info` 属性（默认开启且导出路径为 `/edge/server_info`）。

3.2 访问网关进行验证

1. 获取基础节点状态数据

```
curl -i http://127.0.0.1:9981/edge/server_info
```

预期响应输出（返回当前网关节点元数据）：

```
HTTP/1.1 200 OK
Content-Type: application/json
Connection: keep-alive

{"boot_time":1782210000,"load_time":1782210500,"hostname":"gateway-node-01","version":"3.0.0","id":"abcde12345","workspace":"devel/edge","worksite":"edge"}
```

traceid 插件使用说明书

1. 插件概览

`traceid` 插件用于在网关层为每个请求自动生成或传递一个全局唯一的追踪 ID（Trace ID），并将其注入至请求头中转发给上游后端服务。该 ID 还可以选择性地回显在响应头中返回给客户端。

主要应用场景：

- **全链路追踪**：作为系统入口，为请求分配唯一的 Trace ID，方便与上游微服务、数据库日志、调用链追踪系统（如 Zipkin, Jaeger 等）打通。
- **问题快速定位**：当客户端请求出错时，可以通过响应头中回显的 Trace ID 快速在网关及后端的日志中检索、定位根因。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

通过管理接口进行全局配置（API 路径：`/edge/admin/plugin_metadata/traceid`），用于设定全局默认的追踪行为。

[!IMPORTANT]
所有插件的数据属性（Metadata Config）变更后，必须执行一次插件热加载操作以使配置生效：

- 请求方法：`PUT`
- 请求路径：`/edge/admin/plugins/reload`

您可以在配置多个插件的数据属性后，统一执行一次热加载操作。

字段名	类型	默认值	描述	设置示例
<code>header_name</code>	String	<code>"X-EDGE-TraceID"</code>	追踪标头名。长度限制在 1 至 64 个字符。	<code>"X-Custom-Trace-ID"</code>
<code>ignore_header</code>	Boolean	<code>false</code>	是否忽略客户端请求中已携带的同名 Trace ID 头字段。若为 <code>true</code> ，网关会无条件忽略客户端传入的值并强制生成新 Trace ID；若为 <code>false</code> ，则优先继承客户端的值。	<code>true</code>
<code>show_resp_header</code>	Boolean	<code>false</code>	是否在响应给客户端的 Header 中回显追踪 ID。	<code>true</code>

2.3 插件基础属性 (Config)

路由级配置，可覆盖全局定义的插件数据属性 (Metadata Config)。

字段名	类型	默认值	描述	设置示例
<code>header_name</code>	String	<code>"X-EDGE-TraceID"</code>	同上	<code>"X-Route-Trace-ID"</code>
<code>ignore_header</code>	Boolean	<code>false</code>	同上	<code>true</code>
<code>show_resp_header</code>	Boolean	<code>false</code>	同上	<code>true</code>

3. 调用与验证示例

3.1 创建并启用路由

使用 Admin 管理接口创建或更新路由，为路径 `/api/trace` 开启 `traceid` 插件，将其配置为：自定义标头 `X-My-Trace-ID` 并在响应头中回显。

Important

调用示例中不包含真实敏感密钥，请将占位符 `API_KEY` 替换为您实际环境中的 Admin API Key。

```
curl -i -X PUT \
-H "Content-Type: application/json" \
-H "X-API-Key: API_KEY" \
-d '{
  "uri": "/api/trace",
  "plugins": {
    "traceid": {
      "header_name": "X-My-Trace-ID",
      "ignore_header": true,
      "show_resp_header": true
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:8811": 1
    },
    "type": "roundrobin"
  }
}' \
http://127.0.0.1:9991/edge/admin/routes/1001
```

3.2 访问网关进行验证

向网关的 HTTP 监听端口发起请求，验证响应头中是否包含自定义的追踪标识：

```
curl -i http://127.0.0.1:9981/api/trace
```

预期响应输出 (部分)：

```
HTTP/1.1 200 OK
Content-Type: text/plain
Transfer-Encoding: chunked
Connection: keep-alive
X-My-Trace-ID: 0a00000168cb7f2d00000001
```

同时，后端上游服务器接收到的 HTTP 请求中，也将携带同名的追踪标头：`X-My-Trace-ID: 0a00000168cb7f2d00000001`。

tracing 插件使用文档

1. 插件概览

`tracing` 插件用于在网关层动态修改、新增或覆盖请求和响应的 HTTP Header。用户可使用系统变量（如客户端真实 IP、随机 UUID 等）组装追踪标识并将其透传至上游服务或客户端，从而实现全链路的可观测性监控。

主要应用场景：

- 请求链路 **Trace ID** 注入：自动向经过网关的 HTTP 请求注入唯一的 Trace ID 标头，方便分布式微服务架构中统一进行日志抓取与追踪。
- 获取并透传网络元数据：提取当前请求的客户端真实地址及地理位置信息并封装入 Header 透传。

2. 配置详情

2.1 插件静态属性 (Attr Config)

无。

2.2 插件数据属性 (Metadata Config)

无。

2.3 插件基础属性 (Config)

路由级配置。

字段名	类型	默认值	描述	设置示例
bypass_missing_key	Boolean	true	若解析变量的值为空时，是否直接忽略该标头注入。	true
bypass_error_key	Boolean	true	解析变量出错时是否忽略。	true
header_name	String	"X-EDGE-Trace-Info"	触发 show_resp_header 时，用于在响应中携带已追踪修改之标头列表的 Header 名称。	"X-Trace-Info"
show_resp_header	Boolean	false	是否在 HTTP 响应头中添加追踪指示 Header，表明哪些追踪头已被网关设置或覆盖。	true
headers	Object	无	需要注入的请求头及其对应的包含变量的格式串映射。最大支持 20 个 Key。	{"X-Request-Id": "\${uuid}"}
replace_header	Boolean	false	是否替换覆盖请求中已存在的同名请求头。	true
resp_headers	Object	无	需要注入的响应头及其对应的包含变量的格式串映射。最大支持 20 个 Key。	{"X-Trace-Response-Time": "\${upstream_response_time}"}
replace_resp_header	Boolean	false	是否替换覆盖响应中已存在的同名响应头。	true

3. 调用与验证示例

本示例介绍如何创建路由并启用 tracing 插件，在发送给上游的请求中注入唯一 Trace ID，在返回的响应中注入上游处理耗时。

 **Important**

调用示例中不包含真实敏感密钥，请将占位符 `API_KEY` 替换为您实际环境中的 Admin API Key。

3.1 步骤 1：创建启用 tracing 插件的路由

创建路由关联路径 `/test_trace`，并在插件配置中定义 `headers` 和 `resp_headers`：

```
curl -i -X PUT \  
  -H "Content-Type: application/json" \  
  -H "X-API-Key: API_KEY" \  
  -d '{  
    "uri": "/test_trace",  
    "plugins": {  
      "tracing": {  
        "headers": {
```



```
        "X-Trace-Id": "${uuid}"
      },
      "replace_header": true,
      "resp_headers": {
        "X-Process-Time": "${request_time}"
      },
      "replace_resp_header": true,
      "show_resp_header": true
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:8811": 1
    },
    "type": "roundrobin"
  }
} ' \
http://127.0.0.1:9991/edge/admin/routes/3004
```

3.2 访问网关进行验证

向网关发起访问请求：

```
curl -i http://127.0.0.1:9981/test_trace
```

预期响应输出（响应头中成功包含指定的追踪头信息）：

```
HTTP/1.1 200 OK
Content-Type: text/plain
Connection: keep-alive
X-Process-Time: 0.003
X-EDGE-Trace-Info: |X-Process-Time
...
```

(提示：由于配置了 `"show_resp_header": true`，响应头中额外附带了 `X-EDGE-Trace-Info`，其值为 `|X-Process-Time`，指示响应中的 `X-Process-Time` 头是通过 `tracing` 插件生成并写入的。而在转发到后端上游的请求包中，已附带了包含随机 UUID 的 `X-Trace-Id` 头)

traffic_limit_count 使用文档

插件概览

`traffic_limit_count` 插件是网关提供的限流插件，支持在特定的时间窗口内限制指定变量（如客户端 IP、请求 URI 等）的累计访问次数。支持本地单机限流（性能最高）以及结合 Redis 的分布式多节点共享限流。

配置详情

1. 插件静态属性 (Attr Config)

该属性用于在项目根目录的 `edge.env` 文件下的 `plugin_attr.traffic_limit_count` 块中为当前节点单独配置限制倍率。

字段名	类型	默认值	描述	设置示例
local_ratio	Number	无	针对 local 本地限流策略特有的限流倍率，必须大于 0。在 local 策略下，允许的请求数调整为 count * local_ratio 并向下取整，最小为 1。	1.5

2. 插件数据属性 (Metadata Config)

通过管理节点接口进行全局配置（路径：`/edge/admin/plugin_metadata/traffic_limit_count`）。

字段名	类型	默认值	描述	设置示例
key	String	\$	默认的限流键，支持网关内置变量（如 \${remote_addr}）。	"\${remote_addr}"
status	Integer	403	超出限制被拦截时，默认返回的 HTTP 状态码。	429
message	String	无	被限流拦截时，默认返回的响应消息。	"Too Many Requests"
bypass_missing_key	Boolean	true	当无法解析提取限流键时是否直接放行。	false
bypass_error_limit	Boolean	true	当执行限流逻辑出错（如 Redis 连接断开）时是否放行。	false
header_name	String	"X-EDGE-LimitCount-Remaining"	展示剩余次数的自定义响应头名称。	"X-RateLimit-Remaining"
show_resp_limit_header	Boolean	false	是否向响应中添加包含剩余请求次数的头部信息。	true
redis_dc	String	"TEST"	默认的 Redis 数据中心标识名称。	"IDC-1"
redis_conf	Object	无	Redis 连接详细配置。	见下文示例

说明: 关于 redis_conf 对象内部配置属性的详细说明：

- 主要子属性：
 - MODE：运行模式，Enum ["redis", "rediscluster", "proxycluster"]。
 - AUTH：连接鉴权密码。
 - DATABASE：数据库索引（单机模式），默认 0。
 - BACKLOG：连接池的积压（backlog）大小，默认 -1（表示不启用）。
 - TIMEOUT：网络超时时间（单位秒），默认 1。
 - MAX_CONNECT_NODES：每个 worker 最大连接的节点数，默认 2，配置为 0 时表示不限制。
 - MAX_IDLE_TIME：连接最大空闲时间（单位秒），默认 60。
 - CONNECTION_MAX_SIZE：连接池大小，默认 1024。
 - CLUSTER_{cluster_name}：各数据中心（集群）对应的 Redis 节点地址列表。其值为包含节点配置子数组的列表（格式如 [["ip/host", port, priority_or_group, weight, recovery_time_seconds], ...]）。例如："CLUSTER_TEST": [["127.0.0.1", 6379, 0, 10, 1000]]。
- 属性值的表示形式：除了 CLUSTER_{cluster_name} 之外，其他的 Redis 配置属性在 redis_conf 中均支持以下两种表示方式：
 - 对象映射方式（Object mapping）：以属性名作为顶层 Key，其值是一个以集群名称为 Key 的 JSON 对象。例如：

```
"MODE": { "TEST": "redis", "IDC-1": "rediscluster" }
```
 - 后缀字段名方式（Suffix keys）：在属性名后加上下划线和对应的集群名称作为顶层 Key。例如：

```
"MODE_TEST": "redis", "MODE_IDC-1": "rediscluster"
```
 - 优先级：对象映射方式的优先级高于后缀字段名方式。

3. 插件基础属性 (Config)

路由级属性，启用该插件时配置在路由下。

字段名	类型	默认值	描述	设置示例
status	Integer	同上	覆盖默认的限流 HTTP 状态码。	429
message	String	同上	覆盖默认的被拦截响应信息。	"Limits Exceeded"
bypass_missing_key	Boolean	同上	无法提取限流键时是否放行。	true
bypass_error_limit	Boolean	同上	限流计算出错时是否放行。	true
header_name	String	同上	覆盖默认的剩余次数响应头名称。	"X-RateLimit-Remaining"
show_resp_limit_header	Boolean	同上	是否在响应体中包含剩余请求次数。	true
redis_dc	String	同上	覆盖 Redis 数据中心名称。	"IDC-2"
redis_conf	Object	同上	覆盖 Redis 连接配置。	见下文示例
limits	Array	见下文	限流规则列表。	见下文示例
policy	String	"local"	限流存储策略, Enum ["local", "redis"]。	"local"
group	String	无	限流组名称，同一组内的不同路由会共享计数。	"api_rate_limiter"

说明: limits 内每个限制项包含以下三个字段:

- key: 限流的特征键，支持动态变量（如 \${uri}, \${host}），默认值为 \$。当设置为 \$ 时，表示限流针对的是当前路由或接口整体，即该路由下的所有请求共享同一个计数器；常用的变量可以参考 [Edge 变量列表](#)。
- count: 指定窗口内允许的最大请求次数，默认 3。
- window: 限流的时间窗口大小，单位：秒，默认 5。

调用与验证示例

1. 全局配置插件数据属性 (Metadata)

首先，我们可以通过管理节点接口配置全局的默认 Redis 连接信息（此处以本地 Redis 为例）：

注意：实际调用管理接口时，请求体需要先经过 SM4 加密，返回结果也需要进行解密，详见 [管理接口调用规范](#)。

```
curl -L -X PUT 'http://127.0.0.1:9990/edge/admin/plugin_metadata/traffic_limit_count' \  
-H 'X-API-KEY: API_KEY' \  
-H 'Content-Type: application/json' \  
-d '{  
  "key": "${remote_addr}",  
  "status": 403,  
  "message": "Too many requests.",  
  "redis_dc": "TEST",  
  "redis_conf": {  
    "MODE": "redis",  
    "TIMEOUT": 1.5,  
    "DATABASE": 0,  
    "CONNECTION_MAX_SIZE": 100,  
    "CLUSTER_TEST": [  
      ["127.0.0.1", 6379]  
    ]  
  }  
}'
```

2. 启用路由限流配置示例

然后配置指定路由以启用该限流规则（以 5 秒内限制最多 3 次访问为例，限流存储策略选用 `local`）：

```
curl -L -X PUT 'http://127.0.0.1:9990/edge/admin/routes/3001' \
-H 'X-API-KEY: API_KEY' \
-H 'Content-Type: application/json' \
-d '{
  "uri": "/test_limit",
  "plugins": {
    "traffic_limit_count": {
      "show_resp_limit_header": true,
      "policy": "local",
      "limits": [
        {
          "key": "${remote_addr}",
          "count": 3,
          "window": 5
        }
      ]
    }
  },
  "upstream": {
    "nodes": {
      "127.0.0.1:8111": 1
    },
    "type": "roundrobin"
  }
}
```

3. 请求验证

第一、二、三次访问（返回 **200 OK**，并带上剩余次数响应头）：

```
$ curl -i http://127.0.0.1:9980/test_limit
HTTP/1.1 200 OK
X-EDGE-LimitCount-Remaining: 2
...

$ curl -i http://127.0.0.1:9980/test_limit
HTTP/1.1 200 OK
X-EDGE-LimitCount-Remaining: 1
...

$ curl -i http://127.0.0.1:9980/test_limit
HTTP/1.1 200 OK
X-EDGE-LimitCount-Remaining: 0
...
```

第四次访问（超出 5 秒内 3 次的限制，拦截并返回 **403**）：

```
$ curl -i http://127.0.0.1:9980/test_limit
HTTP/1.1 403 Forbidden
...
Too many requests.
```

如果在 5 秒之后重新发起访问，由于时间窗口已滑动重置，请求会再次被成功受理并重置剩余计数。